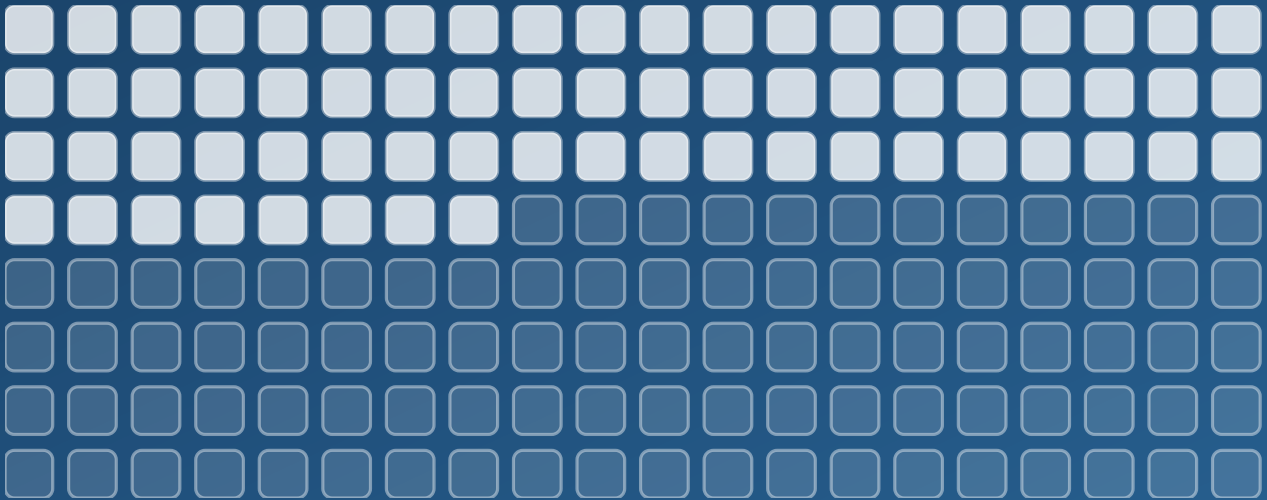


questions dabadaba 160

Every question a jury can ask you, and the first sentence to say when they do.



Each card gives you three things, in the order you need them: **the one line you say out loud**, the longer answer if they push you, and the place to point. Learn the first line of all 160. The rest you already know, because you built it.

Before you read a single question

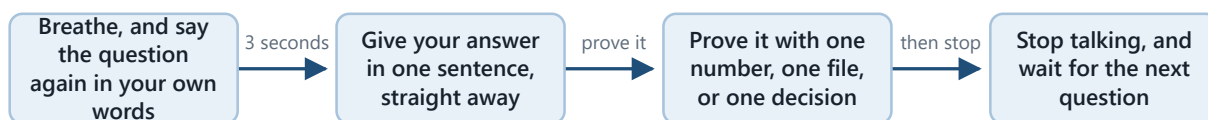
A defence is not a test of facts. The jury already has your report. What they want to know is whether you understand your own work well enough to defend the choices you made — including the choices that did not work out.

How each card works

- 1 **Say this** — the green block. Two or three sentences that take about fifteen seconds to say. This is the only part you learn by heart.
- 2 **If they push** — the longer answer. Real numbers, real files, and the price you agreed to pay for a choice. Do not learn this by heart. Read it enough times that you can say it in your own words.
- 3 **Point to** — the file or the part of the report. Opening your own report at the right page is the strongest thing you can do.
- 4 **Then they ask** — the question that usually comes next, so that it never surprises you.
- 5 **Do not say** — the answer that sounds fine but loses you marks.

The shape of a good answer

Every card in this book has the same four steps. When you are nervous you will forget the details long before you forget the shape, so learn the shape first. It works even for a question nobody prepared you for.



How to answer anything they ask — Saying the question again gives you three seconds to think, and it shows the jury you understood it. At box three they decide whether you really know your own work. Most marks are lost at box four, not box two: a good answer followed by two minutes of nervous extra talk invites a harder question you did not have to face.

Your opening, 90 seconds

"ST2i sells project steering to its clients, and steers its own projects with a spreadsheet. One file per project: twenty-one linked tabs, two thousand and eighty-eight formulas, and one hundred and sixty-nine cells that already return errors in the copy the company uses to decide whether a project makes money. That file is not a bad tool. It is the only written specification of the

company's financial method, and nobody else on the market has that method. No tool I compared supports a loaded cost rate per resource and per year, or a billing milestone as a percentage of a contract. So I did not invent a model. I moved theirs off a medium that had run out of room. PMS is one Spring Boot application, ten feature modules, one PostgreSQL database and an Angular interface. It computes the same earned-value indicators, out of one shared source instead of one file per project. And it adds the thing a spreadsheet cannot have: a developer can enter his workload and never see the margin. Ten sprints, four releases, ninety-seven backend tests, and a pipeline that starts the whole stack on a clean machine. It is delivered and deployable. It is not yet in production, and I will tell you exactly what stands between those two words."

When you really do not know

Say this, word for word. It keeps the jury trusting you, and it gives the conversation back to them:

"I do not know that, and I would rather say so than guess in front of you. What I can tell you is the nearest thing I did check, which is this. If you allow me, I will look up the exact point in the repository and come back to it before the end of the session."

Rules for the room

- 1 Open standing up, without the slides, and say the ninety-second statement from memory while looking at the jury. Only then move to the first figure. The room decides in the first minute whether you own this work.
- 2 Buy yourself three seconds by saying the question back in your own words: "If I understand you, you are asking whether the scope check protects endpoints outside /api/projects." Nobody reads that as stalling, it makes the examiner confirm, and it stops you answering a question nobody asked.
- 3 Answer with a number and a file, never with an adjective. Not "coverage is decent" but "forty-one percent overall, fifty-one to eighty-one on the eight modules the suite targets". An adjective invites them to dig. A number ends the exchange.
- 4 Say every limit yourself before they find it. Say it once, in one sentence, then put a boundary around it: "There is no frontend test. That is a gap, not a boundary I set on purpose. The fix is Karma on the guards and the interceptor first." Saying it costs one sentence. Being caught costs the whole topic.
- 5 When you are wrong, admit it in one sentence and give the right fact straight away: "You are right. Page 33 draws that as one-to-many, and it should be zero. Page 94 is the accurate figure." Never defend a fact you cannot check in the room.

- 6 To disagree, first agree with what they said, then move the ground: "That is exactly right for a product sold to clients. For one company's internal tool, the trade goes the other way, because..." Never open with "no".
- 7 Stop talking when the answer is finished. The silence after it belongs to the jury, not to you, and anything you add to fill it is material you did not prepare, and they will question it.
- 8 Hands: one on the clicker, the other resting or pointing at the figure you are naming. Never read a slide out loud. Say the claim, then point at the one thing on the figure that proves it. If a slide holds more than one idea, say which one you want them to look at.
- 9 Know four page numbers by heart: Table 1.3 build-versus-buy, Table 2.6 releases and sprints, Table 7.4 coverage, Figure 7.7 the pipeline run. Turning to the right page in two seconds does more for your credibility than any sentence.
- 10 Keep the repository open in a terminal behind you. If an examiner challenges a fact, offer to show the file instead of arguing: "That is KpiService, may I show you the eight injected repositories?" Offering evidence you really have turns an attack into a demonstration.

Part one — the 8 areas (100 questions)

1	Context, company and problem — why PMS exists at all	12 questions
2	Project Management: Scrum, Sprints and Releases	12 questions
3	Architecture and design decisions	13 questions
4	Authentication, authorization and data protection	13 questions
5	The financial engine — EVM indicators, billing and margin	13 questions
6	Database and data integrity	12 questions
7	The Angular application and the interface decisions	12 questions
8	Testing, CI/CD and Deployment — how he proves it works and how it ships	13 questions

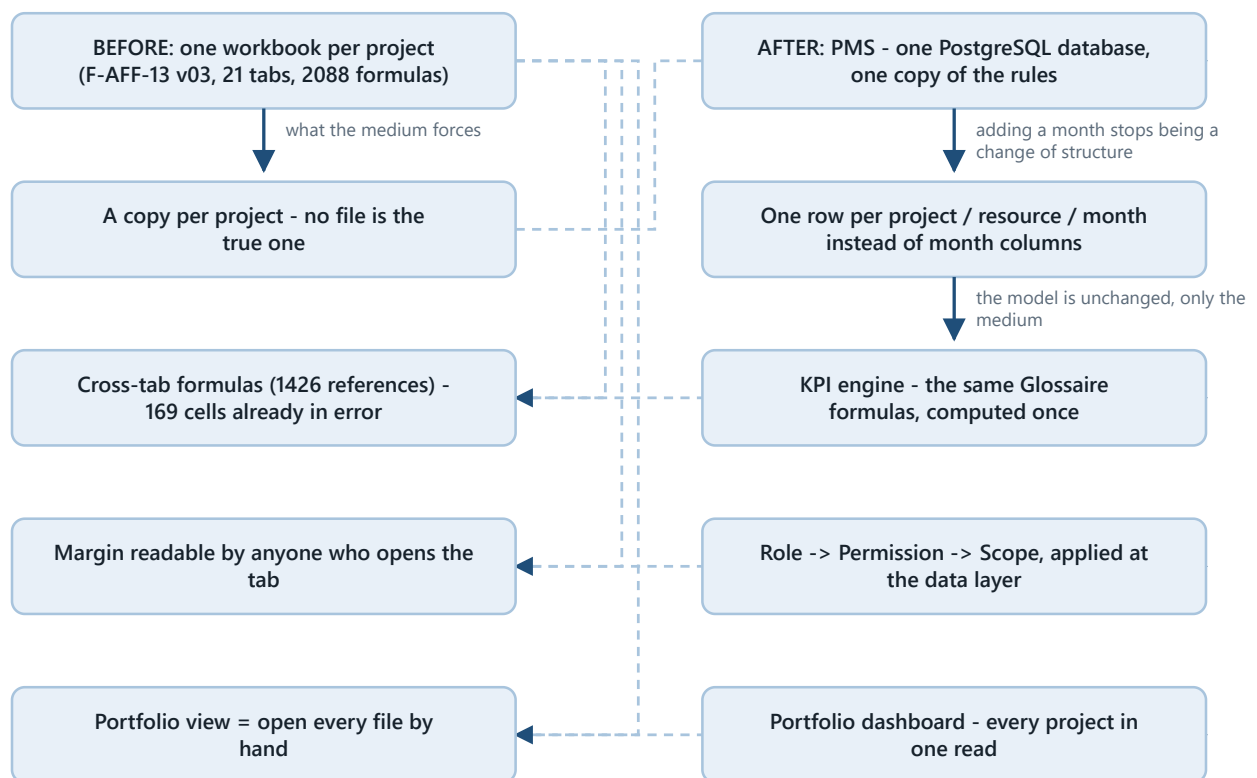
Part two — the list you were given (60 questions)

1	A. Problem, context and business analysis	10 questions
2	B. Requirements and business rules	10 questions

3	C. UML and analysis / design	10 questions
4	D. Architecture	10 questions
5	E. Security and dynamic RBAC	10 questions
6	F. The ten that can hurt you most	10 questions

Context, company and problem — why PMS exists at all

This examiner is testing two things. Did you understand a business before you wrote any code? And can you defend "build" against "buy" without sounding like a student who just wanted to write code? Every question here is really the same question: did the problem exist before you arrived, and did you measure it? The posture is calm ownership. You read a real file, you counted what was in it, and you can say exactly where your work stops.



The workbook world versus the PMS world — Drawing this proves what the whole project claims: the business model crosses from left to right without changing, and the only things PMS replaces are the four weak points of the medium. The files are spread out, they break easily, they have no access control, and nothing is brought together in one place.

1

Before we go into the system, tell us where you did this work. Who is ST2i?

EASY

SAY THIS

ST2i is a Tunisian consulting firm. It was created in the early nineties and it is part of STUDI Group. It works in information systems, geomatics, telecoms and engineering software, on projects in more than fifteen countries. My work is about one of its five activity lines: development and integration of solutions.

IF THEY PUSH

The head office is in Tunis, in the Lac II district. The firm is organised by activity line under one general management, and that organisation matters for my subject. Below the activity line there is no fixed hierarchy. ST2i builds a team for each project, with people from several fields, taken from a shared pool: geomatics experts, developers and technicians. So the same person is committed to several projects at the same time, and no single project manager sees all of that person's free time. That is what makes steering workload a real daily problem at ST2i, and not an abstract one. There is a second point that is special to this subject. ST2i steers IT projects for its clients, so the company that sells project steering is itself the user of the platform. That is why I did not go and collect needs from users who do not exist. I read them from the tool the company already uses every day.

POINT TO report/chap_01.tex – §1.1 Host organisation · Figure: functional organisation of ST2i (label fig:st2i-org, img/st2i_organigramme.png) · report/chap_01.tex – §1.2.1 The assigned subject

THEN THEY ASK

Which activity line actually pays for this platform? — Development and integration of solutions; the platform steers the money of that line.

DO NOT SAY

Do not list the countries and do not turn this into a company brochure. The jury wants the one fact about the organisation that creates your problem: teams are built with people taken from several lines.

2

You say the company already had a tool. What exactly was this Excel file?

EASY

SAY THIS

It is the project review sheet, reference F-AFF-13 version 03. It has twenty-one linked tabs and two thousand and eighty-eight formulas, and there is one copy per project. It holds cost rates, planned and actual workload, billing milestones, missions, the governance registers and a complete earned-value engine.

IF THEY PUSH

The tabs fall into six families: identification, costs, billing, missions, steering and governance. They are not all equally dense. The Statistiques sheet alone has more than six hundred formulas, and it is the real computation engine. Actual workload has more than five hundred. The workload plan has about four hundred. One sheet made the whole analysis possible: the Glossaire, which defines every indicator, its unit and the way it is calculated. That sheet is why two project managers who read the same margin figure agree on what it measures. It is also what gave me an acceptance criterion I could test against. Nobody at ST2i ever wrote the business rules down as a specification document. This file is the specification. That is why my decision log puts business reality taken from the Excel above the written SRS and above the project plan in the source-of-truth order.

POINT TO report/chap_01.tex – §1.3.1, Table 1.1 (functional structure of the workbook) · docs/DEV_Fiche_revue_projet-Real.xlsx · docs/BUSINESS_ANALYSIS.md – §10 formula coverage map · DECISIONS.md – ADR-011 source-of-truth priority

THEN THEY ASK

Do you still have the file? — Yes, it is in the repository under docs/, and the report shows eleven of its sheets as captures with the private data hidden.

DO NOT SAY

Do not call it "just a spreadsheet". You are about to argue that it is a valid specification, so treat it as one from your very first sentence.

3

You use the word steering all the way through. What do you mean by it?

EASY

SAY THIS

Steering means answering one question at any moment: will this project still make money, and will I know in time to act? It is not task management. It is comparing what was sold, what has been spent, what has been invoiced, and what is left to do.

IF THEY PUSH

Those four figures move on their own, apart from each other. A contract fixes a price and a number of days sold. The cost depends on who really does the work, and at what loaded rate. The invoicing follows a milestone calendar that has nothing to do with either of them. And the remaining effort is an estimate that changes every month. If you lose sight of one of them, the news that a project lost money arrives after it has closed, and that is the one moment when nothing can be done. In this company, steering means the indicators the workbook computes: earned value as a percentage, estimate to complete, estimate at completion as actual cost plus ETC, the net margin as budget minus EAC, the accrued income as production revenue minus what has been invoiced, and the drift in man-days between sold, consumed and remaining. The Dashboard sheet is the one the project manager and the director really look at in the monthly review, and it answers exactly one question: is this project still making money?

POINT TO report/introduction.tex · report/chap_01.tex – Dashboard and Statistiques sheets, and the Glossaire formulas · docs/BUSINESS_ANALYSIS.md – KPI formula catalogue

THEN THEY ASK

So it is accounting? — No. Accounting writes down what happened; steering estimates the end state from the current month, and that is why the central indicator is the estimate at completion.

DO NOT SAY

Do not define steering as following tasks and dates. That is scheduling, and it is the one thing the company did not ask for.

4

Who are the users of this platform?

EASY

SAY THIS

Four roles. An administrator, who manages users, roles, permissions and cost rates. A director, who owns budgets and the portfolio. A project manager, who is the main daily user. And a developer, who enters their own actual workload and nothing else.

IF THEY PUSH

In the report each actor is defined as much by what they may not do as by what they may do. The developer cannot reach any financial data at all: not budget, not margin, not EAC, not billing. The administrator runs the platform, but has no financial or operational data about projects unless a capability is given on purpose. The director sees every project. The project manager sees only the projects where they are the active manager. This is enforced by permission and scope together. The permission says whether the action is allowed at all. The scope says on which rows. And the scope filter is applied at the repository boundary, not in each controller. There is also one difference the jury may not expect: a user and a resource are two different things. A user is an identity that logs in. A resource is a person who uses up man-days and carries a cost. An outside consultant has a cost in the system without ever having an account.

POINT TO report/chap_02.tex – Table 2.1, PMS actors and their boundaries · DECISIONS.md – ADR-021 authorization scope enforcement · DECISIONS.md – ADR-022 separation of User and Resource

THEN THEY ASK

Is a person always exactly one of these? — No; the model keeps User and Resource apart, so a subcontractor can be planned and costed with no login, and a developer is both.

DO NOT SAY

Do not say "everyone in the company". The interesting part of the answer is where the line is drawn, not the list.

5

If the workbook breaks as easily as you say, why did the company keep it for so long?

MEDIUM

SAY THIS

Because it works, and because it is theirs. It holds rules that are written down nowhere else. It costs nothing, and it changes the same day the rule changes. The company did not keep it because it did not know better. It kept it because no tool on the market has that model.

IF THEY PUSH

The workbook holds knowledge built up over years, in the form of constants. The loaded cost rate is the annual loaded cost times one point seven, divided by the working days of the year, and that number is twenty-two in 2024 and twenty in 2025. The risk provision is five percent of the budget. Milestone percentages may not go over one hundred. Hours become man-days when you divide by eight. A spreadsheet lets the person who owns a rule change it in a minute with no deployment, and that is a real advantage. The cost only shows up at scale. One copy per project means there is no file anyone can point to as the true one. Seeing the portfolio means opening every file one after the other. And nothing stops a developer from opening the margin tab. The medium also fails quietly: the file I was handed has one hundred and sixty-nine cells in error, hash-REF and hash-VALUE, sitting inside a document used to decide whether a project makes money. So the way to put it is not that Excel is bad. The model is good, and the medium ran out of room.

POINT TO report/chap_01.tex – §1.3.5 Critique of the existing system · docs/BUSINESS_ANALYSIS.md – §10, category F9: 169 broken formulas · DECISIONS.md – ADR-008 business parameters are configurable data

THEN THEY ASK

So what finally forced the change? — Nothing forced it. The company put it on its PFE book as subject 6, which is a decision to industrialise before it hurts instead of after.

DO NOT SAY

Do not laugh at the spreadsheet or at the people who built it. A jury member has probably built one, and the file is your specification.

SAY THIS

I compared those four against nine requirements taken from the workbook, not from a generic checklist. None of them handles a loaded cost rate per resource and per year. None of them handles a billing milestone as a percentage of a contract, or revenue earned but not yet invoiced. Those lines are the heart of this company's model.

IF THEY PUSH

Microsoft Project is the reference for scheduling, and it does put a cost on a resource. But that cost is a rate on the resource, not a rate on the resource for a given year, and it has no idea of a billing milestone as a percentage of a contract. It plans work; it does not steer a contract. Jira is richer than anything I built when it comes to agile planning, but it carries no financial model of its own. Costing needs a marketplace application like Tempo, and even then the model is built on time and not on the contract. Its permission schemes work on issues and on projects, not on single financial fields, so the rule that a developer must never see the margin is hard to say in it. Odoo Project is the closest, because invoicing and timesheets are in the same suite, but its cost model is one hourly cost per employee. monday.com works out formulas board by board, and they do not join up across boards, which brings back the exact fragility we are leaving behind. The table should be read in both directions. The tools are strong exactly where the workbook is weak, and they are missing exactly where it is strong. That difference is the reason to build.

POINT TO report/chap_01.tex – §1.3.3 and §1.3.4, Table 1.3 comparison of existing tools · Figure: the four market solutions examined (label fig:market-logos) · report/chap_01.tex – paragraph following Table 1.3

THEN THEY ASK

Are those criteria not chosen so that you win? — They are chosen from the workbook, and that is why PMS loses the one row where the workbook is weak: scheduling.

DO NOT SAY

Do not say the tools are bad. They are better than PMS at what they do, and saying so is what makes the rest of the comparison believable.

7

You admit Odoo comes closest. Why not just customise Odoo?

MEDIUM

SAY THIS

Because customising Odoo means one of two things: accept the way Odoo computes the margin, or write it again inside a full ERP. Odoo costs one hourly rate per employee. ST2i costs a loaded daily rate per resource and per year. That change reaches the core of the costing module, not a plug-in.

IF THEY PUSH

Once you touch that, everything else follows: the workload plan as a monthly matrix per resource, the estimate to complete as a running remainder, the estimate at completion, the accrued income, and three separate margins (sold, actual, and at EAC). You would be building the whole financial chain again inside somebody else's data model, and then carrying that fork through every ERP upgrade. For the internal steering need of one company, a focused application with one schema and twenty-eight versioned migrations is a smaller commitment than owning a fork of an ERP. There is one case where I would answer differently. If ST2i decided to move invoicing, purchasing and payroll into the same system, the weight of the ERP would win and PMS would be the wrong shape. That is a real condition, and it is not the situation today.

POINT TO `report/chap_01.tex` – §1.3.3, Odoo Project paragraph · `DECISIONS.md` – ADR-016 modular monolith, microservices rejected as over-engineering · `backend/src/main/resources/db/migration` – V1 to V28

THEN THEY ASK

Isn't that just a developer's reflex to build? — The test is the cost of change: in PMS a rule change is one parameter row, and in a forked ERP it is a merge conflict at every upgrade.

DO NOT SAY

Do not claim Odoo cannot do it. It can, but at a cost. The cost is the argument, and saying anything else is easy to contradict.

8

Where does the scope of this project stop? What did you decide not to build?

MEDIUM

SAY THIS

The scope is the workbook plus what the subject asked for. I did not build scheduling. I did not build an Excel importer. I did not build document export. The automatic import of time tracking is designed in the data model but not built, and I say so in the conclusion.

IF THEY PUSH

What is in: projects with their contractual baseline, resources and yearly cost rates, the workload plan and the actual workload, billing milestones and payments, missions, the five governance registers (risks, deliverables, stakeholders, changes and amendments), the internal quote, the earned-value engine, the dashboards, dynamic authorisation, and an agile module with a backlog and sprints. What is out, and on purpose: critical-path scheduling; importing the existing workbooks, which is why there is no Apache POI dependency anywhere in the backend; and producing the monthly review document in the format the company already sends round, which is the last manual step of the review. On the time-tracking side, the workload table carries a source discriminator with the values MANUAL and KIMAI, and a reconciliation rule of one accepted value per cell. So the shape is there. The import pipeline is not. One more boundary: the internal quote ships as an empty structure, because I was allowed to reproduce the company's structure and never its data.

POINT TO DECISIONS.md – ADR-003 manual and KIMAI sources, ADR-004 full scope plus five modules, ADR-027 agile module · report/conclusion.tex – perspectives · backend/src/main/resources/db/migration/V27__schema_agile.sql

THEN THEY ASK

Why design the KIMAI path and not build it? — Because the shape of the data decides whether it can ever be added: a second source needs a unique key per source and an accepted flag, and adding that later would mean migrating every workload row.

DO NOT SAY

Do not present everything you did not build as "we ran out of time". Separate the choices from the gaps, or the jury will think all of it was a gap.

9

In Table 1.3 you mark scheduling and critical path as absent for PMS. You built the tool yourself. Why is your own product the only one that fails a line?

HARD

SAY THIS

Because the table checks tools against what the workbook needs, and the workbook has no scheduling either. Not one of its twenty-one sheets holds a task dependency or a critical path. That mark is not a hole I forgot to fill. It is the honest result of the rule I chose.

IF THEY PUSH

I took the criteria in that table straight from the workbook. A tool that replaces the workbook is only useful if it can carry the business rules the workbook already holds. Scheduling is in the table because it is the one need where the market tools are truly better. If I had removed that row, the comparison would have been unfair in my favour. The ST2i subject is also very clear. It asks for agile planning with sprints and a backlog, and PMS has that as its own module with its own migration. The subject never talks about scheduling, about dependencies, or about a critical path. So I have two separate sources, the workbook and the subject I was given, and neither one asks for it. If ST2i needs critical-path analysis later, the right answer is still Microsoft Project used next to PMS. It is not a weaker copy built inside

PMS. I would draw that line the same way again. Scheduling only pays off when you plan task by task, and ST2i plans at the level of a resource and a month.

POINT TO report/chap_01.tex – Table 1.3, last row, and the paragraph that follows it ·
report/chap_01.tex – §1.2.1, wording of subject 6 of the ST2i PFE Book 2025-2026 ·
DECISIONS.md – ADR-027 agile planning module ·
backend/src/main/resources/db/migration/V27__schema_agile.sql

THEN THEY ASK

So the table is biased in your favour? — It is biased towards the workbook, and that is the point. The same bias is what makes PMS lose this row.

DO NOT SAY

Do not say sorry and do not promise to add Gantt charts. You are defending this line, not giving it up. If you give it up, the whole build-versus-buy argument falls apart.

10

You studied the workbook of one project, and the screenshots are blanked out. How can one single file define what a whole company needs?

HARD

SAY THIS

It is the company's standard review form, not one manager's private file. Its reference is F-AFF-13, version 03, and it is a controlled document used once per project. What I hid is the data: the client, the names, the rates, the amounts. What I kept is the structure, the headings and every formula, and the formulas are what I built.

IF THEY PUSH

Two sheets are the strongest proof that this is a company tool and not a personal one. The Glossaire defines every indicator, its unit and how it is computed. Nobody writes that unless several people on several projects have to read the same number the same way. The Paramètres sheet does the same for the words used to judge things: the probability scale, the severity scale with its thresholds, and the treatment options. That way a risk marked major means the same thing on every project. I also checked coverage instead of trusting one reading. All two thousand and eighty-eight formulas fall into nine groups: eight groups of live business logic that I mapped to calculations in the platform, and one group of broken cells that the platform simply computes again. Nothing was left unexplained. The limit is real and I will say it plainly. I checked against one copy of the template, so a rule used only on a different kind of project could have slipped past me. That is exactly why every constant is a parameter you can configure, and not a number written inside the code. An unknown variant becomes a change of data, not a rewrite.

POINT TO report/chap_01.tex – §1.3.2, workbook reference F-AFF-13 v03 and the redaction paragraph ·
Figures: Glossaire sheet (label fig:xl-glossary) and Paramètres sheet (label fig:xl-parameters) · docs/BUSINESS_ANALYSIS.md – §10, coverage verification of the 2 088 formulas · DECISIONS.md – ADR-008

THEN THEY ASK

Which missing rule would worry you most? — A separate cost model for subcontractors. The Resource entity already exists without a login, so that would be a rate row, not a redesign.

DO NOT SAY

Do not claim the workbook is the company's whole process. Claim only what you can prove: it is the company's controlled project review form, and you covered every formula in it.

11

Be honest with us. If PMS disappeared tomorrow, what would ST2i really lose?

HARD

SAY THIS

Honestly, today, not much in daily work. The platform is finished and ready to deploy, but it does not run at ST2i and no real project data has been moved into it. What the company would lose is a checked, repeatable build of its own rules, and the analysis behind it.

IF THEY PUSH

I want to be exact rather than flattering. What runs is a stack of three containers. It starts from a clean database and fills it with demonstration data, and the users and projects inside are made up, not ST2i's. The internal quote module ships as an empty structure on purpose, because I was allowed to reproduce the company's structure and never its figures. So what I deliver is the model and the mechanism, not a live system with history. If it vanished, ST2i would carry on with the workbook exactly as before. What it would lose is the work of writing out two thousand and eighty-eight formulas clearly, including the one hundred and sixty-nine that were already broken in the file they use. It would also lose a platform where the margin of a project is no longer visible to whoever opens the tab. Turning that into a real gain is a rollout decision: load the real cost rates, import one closed project, then compare its margin with the workbook figure. That is a company decision, not a technical one. The platform already reproduces that computation, and that is how each increment was accepted during the project.

POINT TO PROJECT_STATUS.md – stack verified running 2026-08-20, seeded demonstration data ·
README.md – demo accounts warning, see docs/DEPLOYMENT.md §5 · DECISIONS.md – ADR-026
containerized deployment · report/chap_01.tex – §1.4 Problem statement

THEN THEY ASK

What is the first step to make it real? — Load the true cost rates and one finished project, then compare the computed margin with the workbook's. That one check is what would win the company's trust.

DO NOT SAY

Do not say the company uses it now, or that it is in production. The industrial supervisor is in the room, and one question would show it is not true.

SAY THIS

That is a fair objection and I will not talk it away. One developer wrote this, and today one developer knows it. What I could control was making changes cheap and the knowledge easy to pass on, and most of my design choices are about exactly that.

IF THEY PUSH

Three concrete things. First, the business rules are data, not code. The overhead coefficient, the working days per year, the risk provision and the risk scales all sit in a parameter store. So the rules most likely to change need no developer at all. Second, authorisation is data too. Moving a responsibility is a permission grant, not a new release. My decision log records a real disagreement between the company's own documents about who may assign developers, and one database row settled it instead of a code change. Third, the project writes itself down as it is built. There are twenty-seven architecture decision records, and each one gives the context, the decision and the consequence. The schema is owned by twenty-eight versioned migrations instead of guesswork. And a pipeline builds, tests and really runs the full stack on every push. So the next person inherits a working machine, not a story. The honest limit is that documents are not a person. If ST2i adopts this, it has to own it. But the other road was paying licences for a tool that still cannot express the company's financial model, which means paying every year and keeping the gap for ever. I would take that trade again for an internal tool of this size, and I would not take it for a product sold to clients.

POINT TO DECISIONS.md – ADR-001, ADR-005, ADR-008, ADR-019, ADR-026 (27 decision records) ·
backend/src/main/resources/db/migration – V1 to V28 · .github/workflows/ci.yml ·
docs/ARCHITECTURE.md and docs/DEPLOYMENT.md

THEN THEY ASK

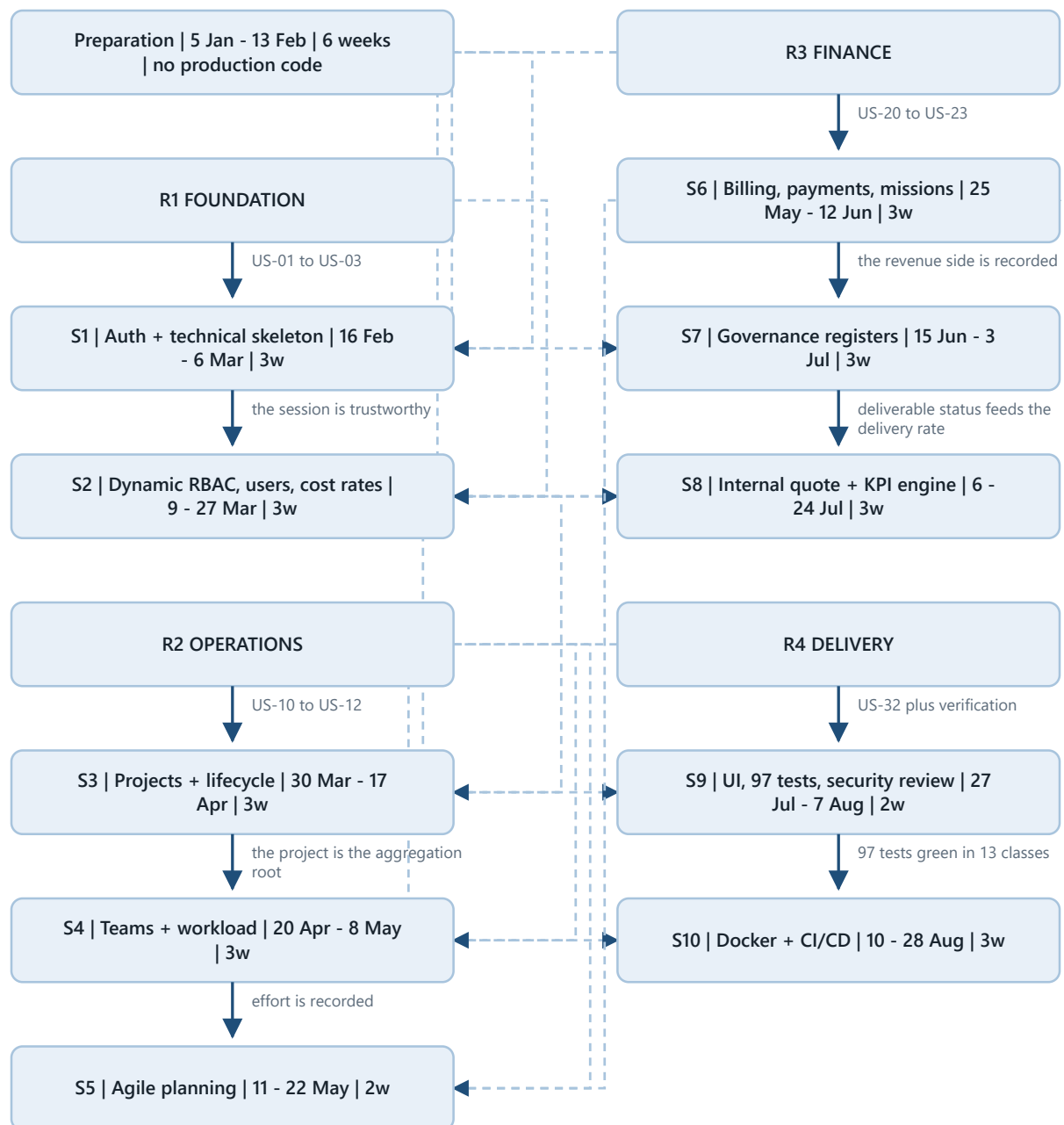
How fast could a successor take it over? — One command brings the whole stack up, and DECISIONS.md explains why each choice was made. That is the honest measure of what I could do about the risk.

DO NOT SAY

Do not answer that the code is clean and speaks for itself. That invites a jury member to open a file at random and test the claim right there.

Project Management: Scrum, Sprints and Releases

A jury that asks about method is not testing whether you can recite the Scrum guide. They are testing two things. Do you know which parts of the framework really carry the value? And can you say out loud what you did not do? So start with the reduced form, named plainly, and then defend it. A candidate who says "there was no daily scrum, and here is what replaced it" scores higher than one who claims a ceremony he never held. Every soft spot in this area has an honest one-sentence answer followed by a boundary. Use that shape, and never slide into defending yourself.



13

Why did you choose Scrum for a project where you were the only developer?

EASY

SAY THIS

I chose Scrum for its idea of working in small steps, not for its ceremonies. The PMS business model has many parts that depend on each other. So each block had to be checked against the Excel workbook before the next one was built on top of it. Building it in one single pass would have hidden every integration error until the very end.

IF THEY PUSH

The indicator engine reads workload, billing, cost rates and the internal quote. If I had built all of that in one pass, the first time I would have learned that my cost figures were wrong is the day the whole thing was finished. Going forward in small steps meant that each functional block was checked against the reference workbook before anything depended on it. I kept the parts that create that loop: sprints with a fixed length, an ordered product backlog, a sprint backlog, one increment per sprint that can be demonstrated, and acceptance by the Product Owner. I dropped the parts that need a team. Chapter 1 says which parts were kept and which were not, in a section written for exactly this question, because hiding the reduction would have been worse than the reduction itself.

POINT TO report/chap_01.tex – section 'Adaptation to this project' (label sec:scrum-adapt) · report/chap_02.tex – section 2.4 'Project management with Scrum' · Report Figure 1.15, 'The Scrum cycle', page 24

THEN THEY ASK

Why not a waterfall plan, since you had the full Excel workbook from day one? — Because the workbook was never written as a specification. I discovered rules while I was implementing them, and migrations like V12__fix_rbac_matrix.sql and V17__fix_project_date_check.sql exist only because I was looking at a running increment instead of a plan.

DO NOT SAY

Do not say you used Scrum 'because it is the standard agile method'. That answer says you followed a habit. It does not say you made a choice.

14

Who played the Product Owner and who played the Scrum Master?

EASY

SAY THIS

The Product Owner was Mr. Ahmed Amine Amiri, my industrial supervisor at ST2i. He decided the priorities and accepted or refused each increment against what the company expected. I held the Scrum Master role and the developer role myself. My academic supervisor, Mrs. Zrelli Siwar, followed the engineering quality and the documentation quality, outside the Scrum roles.

IF THEY PUSH

The Product Owner did three concrete things. He gave me the Excel workbook as the real specification. He decided the change of scope in July, when the company handed over its full costing method. And he accepted each increment by comparing the platform's figures with the workbook's figures. As Scrum Master I planned each sprint, kept the backlog in order, and solved my own blockers. I will name the weakness in that. When one person holds both the process role and the development role, nobody challenges his decisions about the process. What absorbed that risk was not a person. It was the workbook: an outside acceptance criterion that I could not argue with. Table 2.4 of the report sets out the four roles and who held each one.

POINT TO Report Table 2.4, 'The Scrum roles and who held them', page 35 · report/chap_02.tex – subsection 'Project team and roles' · report/chap_01.tex – section sec:scrum-adapt

THEN THEY ASK

How often did you actually see the Product Owner? — Not on a fixed weekly rhythm. I saw him at the end of each increment and whenever a decision was needed. That irregular rhythm is a real weakness of a one-person Scrum, and the workbook is what absorbed it.

DO NOT SAY

Do not suggest that the supervisor was available every day, or that you held stand-ups with him. If they ask later for meeting records, you will have contradicted yourself.

15

In your report you speak of releases and of sprints. What is the difference?

EASY

SAY THIS

A sprint is a fixed period of two or three weeks that produces one increment. A release is a group of sprints that brings the product to a level of maturity you could hand over as a whole. I have ten sprints, grouped into four releases, and each release is one chapter of the report.

IF THEY PUSH

Release 1, sprints 1 and 2, makes the platform usable and secure: sign-in and the dynamic authorization model. Release 2, sprints 3 to 5, opens day-to-day steering: projects, teams, workload, agile planning. Release 3, sprints 6 to 8, delivers the business value the project was ordered for: billing, missions, governance registers, the internal quote and the indicator engine. Release 4, sprints 9 and 10, adds no business feature: it makes the interface consistent, checks the authorization model with tests, and makes the deployment repeatable. I want to be exact about one word. 'Releasable' here means it can be demonstrated and deployed. It does not mean it is installed in production at ST2i. The report does not claim a production installation, and neither do I. Sprint 10 is what made 'deployable' literally true, with a Docker stack of three containers and a pipeline that starts it from nothing on a machine that has never seen the project.

POINT TO Report Table 2.6, 'Distribution of the backlog across releases and sprints', page 39 ·
Report Table 7.7, 'From the reference workbook to the delivered platform', page 128 ·
Report Figure 7.7, 'Run 40 of the pipeline on GitHub Actions', page 127

THEN THEY ASK

So the client never used it? — Not in production. But I checked that the whole stack runs from one end to the other: the browser goes through nginx, then to the API, then to the database. I checked this on my own machine and also on an empty build runner. An empty machine is the state you start from when you hand a project over.

DO NOT SAY

Do not say, and do not even hint, that the platform is 'in production at ST2i'. It is not in production, and the report never says it is.

16

How did you build your product backlog, and how did you decide the order of the items?

MEDIUM

SAY THIS

The backlog has thirty-two user stories, US-01 to US-32. Each story is one action. They come from four places: the company's Excel workbook, the requirements specification, the use case specification and the business rules document. I put them in order by technical dependency, not by business value.

IF THEY PUSH

The written sources gave fifty-three functional requirements, twenty use cases and sixty-five business rules. The workbook gave the rest: twenty-one sheets that are all linked together, and more than two thousand formulas. That workbook is where the real financial model lives. I mark a story High when later increments depend on it. That is why sign-in and dynamic authorization are High, and the stakeholder register is Low. Nothing can be shown before sign-in works. And nothing can compute a margin before the cost rates exist. I know this is not the textbook rule. In the book, the Product Owner puts the backlog in order by business value. But I was one developer with a fixed end date. If I had worked value first, I would have built a financial engine with nothing to read. That is a line I would draw the same way again, because here the dependency graph is not a taste or a preference. It is a fact of the domain.

POINT TO Report Table 2.5, 'Product backlog of the PMS platform', page 36 · report/chap_02.tex – subsection 'Product backlog' · docs/BUSINESS_ANALYSIS.md

THEN THEY ASK

Did you estimate the stories in story points? — No. I sized them by calendar time. I was one developer with no past history, so a velocity figure would have been a number I made up.

DO NOT SAY

Do not say the Product Owner put the backlog in order by value. The priority column comes from dependency, and the report says that clearly.

17

How did you decide to have ten sprints, and how did you choose how long each one is?

MEDIUM

SAY THIS

The internship lasted thirty-four weeks, from 5 January to 30 August. Six weeks went to the preparation phase, which produced no code. The other twenty-eight weeks became ten sprints. Eight of them are three weeks long and two are two weeks long, because those two covered less ground.

IF THEY PUSH

The arithmetic is six weeks of preparation, plus eight sprints of three weeks, plus two sprints of two weeks, which makes thirty-four. The two short ones are Sprint 5, agile planning, which is three stories and two entities, and Sprint 9, interface consolidation and testing, which adds no business feature. Three weeks was my default box. It is long enough to deliver a whole module from one end to the other: migration, entity, service, controller, screen and tests. It is also short enough that a wrong direction costs three weeks and not three months. I did not use velocity. I was one developer with no past history, so a velocity number would have been fiction dressed up as a measurement. The honest reading is this: a fixed box with changing content is a weaker tool than a changing box with fixed capacity. But it was the only tool I could set up without inventing data.

POINT TO Report Table 2.7, 'Schedule of the preparation phase and the ten sprints', page 40 · report/chap_02.tex – subsection 'Release planning' · report/chap_04.tex – section 'Preparation phase'

THEN THEY ASK

Why is the writing of the report not a phase in that schedule? — Because I wrote it all the way through. I drafted each chapter while I was doing the work it describes. I only reviewed the whole thing at the end. Chapter 2 says that clearly.

DO NOT SAY

Do not invent a velocity, a burndown chart or a story-point total. None of them exist in the report, and a jury member who has seen real Scrum will ask to see the chart.

SAY THIS

It was not a demo in front of a room. I checked each increment against the Excel workbook. I made the platform do a computation the workbook already did, then I compared the two figures. When they were different, I treated the workbook as right and I fixed my code.

IF THEY PUSH

Three examples, one per release. In Sprint 2 the review was one single check: I removed a permission from a role, and I watched the menu entry disappear and the API call get refused. Then I gave the permission back, and I watched both come back. Nothing was recompiled and nothing was redeployed in between. In Sprint 6 I typed the billing progress sheet again, milestone by milestone, and I compared the invoiced totals line by line. That included the rule that the percentages cannot go over one hundred, which the spreadsheet writes down but cannot enforce. In Sprint 8 I entered one project on the platform and I read the indicators twice: once in the workbook and once on the dashboard. This is stricter than a demo in one way and weaker in another. It proves the figures down to the last decimal. It proves nothing at all about whether a user finds the screen comfortable. That second gap is what Sprint 9 was for.

POINT TO `report/chap_04.tex` – 'Sprint review' subsections of Sprints 1 and 2 · `report/chap_06.tex` – 'Sprint review' subsections of Sprints 6, 7 and 8 · `report/chap_01.tex` – section `sec:scrum-adapt`

THEN THEY ASK

Did the Product Owner attend each of those comparisons? — He accepted the increments. The comparison itself was work I did on my own. The record of it is in the decision documents and in the migrations, not in meeting minutes.

DO NOT SAY

Do not describe a formal demo meeting with an audience for every sprint. The workbook comparison is the true answer and the stronger one. A made-up meeting is easy to knock down.

19

You had no daily scrum and no retrospective. Can you really call this Scrum?

MEDIUM

SAY THIS

It is Scrum in a smaller form, and I say that in the report instead of hiding it. A team of one person cannot sync with itself, and it cannot hold a retrospective in the group sense. What I kept is the part that carries the value: iterations with a fixed time box, a backlog in order, and an increment checked at the end of every box.

IF THEY PUSH

What I kept: sprint planning, a sprint backlog, one increment per sprint, acceptance by the Product Owner, and constant backlog refinement. What I dropped: the daily scrum, the group retrospective, velocity and burndown. The real question is whether the loop of inspect and adapt survived, and it did, because the workbook was my inspection tool. It also produced real changes, and that is the test of whether a loop works or is only decoration. Migration V12 rebuilt the whole role-permission matrix after an audit found the administrator holding twenty-two permissions instead of five, and a developer who could reach financial data he must never see. Migration V20 removed permissions that were declared and never used. Neither of these came from a plan. They came from looking at something that was already running, and that is exactly what the framework is for.

POINT TO report/chap_01.tex – section 'Adaptation to this project' ·
 backend/src/main/resources/db/migration/V12__fix_rbac_matrix.sql ·
 backend/src/main/resources/db/migration/V20__descope_unused_permissions.sql ·
 PROJECT_TODO.md – RBAC audit entry of 2026-06-27

THEN THEY ASK

Then why not Kanban, which fits one person better? — Because I needed fixed stopping points, so I could compare my figures with the workbook. Kanban would have given me flow, but no natural moment to stop and compare.

DO NOT SAY

Do not say you held daily scrums with yourself, or that you wrote a retrospective after each sprint. Inventing a ceremony you never did is the one answer that loses this question.

20

Requirements changed during your internship. How did you handle that?

MEDIUM

SAY THIS

The biggest change came on 5 July, when the company gave me its real costing method. It added three things: an internal quote, the earned-value indicators, and a cost rate that changes by year. It arrived exactly at a sprint boundary, so it set the content of Sprint 8 instead of breaking a sprint that was already running.

IF THEY PUSH

Sprint 8 ran from 6 to 24 July, the day after the method was handed to me. That is the one place where Scrum lets the scope change. It is also why Sprint 8 is called 'internal quote and indicator engine' and not something narrower. The change turned into three migrations. V21 makes the cost rate depend on the year the effort is charged to, and it falls back to the base rate. V22 adds the earned-value indicators: earned value, delivery rate, man-day drift, production revenue, estimate at completion, and actual margin against sold margin. V23 adds the internal quote with its two-pass computation. There was one part of that change I refused. The company's own quote values are commercially sensitive, so the platform ships the structure and seeds no company figure at all. Saying yes to the structure and no to the data is written down as a decision. It is not left there as something I forgot. A second, smaller change came in August. A gap check against the ST2i subject showed that agile planning was the only stated requirement the platform did not cover, so I built a module for it, and I kept it deliberately small.

POINT TO PROJECT_STATUS.md – entry of 2026-07-05, 'Méthode F-AFF-13 complète' ·
backend/src/main/resources/db/migration/V21__tcc_annuel.sql, V22__evm_indicators.sql,
V23__devis_interne.sql · DECISIONS.md – ADR-027 · report/chap_06.tex – Sprint 8

THEN THEY ASK

Did that change push your end date? — No. It replaced the planned content of Sprint 8 instead of being added on top of it, and the schedule in Table 2.7 did not move.

DO NOT SAY

Do not present the July change as something you saw coming. It was new information from the client, and saying so is what makes the whole iterative argument believable.

21

What did you cut from the scope, and why?

HARD

SAY THIS

Four things: the automatic import from the KIMAI time-tracking tool, the export of the monthly review documents, the frontend unit tests, and the burndown and velocity charts in the agile module. Every cut was on purpose and every one is written down. Three of them I would do again. The frontend tests I would not.

IF THEY PUSH

KIMAI is designed but not built. The actual-workload table has a source column with the values MANUAL and KIMAI, one row per source. A partial unique index makes sure there is exactly one accepted value per project, per resource and per month. So the second source can plug in later with no change to the schema. I cut the import itself because I had no real KIMAI instance to test against, and an integration you cannot test is worse than none. The export of the review documents is named in my conclusion as the next step to do: the platform computes the indicators and shows them, but it does not yet produce the document the company sends around. I refused burndown and velocity in the agile module on principle. An honest burndown needs a daily reading of the work that is left, and without those readings the curve would be something I rebuilt afterwards, so the module shows a segmented

bar in man-days instead. The frontend tests are the real gap, not a boundary I chose. There is not one spec file in the frontend. The backend has ninety-seven tests across thirteen classes, but that covers the layer the user never touches.

POINT TO DECISIONS.md — ADR-003 (KIMAI) and ADR-027 (no burndown, no velocity) · docs/ENHANCEMENTS.md — 'frontend engineering depth: zero tests' · Report Table 7.4, 'Tests and coverage per module', page 120 · report/conclusion.tex — perspectives

THEN THEY ASK

With one more month, which would you do first? — The frontend tests, because the backend already refuses what it must refuse, and the layer with no tests is the one the users actually touch.

DO NOT SAY

Do not describe the missing frontend tests as a design boundary you chose. It is a gap, and calling it a choice is the kind of answer a jury remembers badly.

22

You say there was a sprint review for each of the ten sprints. Can you show me the minutes of those reviews?

HARD

SAY THIS

No, and I will not pretend otherwise. I did not keep formal minutes. What you read in the report is written again from my own dated working records. The practice really happened, but what you read is a rebuild of it, not a transcript.

IF THEY PUSH

The records that do exist are dated and cannot be rewritten afterwards. PROJECT_STATUS.md was updated at the end of every phase with what had been delivered and the test count at that moment. DECISIONS.md holds twenty-seven architecture decision records, and each one gives its context, its decision, its reason and its consequences. PROJECT_TODO.md holds the status of each task and the validation gates. And the twenty-eight Flyway migrations are numbered in build order: V1 auth, V3 user and resource, V5 project, V6 team, V7 workload, V8 kpi, V9 billing, V10 missions, V11 governance. That is the sprint order, and migrations cannot be renumbered once they have run against a database. What I should have kept is one page per review, with a date, saying what was shown, what was accepted and what was refused. It would have cost twenty minutes a sprint and it would have made this question unnecessary. I would do it next time. What I will not do is show you working notes and call them minutes.

POINT TO PROJECT_STATUS.md · DECISIONS.md — ADR-001 to ADR-027 · backend/src/main/resources/db/migration/ — 28 migrations, V1 to V28 · PROJECT_TODO.md

THEN THEY ASK

Then how do I know the sprint dates in Table 2.7 are real? — They are the internship calendar and the plan I worked to. The proof that the increments were built in that order is the migration numbering, and I would rather show you that than a document I could have written last week.

DO NOT SAY

Do not offer the git history as a sprint-by-sprint trace. The repository was created partway through the internship, so the commit dates do not match the sprint calendar, and anyone with access to the repo can check that in thirty seconds.

23

Your database migrations are numbered in build order, and the agile planning tables are V27, after the financial ones. Was Sprint 5 really when you built that module?

HARD

SAY THIS

No. I wrote that module in August, not in May. I found the gap when I checked the platform against the ST2i subject: agile planning was the one stated requirement the platform did not cover, so I built it then. The decision record carries its real date, 22 August.

IF THEY PUSH

The report groups the sprints by what they do together, and agile planning belongs with teams and workload in operational steering, which is why it sits in Release 2. That is a choice about presentation, and the report should have said that one module was delivered out of the calendar order. I own that. What I would defend is the module itself. The fact that it was built late is exactly why it is small: two entities only, Sprint and BacklogItem; estimates in man-days and not in story points, because every other figure in the platform is in man-days; no epics, no ceremonies modelled, no burndown. A module built in the last weeks of an internship is exactly the module you must not let grow into a second product, and the decision record argues that line by line.

POINT TO backend/src/main/resources/db/migration/V27__schema_agile.sql and
V28__backlog_item_assignee.sql · DECISIONS.md – ADR-027, dated 2026-08-22 ·
report/chap_05.tex – Sprint 5 · Report Table 2.6, page 39

THEN THEY ASK

Does anything else sit out of calendar order? — The corrections do. V12 rebuilds the role-permission matrix and V20 removes unused permissions. Both come after the modules they touch, because they came out of an audit and not out of a sprint.

DO NOT SAY

Do not brush the migration number aside as a small technical detail. It is the build order, the jury can read it, and pretending otherwise turns a small honest admission into a problem of trust.

SAY THIS

Three things. I would keep one dated page for every sprint review. I would write the frontend tests while I write the screens, instead of leaving them to the end. And I would size each sprint by what I could really finish, instead of fixing three weeks and hoping the content would fit.

IF THEY PUSH

The review record is the cheapest of the three, and it is the one whose absence you can feel while you read my report. Half a page saying what was shown, what was accepted and what was refused, signed by the Product Owner, would have turned a rebuilt story into evidence. The frontend tests are the serious one. The backend has ninety-seven tests, and across the eight modules the suite aims at, it reaches between fifty-one and eighty-one per cent of the lines, with the modules that carry money and permissions at the top of that range. Counted over everything, including the demonstration data seeder, it is forty-one per cent of lines and twenty-four per cent of branches. The frontend has none, and that is the layer the users touch. On sizing: after three sprints I had enough real history to size by content, but I kept the three-week box out of habit and not out of measurement. One thing I would not change: I checked every increment against the workbook, not against my own reading of the specification. That one discipline is why the platform reproduces the company's real financial method and not something close to it.

POINT TO Report Table 7.4, 'Tests and coverage per module, from the run of 13 September 2026', page 120 · Report Figure 7.2, the JaCoCo coverage report, page 121 · docs/ENHANCEMENTS.md – engineering improvement backlog · report/conclusion.tex

THEN THEY ASK

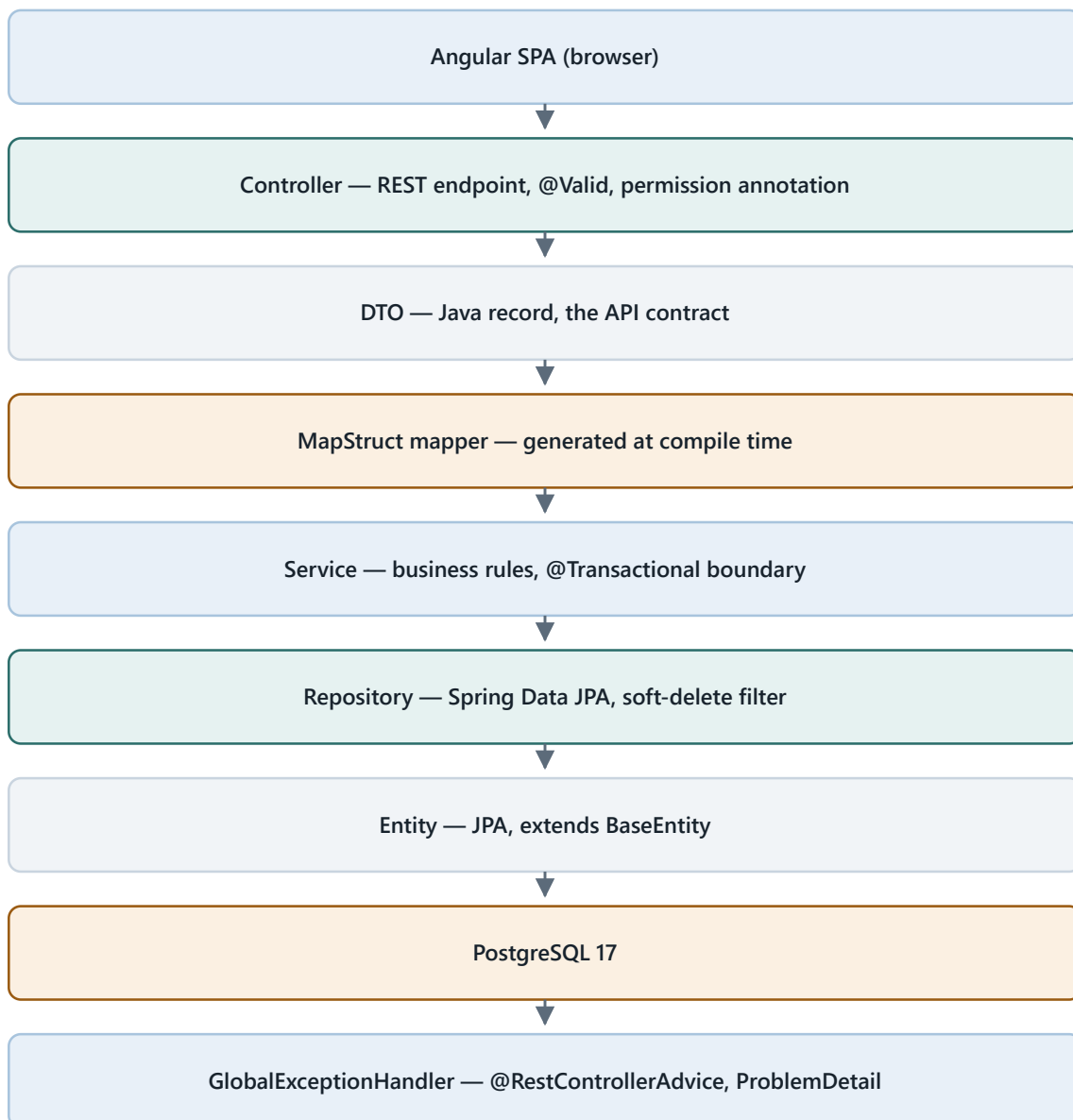
If you could keep only one of the three, which one? — The review record, because it costs almost nothing and it is the one gap that makes an examiner doubt the rest.

DO NOT SAY

Do not answer 'nothing, I would do it the same way'. A candidate who cannot name one weak point in his own process is telling the jury he never looked at it.

Architecture and design decisions

The jury is not testing whether you know what a layered architecture is. They assume you do. They are testing whether you made these choices yourself, or copied them from a tutorial. So every answer must name the constraint that forced the decision, and the price you paid for it. Speak as the person who drew the line. Own the two places where the code does not match the rule you wrote. And never defend a choice you would not make again. A calm "here is where it leaks, and here is why I left it" scores higher than a clean story they can break open.



The four layers of a PMS module and the one-way call rule — Drawing this shows the two rules that make the layering real. Calls go downward, one layer at a time. And the two crossed arrows are the only things you have to remember: a controller never reaches a repository, and an entity never reaches the browser.

25

In a few sentences, tell me how your application is built.

EASY

SAY THIS

PMS is a three-tier modular monolith. An Angular single-page application talks over REST to one Spring Boot deployable, and that talks to one PostgreSQL 17 database. Inside that single deployable, the code is cut into ten feature modules plus a shared one.

IF THEY PUSH

Figure 3.1 on page 42 shows the three tiers and the protocol that separates each one. Figure 3.2 on page 43 shows the modules themselves: auth, user, project, team, workload, billing, mission, governance, agile, kpi, and shared. That is 192 Java files under com.pms. Every feature module has the same folders inside it: controller, service, repository, entity, dto, mapper. So a developer who has read one module can read any of them. The Angular side is cut the same way: thirteen folders under src/app/features, with core and shared next to them. For deployment there are three containers in docker-compose: the database, the Spring Boot backend, and an nginx container that serves the Angular bundle and passes the API calls through.

POINT TO report/chap_03.tex, section 3.1, page 42 • Figure 3.1 (three tiers, p. 42) and Figure 3.2 (modules and dependencies, p. 43) • backend/src/main/java/com/pms/ • docs/uml/02-package.puml • docker-compose.yml

THEN THEY ASK

So the frontend is a separate deployable, which means it is not really a monolith? — The frontend container serves a static bundle with no server-side logic. So there is still exactly one process that holds the business rules, and one transaction boundary. The monolith claim is about the backend.

DO NOT SAY

Do not open with the list of tools ("Spring Boot 3.3.6, Angular 21, PostgreSQL 17..."). They asked about architecture, not tools, and starting with versions makes you sound like you are reading the CV of the project instead of describing its shape.

26

What are the layers inside a module, and what stops a developer from jumping over them?

EASY

SAY THIS

Four layers: Controller, Service, Repository, Entity. The rule is that calls only go downward, and only one layer at a time. A controller never injects a repository, and an entity never comes back out through the controller.

IF THEY PUSH

The controller is the REST entry point. It checks the request body with Jakarta Validation and it carries the authorization annotation. The service holds the business rules and is the transaction boundary. There are 121 `@Transactional` annotations in the backend, and all of them are on services. The repository is Spring Data JPA, and it is the only thing that talks to the database. The entity holds the invariants and extends `BaseEntity`, which gives it the id, the four audit columns and the soft-delete flag. Figure 3.3 on page 44 follows one request through those four layers and marks the exact spot where a DTO becomes an entity. What holds the rule today is code review and the package layout, not a test. Across 18 controllers, one imports a repository and one imports an entity, and both are in the project module. That is a real gap, and I call it a gap.

POINT TO report/chap_03.tex, section 3.1.2 "Layered organisation", p. 44 · Figure 3.3 – "The four layers of a module, and the boundary at which a DTO becomes an entity", p. 44 ·
backend/src/main/java/com/pms/shared/entity/BaseEntity.java ·
backend/src/main/java/com/pms/project/controller/ProjectController.java · DECISIONS.md,
ADR-016

THEN THEY ASK

Then the rule is a habit, not a real constraint? — Correct. The fix is a ten-line ArchUnit test that breaks the build when a controller imports a repository. I did not write it, and it is the first thing I would add.

DO NOT SAY

Do not say the layering is "enforced" or "guaranteed". One grep by a curious jury member finds the two controllers that break it, and then the whole answer looks doubtful instead of just those two files.

27

Why did you build a REST API instead of GraphQL, or SOAP, or something else?

EASY

SAY THIS

The client is one Angular application that I wrote myself, and the resources map cleanly onto the business objects: a project, its milestones, its workload, its indicators. REST gives me HTTP status codes, caching and browser tools for free. GraphQL solves a problem I do not have.

IF THEY PUSH

GraphQL is worth its extra complexity when many different clients need different shapes of the same data, or when pulling too much data over a mobile network costs you money. I have one client and one network: an internal browser application. What GraphQL would have cost me is a schema layer, a resolver layer, and the N+1 query problem coming in through the back door, on a data model that is already relational and financial. REST also gave me the error contract almost for free. Spring's `ProblemDetail` is built on HTTP status codes, so 404, 403 and 422 already mean something. The API is 108 endpoints across 18 controllers: 32 GET, 24 POST, 18 PUT, 15 PATCH, 19 DELETE. They are documented by springdoc, and every controller carries an `@Tag`. The URL shape shows who owns

what: `/api/projects/{projectId}/kpi` says the KPI belongs to the project, and that is also what makes the one central scope check possible.

POINT TO `backend/src/main/java/com/pms/kpi/controller/KpiController.java`
(`@RequestMapping("/api/projects/{projectId}/kpi")`) ·
`backend/src/main/java/com/pms/shared/config/OpenApiConfig.java` ·
`backend/src/main/java/com/pms/shared/config/ProjectScopeInterceptor.java` ·
`report/chap_03.tex`, section 3.1.2

THEN THEY ASK

What about versioning? What happens when you need to change the shape of a response? —

There is no version part in the URL today, because there is one client and I deploy it at the same time as the backend. The day a second client exists, I would add `/api/v2` instead of breaking the contract where it stands.

DO NOT SAY

Do not say "REST is the industry standard". That is not a reason, it is a shrug, and a jury member who likes GraphQL will take it as an invitation.

28

Your services return DTOs, not entities. Why add that extra object?

EASY

SAY THIS

Three reasons. The entity is a database shape and the API is a contract, and I do not want renaming a column to break the frontend. An entity is lazy-loaded, so turning it into JSON fires queries I never asked for. And most important of all, the entity holds fields the caller must never see.

IF THEY PUSH

The third reason is the one that really counts in this project. A developer on PMS has no right to financial data. That is business rule BR-050. So if a Project entity went straight to JSON, `initialBudget` and `margeNetteVendue` would land in a developer's browser, even if the screen never showed them. The DTO is where I decide what leaves the building. There are 38 entities and 52 DTOs in the backend, which is the shape you expect: some entities have a list DTO and a detail DTO, and some internal entities have none at all. On the technical side it also removes the lazy-loading trap. Project holds two `@ManyToOne(fetch = LAZY)` links to User, and turning the entity into JSON outside a transaction either throws or quietly fires extra queries. Every DTO in the project is a Java record, so it cannot be changed, and it cannot be used by accident as a way to write back into the domain.

POINT TO `backend/src/main/java/com/pms/project/entity/Project.java` (`director` and `chefProjet` are LAZY `@ManyToOne`) · `backend/src/main/java/com/pms/kpi/dto/KpiResponse.java` · `DECISIONS.md`, ADR-016 ("entities never cross the API boundary") and ADR-021 (BR-050) ·
`report/chap_03.tex`, section 3.1.2

THEN THEY ASK

Is that not a lot of repeated code for 38 entities? — It would be if I wrote the conversions by hand. MapStruct generates them at compile time, so it costs me an interface of a few lines per feature, and nothing at runtime.

DO NOT SAY

Do not start with "separation of concerns" or "best practice". Start with the developer who would have seen the project margin, because that is a result the jury can picture.

29

What does MapStruct actually do, and why not just write the mapping by hand?

MEDIUM

SAY THIS

MapStruct reads my mapper interface at compile time and writes the Java class that copies entity fields into DTO fields. It generates code, it does not use reflection, so it costs nothing at runtime. I chose it because a mapping mistake then breaks the build instead of the demo.

IF THEY PUSH

Each feature declares an interface marked `@Mapper(componentModel = "spring")`. The annotation processor writes the implementation into `target/generated-sources` and registers it as a Spring bean. There are 18 of them, one per feature area, on MapStruct 1.5.5.Final. The `@Mapping` lines are where the real work shows. In `SnapshotKpiMapper` I flatten `project.id` and `project.code` onto the response, and I take `margeVenduePct` from the project instead of the snapshot. Hand-written mappers would have worked, but with 38 entities and 52 DTOs they become the most boring code in the repository and the code you edit most often. That is exactly where a quietly missing field hides. `ModelMapper` was the other option, and I said no to it because it matches by reflection at runtime. The mistake then shows up as a null in production instead of a red build. There is one honest gap. ADR-018 says I would set `unmappedTargetPolicy = ERROR`, so that a target field with nothing mapped onto it breaks the compile. That setting is not in the code. Today such a field is only a warning, so the safety net is thinner than the decision record says.

POINT TO DECISIONS.md, ADR-018 · `backend/pom.xml` (`mapstruct 1.5.5.Final` plus `mapstruct-processor` in `annotationProcessorPaths`) · `backend/src/main/java/com/pms/kpi/mapper/SnapshotKpiMapper.java` · 18 mapper interfaces under `backend/src/main/java/com/pms/*/mapper/`

THEN THEY ASK

So how do you know today that no field is quietly dropped? — I do not know it in a systematic way. I know it for the paths the 97 tests go through. Turning on the strict policy is a one-line change to every `@Mapper` annotation, and I would make it before the next release.

DO NOT SAY

Do not present MapStruct as "faster than reflection" as your main point. The speed does not matter at this scale. The failure at compile time is the real argument, and reaching for performance makes it

sound like you are quoting a benchmark you never ran.

30

A management platform like this is usually presented as microservices. Why a monolith?

MEDIUM

SAY THIS

Because computing one financial indicator reads workload, billing, cost rates and the internal quote all in one go. Splitting those across services would mean distributed transactions, for a platform used inside one company by a few dozen people. I took the boundaries without the distribution.

IF THEY PUSH

Chapter 3 gives three arguments and I stand behind all three. First, transactional consistency: KpiService reads seven repositories across five modules to produce one figure, inside one read-only transaction. As microservices that becomes seven network calls. Each one can fail, or can return a figure from a different moment, on data where a wrong margin is the whole point of failure. Second, deployment scale: this is one Tunisian IT company steering its own projects. The running cost of a service mesh, distributed tracing and one database per service would never be paid back by the load. Third, reversibility: the modules and their dependency graph, which has no cycles, are drawn as if I might pull one out one day. So the option is kept, not spent. And there is a constraint I should say plainly. I was one developer with eight months, and microservices are an architecture for organisations. They pay off when several teams need to deploy on their own. There were no several teams.

POINT TO report/chap_03.tex, section 3.1.1, p. 42 (the three arguments) · DECISIONS.md, ADR-016 ("Microservices rejected as over-engineering") · backend/src/main/java/com/pms/kpi/service/KpiService.java (7 repositories, 5 modules, one transaction) · Figure 3.2, p. 43

THEN THEY ASK

And if ST2i grows to 500 projects and the dashboard gets slow? — That is a database problem before it is an architecture problem. The snapshot table, the composite (project, month) index and a read replica all come before splitting a service, because the load is reads, not writes.

DO NOT SAY

Do not say microservices are "too complicated" or hint that they are bad engineering. Say they solve a problem this project does not have. Otherwise you sound like you ran away from them instead of ruling them out.

SAY THIS

You get an RFC 7807 problem document: a JSON body with a status, a type URI and a detail a human can read. It is made in one place, a `@RestControllerAdvice`, so no controller writes its own error response. A broken business rule is a 422.

IF THEY PUSH

`GlobalExceptionHandler` has twelve `@ExceptionHandler` methods, and every one of them returns Spring's `ProblemDetail`, which is the built-in RFC 7807 type in Spring Boot 3. The mapping is on purpose. `NotFoundException` gives 404 with type `/errors/not-found`. `BusinessRuleException` gives 422 with `/errors/business-rule`. `AccessDeniedException` gives 403. A bean-validation failure gives 400, with every field error joined into the detail. A `DataIntegrityViolationException` gives 409. The last handler catches `Exception` itself, logs the stack trace on the server and returns a plain 500 that says only "internal server error". The stack trace never leaves the machine. The type URI matters, because it gives the Angular interceptor something stable to branch on, instead of reading an English message. The honest limit is that the detail strings are written in French, and there is no list of field-level errors. So the frontend cannot highlight one input from the payload. It shows the message. For an internal tool with one client I accepted that, and it is the kind of thing I would change the day a second consumer shows up.

POINT TO `backend/src/main/java/com/pms/shared/exception/GlobalExceptionHandler.java` (12 handlers, all returning `ProblemDetail`) ·
`backend/src/main/java/com/pms/shared/exception/BusinessRuleException.java` ·
`backend/src/main/java/com/pms/shared/exception/NotFoundException.java` ·
`backend/src/main/java/com/pms/shared/config/ProjectScopeInterceptor.java` (throws `AccessDeniedException` -> 403)

THEN THEY ASK

Why 422 and not 400 for a business rule? — 400 means I could not read or check the request. 422 means I understood it perfectly and the domain refuses it, for example closing a project that still has an unpaid milestone. The client should show those two cases differently.

DO NOT SAY

Do not say "I handle errors globally" and then stop. Name `ProblemDetail` and one real status-to-exception pair, or it sounds like you added a try/catch and called it architecture.

SAY THIS

Because the indicator engine is the end of the chain. It takes in workload, billing, cost rates and the internal quote, and nothing takes anything from it. That keeps the graph free of cycles, and that is what makes the modules real instead of decoration.

IF THEY PUSH

The direction of the dependencies is a design rule, not an accident. user depends on auth. project depends on user. team depends on project and user. workload depends on team. And billing, mission, governance and agile each depend on project. kpi sits at the bottom, receiving from workload, billing, user and project, and it exports nothing back to them. If billing ever imported kpi, I would have a cycle, and the two modules would really be one. No cycles also keeps the reasoning about transactions simple. A write in billing never causes a write in kpi, because the indicators are computed again when someone reads them, not pushed when something changes. That is the hybrid engine of Figure 3.4: computed on read, so a dashboard is never out of date, and frozen into a monthly snapshot, so the history for the evolution curves survives. Today, having no cycles is kept by discipline and by the diagram. There is no build step that would fail if I added a cycle.

POINT TO docs/uml/02-package.puml · Figure 3.2 – "Functional modules and their main dependencies", p. 43 · Figure 3.4 – "The hybrid indicator engine", p. 45 · backend/src/main/java/com/pms/kpi/service/KpiService.java (computeLive, createSnapshot) · DECISIONS.md, ADR-015

THEN THEY ASK

What made you choose recompute-on-read instead of recomputing when the data changes? —

A push model needs every write path in five modules to remember to fire the recompute, and one forgotten call shows a wrong margin. Reading is the expensive but correct direction, and the monthly snapshot takes the cost for the historical view.

DO NOT SAY

Do not say the graph is "verified" or "enforced by the build". It is drawn and respected, not checked, and there is no reason to oversell it when the design itself is sound.

SAY THIS

Not fully, and I will be precise about it. Twenty files import a repository that belongs to another module. Three of those are demo data seeders, so seventeen are production classes. The rule as written in ADR-016 is the rule I would keep. The code falls short of it in one specific way that repeats.

IF THEY PUSH

The pattern is almost always the same. A service brings in `ProjectRepository` so it can load the parent project and check it before it attaches a child to it. Eleven services do exactly that: `BillingService`, `RiskService`, `LivableService`, `MissionService`, `SprintService` and the others. That is a weak break of the rule, because the only thing they take from the other module is a lookup by id. The strong break is `KpiService`. It brings in seven repositories from five modules: `workload`, `billing`, `governance`, `user` and `project`. Against that, the whole backend has only two real service-to-service calls. `KpiService` calls `DevisInterneService`, and `ProjectService` calls `JalonService`. So the direction is right. Nothing reaches into another module's controller or into its DTO internals, and no module writes through a repository that is not its own. But I read data from other modules directly, instead of going through a service interface they publish. I would draw the rule the same way again, because it is what makes it possible to pull a module out later. What I would change is this: I would enforce it with a test from sprint one, instead of trusting myself to remember it in sprint eight, when the KPI engine needed six sources at the same time and the deadline was real.

POINT TO DECISIONS.md, ADR-016, "Module boundary rules", points 1 and 2 ·
 backend/src/main/java/com/pms/kpi/service/KpiService.java lines 49-58 (the seven repositories) · backend/src/main/java/com/pms/project/service/ProjectService.java (imports `billing.service.JalonService`) ·
 backend/src/main/java/com/pms/billing/service/JalonService.java (imports `project.repository.ProjectRepository`)

THEN THEY ASK

So what would you actually write instead in `KpiService`? — Each source module would publish a read-only query service that returns a DTO. For example `WorkloadQueryService.actualDaysByProjectAndMonth`. Then `KpiService` would hold six service references instead of six repositories. It is a mechanical change of about a day.

DO NOT SAY

Do not answer "yes, the code respects it". A jury member with the repository open finds `KpiService` in thirty seconds. And if you are caught saying more than is true about one rule, every other claim in the defence falls under suspicion.

SAY THIS

Project to Sprint is one to zero-or-many, and Project to BacklogItem is the same. Both are compositions, because a sprint and a backlog item mean nothing without their project, and both die with it. The zero is there on purpose: a project starts with no sprints and an empty backlog.

IF THEY PUSH

The rule I used is simple. A composition means the part cannot live without the whole. So for each relation I only had to ask one question: does the child survive if the project is deleted? A sprint does not. A backlog item does not. So both get the filled diamond. The code says the same thing. In `Sprint.java` and `BacklogItem.java` the `project_id` column is `nullable = false`, so the database refuses an orphan. The multiplicity that really carries information is the third relation: Sprint zero-or-one to BacklogItem zero-or-many. That one is an aggregation, not a composition. A backlog item with a null sprint is a valid state. It is an item still in the product backlog, not yet committed to an iteration. That is exactly why `sprint_id` can be null while `project_id` cannot. So deleting a sprint sends its items back to the backlog instead of destroying them, and that note is printed on the diagram. I should add one thing about the global diagram on page 33. It draws those child ends as *1.. rather than 0..*, which is too generous. The detailed agile diagram on page 94 is the accurate one.

POINT TO Figure 5.13 – "Class diagram – agile planning", p. 94 · Figure 2.2 – "Domain model – principal classes and their associations", p. 33 · `docs/figures/class_specs.py`, module `docstring` (the three relation rules) and `CLASS_AGILE` relations · `docs/uml/16-class-agile.puml` · `backend/src/main/java/com/pms/agile/entity/BacklogItem.java` (`project_id nullable=false`, `sprint_id nullable`)

THEN THEY ASK

In the Java code, does Project hold a list of sprints? — No. The link goes one way only, from the child up. Sprint and BacklogItem each hold a `@ManyToOne` back to Project. A collection on Project going the other way would load a list with no limit that I never need, and I would have to keep it in step by hand.

DO NOT SAY

Do not dodge the *1..** on the global diagram if they have it in front of them. Saying it yourself and pointing at the accurate figure costs you one sentence. Being corrected on it costs you the whole relation.

Why is KpiSnapshot in the class diagram at all? It looks like a cache, and a cache does not belong in a domain model.

HARD

SAY THIS

Because it is not a cache. It is a monthly review that a project manager creates, signs off and compares. You can delete a cache and build it again exactly the same. You cannot do that with a snapshot, because it holds a number no computation can produce again: the earned value the project manager typed in that month.

IF THEY PUSH

That is the whole difference. SnapshotKpi stores evPct, the earned-value percentage of progress, and that figure is human judgement. It is the manager saying how much of the contract is really delivered. Everything else in the snapshot comes from it. caProduction is the budget times that percentage, fae is production revenue minus what has been invoiced, and margeActuelle follows. Delete the snapshot and you cannot build it again, because the judgement is gone. There is a second reason. The live figures are deliberately not stored. Margins that come from the internal quote are computed again on every read and never saved, so no commercially sensitive value is copied into the database. So the snapshot is not a fast copy of the live view. It is a different thing: the frozen official state of one month. That is what the evolution curve in the statistics sheet of the Excel workbook was. I modelled the table with sixteen business columns, including the highlights text the manager writes. That text alone settles the question: nothing can compute a comment again.

POINT TO backend/src/main/java/com/pms/kpi/entity/SnapshotKpi.java (evPct with the comment "saisi par le CdP", faitsMarquants) · docs/figures/class_specs.py, CLASS_KPI comment: "a frozen monthly review is something the project manager creates, consults and compares, not a technical artefact of the code" · Figure 6.9 – "Class diagram – workload and indicator engine", p. 110 · Figure 3.4 and report chap_03.tex section 3.2.3 ("amounts derived from the internal quote are always computed at read time and never stored") · backend/src/main/java/com/pms/kpi/service/KpiService.java (computeLive vs createSnapshot)

THEN THEY ASK

Then why does the snapshot also store eac and marge, which you could compute again? —

Because the snapshot has to be readable later as one month that holds together, and computing those again from today's cost rates would quietly rewrite history the moment a rate changes.

DO NOT SAY

Do not defend it as "a performance optimisation for the dashboard". You would be accepting their premise, that it is infrastructure. The honest answer is stronger: it holds data that exists nowhere else.

36

You say the monolith is reversible. In concrete terms, what would you have to change to pull the billing module out into its own service?

HARD

SAY THIS

Three things, in this order: its foreign key to projects, its shared transaction, and the way other modules read it. Nothing in the controller or the service would change, and that is the point of the boundary.

IF THEY PUSH

First, the data. `billing_milestones`, `payments` and `avenants` (contract changes) all carry `project_id` as a real foreign key into projects, and that constraint cannot cross a database boundary. It becomes a plain identifier that the billing service has to check by calling the project service, and I lose the database guarantee that it points at something real. Second, the transaction. `JalonService` brings in `ProjectRepository` to load the parent inside the same transaction. That becomes a remote call, so an operation that is atomic today becomes two steps that can half-fail. I would then need either a compensating action, or a window of inconsistency I accept. Third, the readers. `KpiService` reads `JalonFacturationRepository` directly for billed totals. That call has to become an HTTP call or a message to the extracted service, and suddenly the indicator computation can fail because the network failed. So the honest cost is not the code. It is those three guarantees. I put the mechanical work at a few days and the consequences at quite a lot more. That is exactly why the modular monolith was the right place to start. I keep the option open, and I pay for it only if someone ever uses it.

POINT TO `backend/src/main/java/com/pms/billing/` (controller, service, repository, entity, dto, mapper – the module is already self-contained) · `backend/src/main/java/com/pms/billing/service/JalonService.java` (injects `project.repository.ProjectRepository`) · `backend/src/main/java/com/pms/kpi/service/KpiService.java` (injects `billing.repository.JalonFacturationRepository`) · DECISIONS.md, ADR-016, "clean path to extract services later" · report/chap_03.tex, section 3.1.1, "Reversibility"

THEN THEY ASK

Would you extract billing first, if you had to extract something? — No. Billing is tied very closely to the project for the margin computation. The natural first one to pull out is auth, because it depends on nothing except user, and it is the only module whose scaling needs are really different.

DO NOT SAY

Do not answer "nothing, the modules are already independent". That claims a cleanliness the code does not have, and the jury's next question will be the foreign key, which you should raise yourself.

SAY THIS

The cost is ceremony. Changing one field means touching an entity, a DTO, a mapper and a controller, and with 38 entities that adds up. And I have to be straight with you on the second half. I know the boundaries hold because I read the code, not because I measured it. There is no architecture test in this repository.

IF THEY PUSH

The list of what is missing is short, and I would rather say it than have it found. There is no ArchUnit and no Spring Modulith, so nothing makes the build fail when a controller imports a repository, or when a module reaches into another one. Both of those have happened, as I said earlier. There are no domain events either. ADR-016 says I would prefer events over synchronous chains when one change has to reach many places, and there are zero `ApplicationEventPublisher` usages in the backend, because the hybrid engine computes on read and never needed that. And the test suite is 97 tests over 13 classes, which measures behaviour, not structure: 41 per cent of lines and 24 per cent of branches overall, with the money and permission modules at the top of that range and the demo seeder pulling the total down. I would pay the ceremony cost again, because the DTO layer is what keeps a project margin out of a developer's browser. The measurement gap I would not pay again. An architecture rule that no test enforces lasts exactly as long as the developer who wrote it remembers it, and I am the proof of that.

POINT TO Table of tests and coverage per module, report/chap_07.tex, sprint 9 (run of 13 September 2026) · Figure of the JaCoCo report, chap_07.tex (161 classes, 41% lines, 24% branches) · backend/pom.xml (no archunit, no spring-modulith dependency) · DECISIONS.md, ADR-016 rule 4 (events preferred) vs zero `ApplicationEventPublisher` in backend/src/main/java · backend/src/test/java/com/pms/ (13 test classes, 97 tests)

THEN THEY ASK

If you had one more week, what would you spend it on? — An ArchUnit test class with four rules: controllers may not import repositories, no module may import another module's repository, entities may not appear in controller signatures, and no package cycles. It turns every architectural claim in chapter 3 into something the build checks on every commit.

DO NOT SAY

Do not finish on an apology, or on "given the time constraints". Say what the gap is, say what it would take to close it, and stop. A gap you name with a fix you name sounds like command of the project. A gap wrapped in excuses sounds like a weak point you were hoping they would not reach.

Authentication, authorization and data protection

This examiner does not want to know whether you can say what JWT means. He wants to know whether the security is real. Is there a check on the server? Can you point at the line? Do you know what your own design does not cover? So answer with a file and a number, never with a slogan. And when they find a limit, name it before they finish the sentence, then say why the line sits there. The one trap in this area is claiming that the interface protects something. It does not, and you must say so first, before they ask.



Login, then the two gates: permission and scope — Drawing this proves the three things the jury really wants to know: the password never leaves BCrypt, the server builds the permission list again from the database instead of trusting the token, and a request has to pass the verb gate and then the row gate before any amount is allowed out.

38

Why did you use JWT instead of a normal server session?

EASY

SAY THIS

Because the backend is a stateless REST API and the frontend is a separate Angular application. A session would tie every request to the memory of one server. So I set Spring Security to STATELESS and I carry the identity in a signed token instead.

IF THEY PUSH

In SecurityConfig I set SessionCreationPolicy.STATELESS, so Spring never creates an HttpSession. In development the Angular app runs on port 4200 and the API on 8090. In Docker the frontend is served by nginx on 8081 while the backend listens on 8080. That is two origins, so I use an explicit CORS allow-list instead of one shared session cookie. Here is the honest trade-off. A session gives you a way to cancel a login for free, and a JWT does not: once a token is signed I cannot take it back. That is why I added the token_version counter, which brings back one small piece of state on the server. So my design is not fully stateless for authorization. The transport is stateless, but every request still does one lookup that the cache answers. I made that choice on purpose and I would make it again.

POINT TO backend/src/main/java/com/pms/shared/config/SecurityConfig.java:52
(SessionCreationPolicy.STATELESS) · backend/src/main/resources/application.yml
(server.port 8090, pms.cors.allowed-origins) · Report chapter 3, section "Cross-cutting design decisions" – Authentication and session revocation

THEN THEY ASK

So you built sessions again? — No. The state I kept is one whole number for each user plus a five-minute cache entry, not a full session object, and it never has to be shared between servers.

DO NOT SAY

Do not say the application is "fully stateless". The permission lookup on every request makes that false, and a sharp examiner will catch it.

39

You have two tokens. What is the difference, and how long does each one live?

EASY

SAY THIS

The access token lives fifteen minutes and goes in the Authorization header on every call. The refresh token lives seven days and is only ever sent to one endpoint, /api/auth/refresh. Short life for the one that travels everywhere, long life for the one that almost never moves.

IF THEY PUSH

The values are 900 seconds and 604800 seconds. They are set in application.yml and repeated in application-prod.yml. The access token carries the email as subject, the role name, the permission list, the token version, a type claim and a firstLogin flag. The refresh token carries only the subject, the token version and its type. That type claim matters. JwtAuthenticationFilter accepts a token only when type equals "access", and AuthService.refresh refuses anything whose type is not "refresh". So you cannot hand over one token where the other one is expected. Both are signed with HMAC, using the JWT_SECRET environment variable, which must be at least 64 characters because the bytes are used as they are. Fifteen minutes is the longest time a stolen access token still works. A shorter life would mean more refresh traffic.

POINT TO backend/src/main/resources/application.yml (access-token-expiration: 900, refresh-token-expiration: 604800) · backend/src/main/java/com/pms/auth/service/JwtService.java:38-65

```
(claims of both tokens) ·  
backend/src/main/java/com/pms/auth/security/JwtAuthenticationFilter.java:43 and  
AuthService.java:91 (type check) · Report chapter 4, Sprint 1, "Two tokens rather than  
one"
```

THEN THEY ASK

Why not five minutes? — Five would work too. Fifteen was my balance between the time a stolen token stays useful and the amount of refresh traffic, for twenty to thirty internal users. And the number is configuration, not code, so it changes without a rebuild.

DO NOT SAY

Do not say the refresh token is "opaque". It is a signed JWT just like the other one, only with fewer claims.

40

Where do you store the refresh token, and why there?

EASY

SAY THIS

In an HttpOnly cookie, so JavaScript cannot read it. I also pinned its path to /api/auth/refresh and set SameSite=Strict, so the browser only ever sends it to the one endpoint that needs it.

IF THEY PUSH

AuthController writes the cookie with HttpOnly, with the Secure flag driven by configuration, with Path=/api/auth/refresh, with Max-Age equal to seven days, and with SameSite=Strict. The reason is how much damage a theft can do. The refresh token is the credential that can create new sessions, so it must be the one thing that a script on my page can never read. On top of that I rotate it: every successful refresh raises the user's token version, which kills the cookie that was just used. If an old one comes back, the version does not match any more, the server answers 401 with "Token révoqué", and it writes a warning about a possible theft. One honest point. The Secure flag comes from COOKIE_SECURE and it is false by default, so on plain HTTP the cookie travels in clear text. It must be set to true the moment the stack sits behind TLS, and that is written in the configuration and in DEPLOYMENT.md.

POINT TO backend/src/main/java/com/pms/auth/controller/AuthController.java:73-95 (addRefreshCookie / clearRefreshCookie) · backend/src/main/java/com/pms/auth/service/AuthService.java:99-119 (rotation and replay detection) · backend/src/main/resources/application-prod.yml (cookie-secure: \${COOKIE_SECURE:false}) · Report chapter 4, figure fig:seq-s1-revoke

THEN THEY ASK

What if someone steals the cookie anyway? — He gets one refresh out of it. The next time the real user refreshes, the version has moved and the stolen token is refused, and I write that replay in the log as a possible theft.

DO NOT SAY

Do not claim the cookie is always Secure. In the compose file you ship, the default is false, and saying anything else is a claim the repo proves wrong.

41

How are the passwords stored?

EASY

SAY THIS

With BCrypt at strength twelve. Never stored in clear text, never written in a log. And on login I run BCrypt even when the email does not exist, so the answer time does not tell an attacker which accounts are real.

IF THEY PUSH

The PasswordEncoder bean is a BCryptPasswordEncoder with strength 12, which is 4096 rounds. BCrypt also puts a random salt for each password inside the hash itself, so two identical passwords give two different hashes. The users table stores only password_hash. For the timing side, AuthService computes a dummy hash at startup and checks against it when the email is unknown, so an unknown account takes the same time as a wrong password. Repeated failures are limited by LoginAttemptTracker: five failures inside a sliding window of fifteen minutes, and then the endpoint answers 429. When an administrator creates an account, the temporary password has twelve characters from SecureRandom, with at least one upper case letter, one lower case letter, one digit and one special character, and first_login forces a change before anything else can be opened. The honest limit: for a password the user chooses himself, the only rule is a minimum of eight characters. No rule about the mix of characters, and no check against lists of leaked passwords.

POINT TO backend/src/main/java/com/pms/shared/config/SecurityConfig.java:81-84 and application.yml (bcrypt-strength: 12) · backend/src/main/java/com/pms/auth/service/AuthService.java:36-67 (dummy hash, timing equalization) · backend/src/main/java/com/pms/auth/service/LoginAttemptTracker.java:18-19 (5 attempts / 15 min) · backend/src/main/java/com/pms/auth/dto/ChangePasswordRequest.java:8 (@Size(min = 8)) · backend/src/main/java/com/pms/user/service/UserCrudService.java:100-112 (generateSecurePassword)

THEN THEY ASK

Why not Argon2? — Argon2 is the better modern choice. BCrypt at cost 12 is the default in Spring Security and it is still acceptable. And because nothing outside that one bean touches hashing, changing it is a one-line change.

DO NOT SAY

Do not invent a password policy. The repo asks for eight characters and nothing more. Say that.

SAY THIS

It is how I cancel a token that is already given out. Every token carries the user's version number. I raise that number in the database, and every token that user is holding stops working on his very next request.

IF THEY PUSH

token_version is a whole-number column on users. It was created in V1 and it is put inside both tokens. On every request, JwtAuthenticationFilter compares the version in the token with the version stored for that user. If they are different, it simply does not authenticate, and the request ends as 401. The counter goes up on logout, on a password change, on a refresh rotation, on a role change, on an account reset, on a deactivation and on a deletion. The migrations do it too: V12, V13, V23 and V25 each end with an UPDATE that raises the version, so a change of policy forces the users concerned to pick up the new state. The other option was a blacklist of cancelled tokens, which is shared state that only grows. One whole number for each user cancels every session of that person at once. The cost is one number comparison, plus a read in the database only when the cache misses, because the authority list is kept in Caffeine under the key email plus version, capped at 2000 entries with a five-minute write expiry. The limit I accept: it is all or nothing for one user. I cannot log out one laptop and leave the phone signed in.

POINT TO backend/src/main/resources/db/migration/V1__schema_auth.sql (token_version INT NOT NULL DEFAULT 0) · backend/src/main/java/com/pms/auth/security/JwtAuthenticationFilter.java:55-85 (version comparison and cache key) · backend/src/main/java/com/pms/user/service/UserCrudService.java:145,165,177,196 (revokeAllTokens on role change, reset, deactivate, delete) · backend/src/main/resources/db/migration/V12__fix_rbac_matrix.sql (final UPDATE users SET token_version = token_version + 1) · Report chapter 4, Sprint 1, "Revocation by version rather than by blacklist"

THEN THEY ASK

Why not just wait fifteen minutes for the token to expire? — Because a deactivated account has to lose access straight away. Fifteen minutes of live access for somebody who has just left the company is not an answer I could defend in front of the client.

DO NOT SAY

Do not call it a blacklist or a token store. It is a counter, and mixing the two destroys the whole reason for it.

SAY THIS

Because the policy changes more often than the code does. The permissions live in the `role_permissions` table, so an administrator can move a permission from one role to another while the app is running. Nothing is recompiled and nothing is redeployed.

IF THEY PUSH

V1 creates three tables: roles, permissions and the `role_permissions` link table. Today there are 25 live permission codes, spread over ten functional modules. The application code has 110 `@PreAuthorize` annotations, and not one of them names a role. They all name a permission code. The proof that this is not a slogan is migration V12: it deletes and rebuilds the whole permission matrix for all four business roles, and not one line of Java changed. V25 then added the administration screens, so the same thing can be done from the interface instead of a migration. Two honest limits. First, the codes are strings, so the compiler does not catch a typo. It fails closed, which means the endpoint simply says no to everybody, and the test suite covers the protected endpoints to catch that. Second, the names of the four business roles are still fixed in the migrations and protected by an `is_system` flag, because the migrations have to be able to call them by name. The names are fixed in the data. The policy attached to them is not.

POINT TO `backend/src/main/resources/db/migration/V1__schema_auth.sql` (roles, permissions, `role_permissions`) · `backend/src/main/resources/db/migration/V12__fix_rbac_matrix.sql` (whole matrix rebuilt, zero Java changed) · `backend/src/main/resources/db/migration/V25__rbac_admin_module.sql` (`is_system` flag, `MANAGE_ROLES` restored) · `backend/src/main/java/com/pms/user/service/RoleAdminService.java:79-99` (runtime edit of a role's permissions) · Report chapter 4, Sprint 2, "Permissions rather than roles in the code", figure `fig:class-rbac`

THEN THEY ASK

An enum would be type-safe. Why give that up? — An enum puts the policy back inside the jar, so every change of policy becomes a new release. I chose a policy that can be edited while the app runs, and I paid for it with string codes and tests.

DO NOT SAY

Do not say "no role name appears anywhere". Role names do appear in the migrations and in the seeders. Say that no role name appears in a business decision.

SAY THIS

A permission says which action you may do. A scope says which rows you may do it on. A project manager holds EDIT_PROJECT, but only for the projects he really manages. The permission alone does not open the whole portfolio.

IF THEY PUSH

There are two gates, one after the other. The first is @PreAuthorize on the service method, which asks whether you hold the code at all. The second is ProjectScopeService, which asks whether this project is inside your perimeter. The perimeter comes from a capability or from relations, never from a role name. Either you hold VIEW_ALL_PROJECTS, which the Director holds, or you are the chefProjet of that project, or you have an active team assignment on it. That is one JPQL query, findAccessibleProjectIdsByEmail, with an EXISTS subquery. If the project is outside, assertCanAccess throws AccessDeniedException and the caller gets a 403, not a quietly empty list, because an empty list hides a real authorization error. ProjectScopeServiceTest covers this with eight tests, including the case where a user without the global capability asks for a project outside the perimeter and is refused instead of being given nothing.

POINT TO backend/src/main/java/com/pms/project/service/ProjectScopeService.java:32-59 ·
backend/src/main/java/com/pms/project/repository/ProjectRepository.java:46-57
(findAccessibleProjectIdsByEmail) ·
backend/src/main/resources/db/migration/V13__add_view_all_projects.sql (VIEW_ALL_PROJECTS
granted to DIRECTEUR) ·
backend/src/test/java/com/pms/project/ProjectScopeServiceTest.java (8 tests) · Report
chapter 4, Sprint 2, Testing subsection

THEN THEY ASK

Why not fold the scope into the permission, like EDIT_OWN_PROJECT? — That multiplies the list by every kind of ownership. Keeping the action and the row filter apart is why I have 25 codes and not fifty.

DO NOT SAY

Do not describe the scope as only a filter on the query results. It also refuses a direct access by id, and that difference is the whole point.

SAY THIS

In three places, in this order. The filter builds the authority list from the database. The service method says which permission it needs, with `@PreAuthorize`. Then the interceptor checks that the project is inside your scope.

IF THEY PUSH

`JwtAuthenticationFilter.authenticate` checks the signature first. Then, and this is the part that matters, it does not trust the permission list inside the token. It loads the user with his role and the role's permissions from Postgres, or from the Caffeine entry keyed on email plus token version, and it turns each code into a `SimpleGrantedAuthority`. The permission claim inside the token exists only so the Angular application can draw its menu. Method security is switched on with `@EnableMethodSecurity`, and the 110 annotations sit on service classes, twenty-one of them, not on controllers. So a second controller that calls the same service still runs into the same check. The scope gate is `ProjectScopeInterceptor`. It is registered in `WebMvcConfig` for the path pattern `/api/projects/**`, and it takes the project id out of the URL and calls `assertCanAccess` before the handler runs.

POINT TO `backend/src/main/java/com/pms/auth/security/JwtAuthenticationFilter.java:55-85`
(authorities rebuilt from the DB, not the token) ·
`backend/src/main/java/com/pms/shared/config/SecurityConfig.java:29`
(`@EnableMethodSecurity`) ·
`backend/src/main/java/com/pms/project/service/ProjectService.java:37,78,99,187` and
`kpi/service/KpiService.java:62` (`@PreAuthorize` examples) ·
`backend/src/main/java/com/pms/shared/config/ProjectScopeInterceptor.java:26-44` and
`shared/config/WebMvcConfig.java:20-23`

THEN THEY ASK

Your report says the filter reads the permission codes from the token. Which one is true? —
The code reads them from the database. That sentence in the report is loose, and the database is the stricter of the two behaviours.

DO NOT SAY

Do not repeat the report's sentence that the filter turns the token's codes into authorities. Open the filter and show the repository call instead.

SAY THIS

You can call it with curl, and you should be able to. It is a REST API, and Angular is only one client. But you will not get any further. Every check is on the server. The Angular guards are there so that people do not click buttons that will fail.

IF THEY PUSH

With no token at all you get 401 from the authentication entry point, because the only open paths are login, refresh, the Swagger documents and the health probes of the container. With a developer's token on GET /api/projects/1/devis-interne you get 403, because DevisInterneService asks for MANAGE_DI, and migration V23 gives that code to the Director only. The reason curl does not help is that the annotations sit on the service layer, so there is no controller that reaches a protected operation without passing the check. On the frontend side, permission.guard.ts does not protect anything. It sends the user to the dashboard with a forbidden flag, and that is a decision about user experience. The honest limit: a service method with no annotation can be reached by any signed-in user. That is discipline backed by the test suite, not something the compiler enforces for me.

POINT TO backend/src/main/java/com/pms/shared/config/SecurityConfig.java:53-66 (permitAll list, anyRequest().authenticated(), 401 entry point) · backend/src/main/java/com/pms/project/controller/DevisInterneController.java:14 and project/service/DevisInterneService.java:37-68 (MANAGE_DI) · backend/src/main/resources/db/migration/V23__devis_interne.sql (MANAGE_DI granted to DIRECTEUR only) · frontend/pms-frontend/src/app/core/guards/permission.guard.ts (redirect, not protection) · Report chapter 7, section on the test run – the suite checks the authorization rule on every protected endpoint

THEN THEY ASK

Show me — curl -H "Authorization: Bearer <developer token>"

http://localhost:8090/api/projects/1/devis-interne returns 403. The same call with the Director's token returns the document.

DO NOT SAY

Do not say "nobody can call the API outside the app". It is a public HTTP API, and saying anything else makes every later answer sound learned by heart.

47

A developer works on a project. How exactly do you stop him from seeing its margin?

HARD

SAY THIS

The backend removes the fields before they leave the server. ProjectService calls withoutFinancials, which gives back the same project with every amount set to null. There is no hidden div. The numbers never reach the browser at all.

IF THEY PUSH

The line is in ProjectService.toResponse. If the caller holds VIEW_KPI, the response goes out as it is. If not, it goes through withoutFinancials. That method sets these fields to null: initialBudget, revisedBudget, effectiveBudget, licenseSubcontractBudget, penaltyProvision, margeNetteVendue, budgetTnd and pprTnd. It keeps soldWorkloadDays and warrantyWorkloadDays on purpose, because man-days are effort and the developer needs them. They are not money. After V12 the developer role holds exactly five codes: VIEW_PROJECT, VIEW_TEAM, SUBMIT_WORKLOAD, VIEW_WORKLOAD and VIEW_MISSION. VIEW_KPI is not one of them. The internal quote, which is where the margin is really computed, is a separate module behind MANAGE_DI, held only by the Director, so there is nothing to remove there: the whole endpoint answers 403. Note that this check asks for a capability, not for a role name. So if tomorrow the client wants one senior developer to see budgets, the administrator grants VIEW_KPI and nothing is recompiled.

POINT TO backend/src/main/java/com/pms/project/service/ProjectService.java:222-227
 (hasAuthority("VIEW_KPI") ? response : response.withoutFinancials()) ·
 backend/src/main/java/com/pms/project/dto/ProjectResponse.java:52-70 (withoutFinancials)
 · backend/src/main/resources/db/migration/V12__fix_rbac_matrix.sql (DEVELOPPEUR: 5 codes,
 no VIEW_KPI) · Report chapter 4, figure fig:ui-dev-dashboard – the same build, a
 developer session, no indicators and no billing

THEN THEY ASK

You remove the fields in one place. What about all the other endpoints that return money? — KPI and the internal quote are whole endpoints behind VIEW_KPI and MANAGE_DI, so there is nothing to remove there. Removing single fields is only needed on the project object, which everyone is allowed to read.

DO NOT SAY

Never say the field is "hidden in the interface" or "not displayed". That wording suggests the value was sent, and that is exactly what the question is looking for.

SAY THIS

His very next API call fails with 403. The server never reads permissions from the token. It reads them from the database, and editing a role clears the security cache. His menu is out of date until he reloads the page, but the API is not.

IF THEY PUSH

RoleAdminService.update saves the new set of permissions and then calls `evictSecurityContextCache`, which empties the whole Caffeine cache. The next request from that user misses the cache, so `JwtAuthenticationFilter` loads the role's permissions again from Postgres, the removed code is no longer in the authority list, and `@PreAuthorize` raises `AccessDeniedException`, which comes back as 403. His access token still carries the old permission claim for up to fifteen minutes, and that is harmless exactly because nothing on the server reads that claim. If instead the administrator changes the user's role, and not the contents of the role, `UserCrudService` raises `token_version`, so the session dies at once with 401 and he has to sign in again. The honest gap is on the client side. The Angular menu keeps the old entry until the next call to `/api/me/context` or until the next login, so he can click an item and get a 403 instead of seeing the item disappear. I did not build a push channel for that. This exact scenario is what the Sprint 2 review was accepted on: take a permission away, watch the menu entry and the API call, give it back, watch both come back, with nothing recompiled.

POINT TO `backend/src/main/java/com/pms/user/service/RoleAdminService.java:94-98,143-146` (save then `evictSecurityContextCache`) · `backend/src/main/java/com/pms/auth/security/JwtAuthenticationFilter.java:67-81` (cache miss reloads from the database) · `backend/src/main/java/com/pms/user/service/UserCrudService.java:135-153` (role change bumps `token_version`) · `backend/src/main/resources/application.yml` (Caffeine: `maximumSize=2000, expireAfterWrite=5m`) · Report chapter 4, Sprint 2, Sprint review

THEN THEY ASK

Is a 403 after a click not a poor experience? — It is, and asking `/api/me/context` again and again would fix it in a few lines. I chose correctness first, so the API is never wrong and only the menu is out of date for a short time.

DO NOT SAY

Do not say the user has to log out and log in again. That is what happens for a role change, not for a permission change, and mixing the two sounds like you are guessing.

49

You switched CSRF protection off in your security configuration. Justify that.

HARD

SAY THIS

I switched it off on purpose, not by mistake. Every signed-in call is authorised by an Authorization header, and a page on another site cannot set that header. The only endpoint that is authenticated by a cookie is refresh, and that cookie is SameSite=Strict and pinned to that one path.

IF THEY PUSH

CSRF is an attack on credentials that the browser attaches by itself. My access token is not one of those. JavaScript puts it in a header, and a bad page cannot make the browser add that header to a request that goes to another origin. The refresh cookie is the one automatic credential in the system, and it is limited in three ways: HttpOnly, Path=/api/auth/refresh so it is never sent to any other endpoint, and SameSite=Strict so the browser will not send it from another site at all. CORS is also an explicit list of origins with credentials allowed, never a wildcard. Now suppose all of that failed and an attacker could force a call to /api/auth/refresh. The effect is that the victim's own token rotates and he is logged out. Annoying, but no data is changed. The point I want to make is that this is a reasoned exception, tied to where the credential lives. The day the access token moves into a cookie, CSRF protection has to be switched back on. That is a rule I can state now, not something to be discovered later.

POINT TO backend/src/main/java/com/pms/shared/config/SecurityConfig.java:45 (csrf disable) and 86-96 (explicit CORS origins, allowCredentials true) ·
backend/src/main/java/com/pms/auth/controller/AuthController.java:82-86
(Path=/api/auth/refresh; HttpOnly; SameSite=Strict) ·
backend/src/main/java/com/pms/auth/service/AuthService.java:108-119 (refresh rotates the version)

THEN THEY ASK

So switching CSRF off is always fine with JWT? — No. It is fine only while the credential is a header. A credential carried in a cookie, on an endpoint that changes data, needs a CSRF token. No exception.

DO NOT SAY

Do not answer "we use JWT so CSRF does not apply". That is a slogan, and the refresh cookie is the counter-example sitting in your own code.

SAY THIS

Cross-site scripting. I keep the access token in localStorage, so a script injected into my page could read it and act as that user for up to fifteen minutes. The refresh cookie is HttpOnly, so it cannot create new sessions. That is exactly why I split the two tokens.

IF THEY PUSH

The line is in auth.service.ts, in storeSession, which writes the token in localStorage under the key pms_access. I will correct myself in front of you rather than let you find it: my report and my architecture document say the access token is kept in memory, and the code puts it in localStorage. localStorage is the weaker of the two, and the fix is to hold it inside the Angular service and to refresh again on every page reload. Here are the other things this design does not cover, so the list is complete. There is no Content-Security-Policy header: the configuration sets frame-options, content-type-options and referrer-policy, and nothing more. The compose file I ship has no TLS, so COOKIE_SECURE is false by default. There is no multi-factor authentication. The login rate limiter is a map in memory, so it resets when the app restarts, and it would not hold across two instances. There is no audit log, because VIEW_AUDIT_LOG was removed in V20 and never built again. And nothing here has been penetration-tested. That is the boundary, and I would draw it in the same place again for a final-year project, because a small number of mechanisms I can point at and test is worth more than a long list I cannot defend.

POINT TO frontend/pms-frontend/src/app/core/services/auth.service.ts:81-83,102-113 (localStorage pms_access) • backend/src/main/java/com/pms/shared/config/SecurityConfig.java:47-51 (headers set – no CSP) • backend/src/main/java/com/pms/auth/service/LoginAttemptTracker.java:21 (in-memory ConcurrentHashMap, per instance) • backend/src/main/resources/db/migration/V20__descope_unused_permissions.sql (VIEW_AUDIT_LOG removed) • docs/ARCHITECTURE.md section 5.3 – states "in memory"; the code does not

THEN THEY ASK

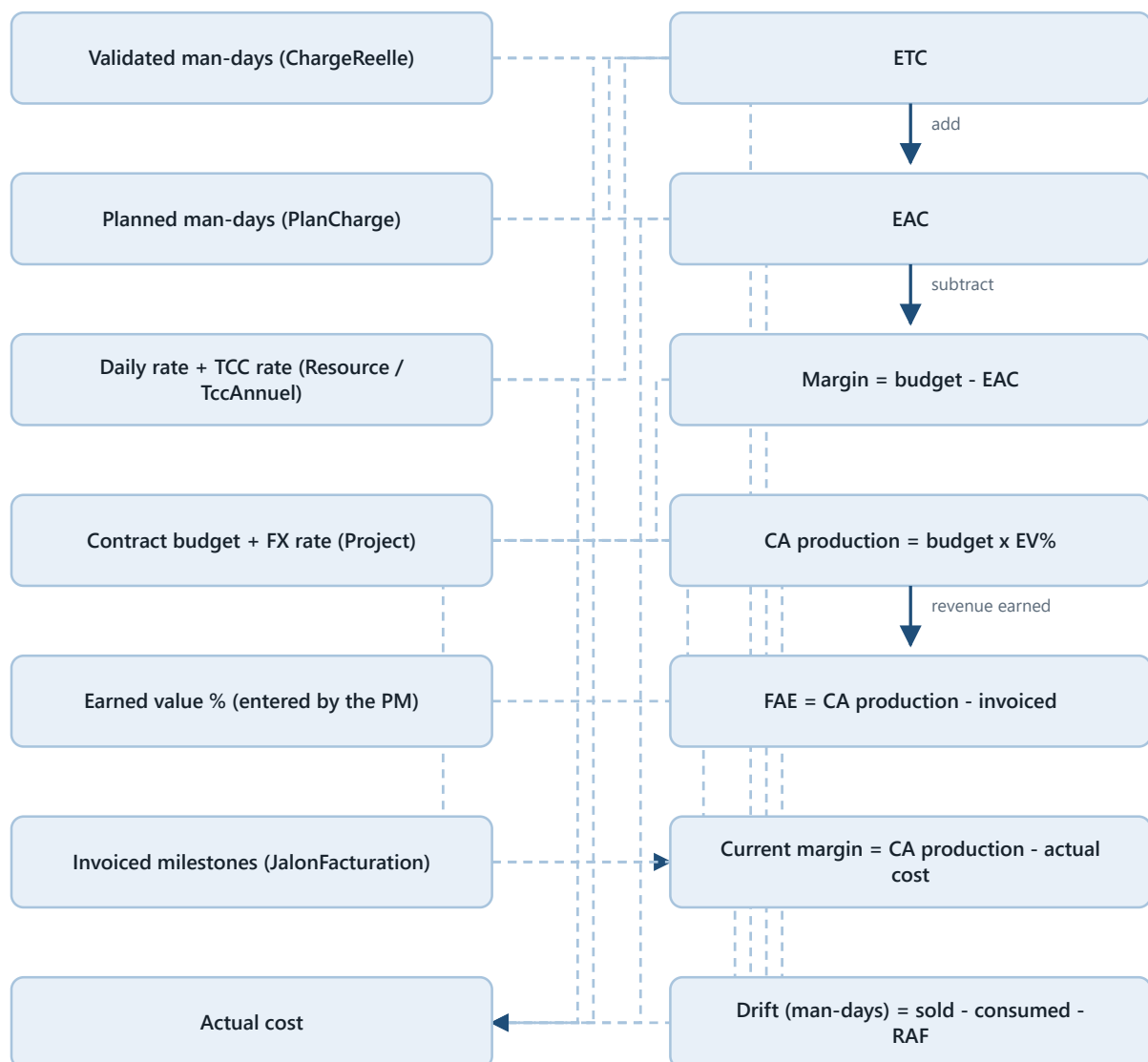
Which one would you fix first? — The token in localStorage, then a Content-Security-Policy header, because those two together are what really close the XSS path. Everything else on my list is narrower.

DO NOT SAY

Do not claim that the HttpOnly cookie stops XSS. It only limits the damage to fifteen minutes. It does not stop the attack, and saying too much here destroys the credit you just earned by naming the gap yourself.

The financial engine — EVM indicators, billing and margin

This is the heart of the project and the jury knows it, so this is where they stop being polite. They are not testing whether you know textbook earned value management. They are testing whether you can take any number on your dashboard, name the rows it came from, and do the arithmetic out loud with no notes. Give the numbers first and the vocabulary second. And whenever a formula has a boundary, say the boundary yourself before they find it.



From man-days and rates to the financial indicators — Drawing this proves that the engine has only five real inputs and one human judgement, and that every indicator on the dashboard is one named arithmetic step away from them.

51

Let us start with something simple. In one sentence, what is earned value management, and why did your project need it?

EASY

SAY THIS

Earned value management looks at three things on the same date. What the work was worth when we sold it. What we have really spent. And how much of that value we have really earned. ST2i already did this by hand in the workbook. My engine does it on its own.

IF THEY PUSH

The workbook has a sheet called Glossaire. It gives the meaning of every indicator and the way to calculate it. That sheet is the real specification of my engine. My code stands on three things: the contract budget, the actual cost worked out from validated man-days, and the earned value percentage that the project manager gives at the monthly review. From those three I get ETC, EAC, production revenue, accrued income and two margins. All of that happens inside one method, `KpiService.buildKpi`. None of this is generic library code. Every formula is the one written in the client's glossary sheet. That point matters, because I did not build textbook EVM. I built ST2i's EVM. The glossary also gives two indicators that no standard tool has: RAF and drift, and both are counted in man-days against a sold workload.

POINT TO `report/chap_01.tex` – "Glossary of indicators" sheet, Fig. fig:xl-glossary ·
`backend/src/main/java/com/pms/kpi/service/KpiService.java` – `buildKpi` ·
`docs/BUSINESS_ANALYSIS.md` §7 "Financial & KPI business model"

THEN THEY ASK

Then why not just take an EVM library that already exists, or a product like MS Project? —

Because a library gives you the textbook formulas. ST2i's glossary defines RAF and drift in man-days against a sold workload, and no ready-made product has those.

DO NOT SAY

Do not say the PMI definitions of CPI and SPI out loud. I do not compute them, and naming them invites a question my code cannot answer.

52

You turn a man-day into money. Show me exactly how you do it.

EASY

SAY THIS

One man-day costs the person's daily rate times one plus the TCC rate. TCC is the extra load for everything the salary does not show: charges, overheads, and the cost of the company itself. In the workbook that extra load is about seventy percent. So a day at four hundred dinars costs the company six hundred.

IF THEY PUSH

TCC means Taux de Coût Chargé, which is the fully loaded cost rate. In the Excel, the TCC sheet works it out this way: the yearly loaded cost times 1.7, divided by the working days of that year. In my model I keep the two parts apart on the Resource entity, dailyRate and tccRate. The cost of one charge line is days times dailyRate times one plus tccRate, rounded to two decimals. That is the method chargeCost in KpiService, and it is the only place in the whole backend where a day becomes money. I kept the salary rate and the extra load as two fields, instead of one number already multiplied. The director needs to see them apart, and the load changes for reasons that have nothing to do with the person. Here is one honest limit. If someone has effort booked but no Resource row, their cost counts as zero. I do not throw an exception. I add a warning message to the KPI response and the dashboard shows it, because one missing rate should never wipe out a whole portfolio screen.

POINT TO backend/src/main/java/com/pms/kpi/service/KpiService.java – chargeCost, and the warnings list built in buildKpi · backend/src/main/java/com/pms/user/entity/Resource.java · report/chap_01.tex – "TCC sheet", Fig. fig:x1-tcc

THEN THEY ASK

Why not just store the loaded rate directly? — Because the audit trail needs the salary rate and the overhead to stay visible apart and editable apart, and the Excel TCC sheet keeps them apart for the same reason.

DO NOT SAY

Do not present seventy percent as a fixed value written into the code. It is a field on each resource, and someone will ask to see the line where it is written.

53

What is the PPR, and where does the five percent come from?

EASY

SAY THIS

PPR is the provision for risk. It is five percent of the contract budget in dinars, put aside before anyone claims the margin. The five percent is not my number. It is the company's rule. It is written in the identification sheet of the workbook as budget times five percent.

IF THEY PUSH

In the code it is one small derived getter, Project.getPprTnd, which returns the budget in dinars times 0.05. It is never saved in the database, so it can never drift away from the budget it comes from. It is shown on ProjectResponse as pprTnd. It is also removed, together with every other amount, for any role that may not see financial data, and ProjectResponse.withoutFinancials does that. The honest limit is that the five percent is a constant in the entity, not a business setting you can change. I would turn it into a setting before a second client uses the platform. For ST2i it is a fixed company rule, and making it a setting nobody would ever change would have added a configuration point for nothing. The example project I studied in the real workbook had a PPR of about 63,493 dinars on a contract of about 1,269,861 dinars, which is the same five percent.

POINT TO

```
backend/src/main/java/com/pms/project/entity/Project.java – getPprTnd ·  
backend/src/main/java/com/pms/project/dto/ProjectResponse.java – withoutFinancials ·  
report/chap_01.tex – "Project identification sheet", Fig. fig:xl-identification ·  
docs/BUSINESS_ANALYSIS.md §7
```

THEN THEY ASK

Is the PPR taken away from your margin? — No. My margin is budget minus EAC. The PPR sits next to it, as a provision that the director reads. That is exactly what the identification sheet of the workbook does.

DO NOT SAY

Do not call it a buffer I designed. It is a company rule, and saying it is my design invites "on what basis did you choose five percent?"

54

Tell me what EV, ETC and EAC mean in your platform, and tell me which of the three the user types in.

EASY

SAY THIS

EV is the share of the contract value earned so far, and the project manager types it in. ETC is the cost of the months still to come. EAC is the actual cost plus ETC, so it is the total the project will have cost when it ends. Only EV is typed in. The other two are computed.

IF THEY PUSH

Actual cost is the sum of the validated man-days. Each one is priced at the rate of the year it was booked in. ETC prices the planned months the same way, but only the months that have no validated actual yet. In the code that is the list remainingPlan. I build it by taking out every plan line whose user-and-month key is already in the validated set. EAC is simply budgetConsume.add(etcCost). EV comes in through SnapshotRequest as a percentage between 0 and 100, and validation annotations on the DTO keep it inside those limits. There is one result I have to own. ETC works month by month, so a month that is only half used but already validated adds nothing to ETC. The whole platform works month by month, the workbook worked month by month too, and I would draw that line the same way again. Going finer would mean building a weekly timesheet product, and that is not what ST2i asked for.

POINT TO backend/src/main/java/com/pms/kpi/service/KpiService.java – remainingPlan, etcCost, eac ·
backend/src/main/java/com/pms/kpi/dto/SnapshotRequest.java · report/chap_06.tex – Sprint
8, "The computed indicators"

THEN THEY ASK

So your EAC can be lower than the actual cost plus the whole plan? — Yes, and that is on purpose. Once a month is validated it is history, and the actual figure takes the place of the plan for that month.

DO NOT SAY

Do not say EAC means budget at completion. That is a different EVM term, and I do not compute it.

55

Explain CA production and FAE, and tell me why you show two different margins on the same screen.

MEDIUM

SAY THIS

CA production is the revenue we have earned: the contract total times the EV percentage. FAE is what we have earned but have not invoiced yet, so it is CA production minus the invoiced total. Then I have two margins. Budget minus EAC says where the project is going. CA production minus actual cost says where it stands today.

IF THEY PUSH

CA production is $\text{budgetTnd} \times \text{evPct}$ divided by one hundred. The invoiced total adds up only the milestones in status FACTURE or PAYE, changed into dinars with the project rate, so a milestone that is only planned adds nothing. FAE is CA production minus that total. FAE means Facture À Établir. In French accounting it is accrued income: revenue you record before the invoice exists. ST2i's glossary says exactly that. The forward margin, *marge*, is $\text{budgetTnd} - \text{EAC}$. Next to it I return *tauxConsommation*, which is actual cost over budget. The current margin, *margeActuelle*, is CA production minus actual cost, and *margeActuellePct* is that as a ratio. I kept both, because they answer two different questions. The director asks if this project will make money. The project manager asks if it is making money right now. Both are null when nobody has ever entered EV. In that case the response carries a clear warning instead of showing a zero that would fool the reader.

POINT TO `backend/src/main/java/com/pms/kpi/service/KpiService.java` – `caProduction`, `totalFacture`, `fae`, `margeActuelle` · `backend/src/main/java/com/pms/kpi/dto/KpiResponse.java` · `report/acronymes.tex` – FAE entry · `docs/BUSINESS_ANALYSIS.md` §7

THEN THEY ASK

Why do you measure the current margin against actual cost and not against EAC? — Because it is a photo of today: revenue already earned against money already spent. Putting earned revenue against a forecast cost would mix a fact with a forecast.

DO NOT SAY

Do not say FAE is revenue we are sure to collect. It is an accrual built on an estimate, so if the EV is wrong, the FAE is wrong with it.

56

Your earned value is typed in by a person. Is that not the weak point of the whole engine?

MEDIUM

SAY THIS

It is the most subjective input in the system, and I made it visible instead of hiding it. The client's own glossary says EV is the project manager's estimate, not a measurement. If I had worked it out from consumed days, I would have made a number that looks objective but is not.

IF THEY PUSH

The obvious other way is EV equals consumed days over sold days. That would mean a project that burns days and delivers nothing still shows progress. That is exactly the failure earned value management exists to catch. The second other way is to take EV from the deliverables register. I do compute that on its own, as deliveryPct, which is the delivered or validated deliverables over the total, and I show it right next to EV. So anyone can see when the manager's judgement and the register do not agree. That gap is the useful signal. The EV entry is limited from 0 to 100 at the DTO. It is captured together with the estimated end date and the faits marquants of that month, which are the notable events. Then it is frozen into the snapshot, so a later change cannot rewrite a review that a decision was already based on. If you push me further, I would add a field next to the EV where the manager must write why. That is already item B-1 in my backlog.

POINT TO backend/src/main/java/com/pms/kpi/service/KpiService.java – deliveryPct ·
 backend/src/main/java/com/pms/kpi/dto/SnapshotRequest.java · report/chap_06.tex – Sprint
 8, design decision "The earned value is entered, not computed" · docs/ENHANCEMENTS.md –
 item B-1

THEN THEY ASK

Could a manager put the EV too high to hide an overrun? — For one month, yes. Not for long. The snapshot cannot be changed, and the delivery percentage next to it does not move, so the gap shows up in the history.

DO NOT SAY

Do not promise a future version that computes EV on its own from tasks. The client's method says it is a judgement, and turning that around turns the specification around too.

57

How does a billing milestone turn into an amount of money, and what is this hundred percent rule?

MEDIUM

SAY THIS

A milestone has a label, a percentage and a due date. Its amount is the effective budget times that percentage. All the percentages on one project may not add up to more than one hundred. The Excel states that rule but cannot apply it. My service layer refuses the save.

IF THEY PUSH

computeMontant in JalonService is budget times percentage divided by one hundred, rounded to two decimals. The budget there is getEffectiveBudget: the revised budget if there is an amendment, and the initial one if there is not. Before every create and every update, validatePourcentageSum reads the current sum from the database, takes away the old value of the line being edited, adds the new one, and throws a BusinessException if the result goes past one hundred. Taking away the old value matters. Without it, changing a milestone from thirty to twenty-five percent on a project already at one hundred would be refused, and that would be wrong. JalonServiceTest covers both sides: a sum that would reach one hundred and ten is refused, and a sum of exactly one hundred is accepted. The rule is not more than one hundred, not exactly one hundred. A project may fairly have only part of its contract cut into milestones while the rest is still being negotiated. Here is one thing I will own. With the default isolation level, two inserts at the same time could both read the same sum and both pass. I accepted that, because one project manager owns the billing plan of a project. A unique constraint cannot say "sum at most one hundred", so the clean fix is a pessimistic lock on the project row.

POINT TO backend/src/main/java/com/pms/billing/service/JalonService.java – computeMontant, validatePourcentageSum ·
backend/src/test/java/com/pms/billing/service/JalonServiceTest.java – 10 tests ·
report/chap_06.tex – Sprint 6, design decision "The sum of percentages may not exceed 100%"

THEN THEY ASK

Where is that rule in the Excel? — It is written in the billing progress sheet as a convention, and the spreadsheet has no way to apply it. That is exactly the kind of rule the platform exists to make real.

DO NOT SAY

Do not say the milestone amount is never stored. It is a column on the row. What is true is that it is always recomputed from the percentage and the budget, and never typed by hand.

58

Why can one resource have a different rate for each year? Is that not the same data stored twice?

MEDIUM

SAY THIS

Because a day worked in 2024 did not cost what a day worked in 2025 costs. The workbook keeps one TCC column per year for the same person. If I priced everything at today's rate, every past month would quietly change price the moment someone got a raise.

IF THEY PUSH

The Resource row has a base daily rate and a base TCC rate. The TccAnnuel table holds, for one resource and one year, a replacement for both. In chargeCost I read the year of the charge's period. If a TccAnnuel exists for that year I use it. If not, I fall back to the base rate on the resource, and that fallback is the normal case for a resource that was just created. This is rule 4 of the F-AFF-13 specification. The result is that a validated month is priced once and keeps that price. That is what

makes a frozen snapshot comparable with the one before it. This is not the same data twice. It is a time dimension: the base rate is the default, and the yearly row is the exception. For speed, I load every TccAnnuel for every user involved in one single query and put them into a map by user and year. So a project with forty people across three years is still two queries, not a hundred and twenty.

POINT TO backend/src/main/java/com/pms/user/entity/TccAnnuel.java ·
backend/src/main/java/com/pms/kpi/service/KpiService.java – tccByUserYear, chargeCost ·
report/chap_04.tex – Sprint 2, design decision "A cost rate per year, not per resource"

THEN THEY ASK

What if a rate changes in the middle of a year? — My model cannot say that, because the workbook works year by year. To handle it, a rate would need dates saying when it is valid, and that is a schema change I would only make if the company asked for it.

DO NOT SAY

Do not say the fallback to the base rate never happens. It is the normal path for any resource that has no yearly row yet.

59

You compute every indicator again on every read instead of storing them. Defend that. And then explain why you have a snapshot table anyway.

MEDIUM

SAY THIS

A stored amount can drift away from the numbers it came from. That is exactly how the Excel broke: someone pasted a fixed value over a formula and nobody saw it. So live figures are computed again from quantities and rates on every read. The snapshot is a different thing. It is the monthly review, frozen on purpose.

IF THEY PUSH

The live path is computeLive. It is marked read-only and it has no side effect at all, so two reads one after the other always agree, because they read the same rows. The snapshot path, createSnapshot, runs that very same buildKpi and then writes the result into a SnapshotKpi row dated today, together with the EV, the estimated end date and the faits marquants the manager entered. There is one snapshot per project per day. A second try on the same day returns 409, and there is a test for that. The idea is simple: live figures to steer, frozen figures for history. A correction made in October must not rewrite the September review that a decision was already taken on. That follows the workbook, where the dashboard is live and the Statistiques sheet keeps one column per month. The known cost is speed. The portfolio screen fires one KPI call per project, and each one computes again from raw rows. I have measured nothing, because the volumes are tiny. I wrote it down as debt I chose on purpose, item A-4, and the fix is a short-lived cache, added when it is really needed and not before.

POINT TO backend/src/main/java/com/pms/kpi/service/KpiService.java – computeLive, createSnapshot ·
backend/src/main/java/com/pms/kpi/entity/SnapshotKpi.java ·
backend/src/test/java/com/pms/kpi/KpiControllerTest.java – 6 tests, including the same-

THEN THEY ASK

So a snapshot can disagree with a fresh computation for the same date? — Yes, and that is the point. If someone fixes an old man-day entry, the live figure moves and the frozen review does not. The gap between the two is the audit trail.

DO NOT SAY

Do not say the read path is cached or optimised. It is neither, and the code shows it in ten seconds.

60

Walk me through the numbers. Take a small project and tell me the actual cost, the EAC and the margin out loud.

HARD

SAY THIS

Take a project sold at thirty thousand dinars, with forty-five man-days sold, two people and two months. I will give you an actual cost of ten thousand eight hundred, an EAC of twenty-one thousand three hundred, and a forward margin of eight thousand seven hundred.

IF THEY PUSH

Two people. Anis has a daily rate of 400 and a TCC rate of 0.50, so his day costs 400 times 1.5, which is 600. Sonia is 300 times 1.5, which is 450. The plan is ten days each in January and ten days each in February. So the planned cost is twenty days at 600 plus twenty at 450, which is 21,000. Only January is validated. Anis did twelve days and Sonia did eight. Actual cost is 12 times 600 plus 8 times 450, so 7,200 plus 3,600, which is 10,800, on twenty man-days consumed. February has no validated actual, so it stays in the remaining plan. ETC is 10 times 600 plus 10 times 450, which is 10,500, and RAF is twenty man-days. EAC is 10,800 plus 10,500, which is 21,300. So the margin is 30,000 minus 21,300, which is 8,700, and the consumption rate is 10,800 over 30,000, which is thirty-six percent. Drift is 45 sold minus 20 consumed minus 20 remaining, so plus five man-days, still inside the envelope. Now the manager states EV at forty percent. CA production is 30,000 times 0.40, which is 12,000. One milestone at thirty percent has been invoiced, so the invoiced total is 9,000 and the FAE is 3,000. The current margin is 12,000 minus 10,800, which is 1,200, and that is ten percent of CA production. The forward margin is 8,700 on 30,000, which is twenty-nine percent. That gap between ten and twenty-nine percent is the real message of the screen. January went over the plan, and the rest of the project has to beat the plan for it to land where it was sold.

POINT TO backend/src/main/java/com/pms/kpi/service/KpiService.java – buildKpi, the full computation in one method · backend/src/main/java/com/pms/kpi/dto/KpiResponse.java · report/chap_06.tex – Sprint 8, Fig. fig:ui-kpi

THEN THEY ASK

Where does the thirty thousand come from if there has been an amendment? — From `getEffectiveBudget`, changed into dinars with the project rate: the revised budget when there is one, and the initial budget when there is not.

DO NOT SAY

Do not round in the middle and do not drop the units. They are checking if I can do the arithmetic live, not if I learned a result by heart.

61

Walk me through the numbers again, but this time on a project sold in euros, with an amendment in the middle.

HARD

SAY THIS

The contract is one hundred thousand euros, the rate is three point three dinars to the euro, and there are three milestones at twenty, fifty and thirty percent. The first one is invoiced at twenty thousand. Then an amendment of twenty thousand arrives. I will show you why the milestones no longer add up to the budget, and why that is correct.

IF THEY PUSH

The first amounts are 20,000, 50,000 and 30,000 euros. Each one is its percentage of the effective budget, which is the initial budget while there is no amendment. The kick-off milestone is invoiced, so its status becomes `FACTURE` and its amount is frozen at 20,000 euros. Now the director records an amendment of plus 20,000. `AvenantService` sets the revised budget to 120,000, saves the project, and calls `recomputePrevuMontants`. That method recomputes only the milestones still in status `PREVU`. So the middle milestone becomes fifty percent of 120,000, which is 60,000, and the last one becomes 36,000. The invoiced one is not touched. The percentages still add up to one hundred, but the amounts add up to 20,000 plus 60,000 plus 36,000, which is 116,000, not 120,000. The missing 4,000 euros is exactly twenty percent of the amendment, that is the share already invoiced at the old budget. This is on purpose, because a closed invoice is never rewritten, and what is left has to be picked up by a new milestone. That last step is manual, and I did not automate it. On the KPI side everything is changed into dinars with the project rate. The budget in dinars is 120,000 times 3.30, which is 396,000. The invoiced total is 20,000 times 3.30, which is 66,000. And the PPR is five percent of 396,000, which is 19,800.

POINT TO `backend/src/main/java/com/pms/billing/service/AvenantService.java` – `create` ·
`backend/src/main/java/com/pms/billing/service/JalonService.java` – `recomputePrevuMontants`,
`computeMontant` · `backend/src/main/java/com/pms/project/entity/Project.java` –
`getBudgetTnd`, `getPprTnd` · `backend/src/test/java/com/pms/billing/AvenantServiceTest.java` –
5 tests

THEN THEY ASK

So your system knowingly leaves four thousand euros unbilled? — It leaves them attached to no milestone, and the gap is visible because the percentages and the amounts stop agreeing. I would rather show a visible gap than quietly reopen a closed invoice.

DO NOT SAY

Do not say the milestone amounts always add back up to the budget. After an amendment they do not, and a jury doing the arithmetic will catch it.

62

Be precise now. What exactly does an amendment change, and what does it leave alone on purpose?

HARD

SAY THIS

It writes the revised budget, and nothing else in the system writes that field. The milestones still planned are recomputed against the new budget. The milestones already invoiced or paid are frozen. If you delete the amendment, the budget goes back and the planned milestones are recomputed again.

IF THEY PUSH

AvenantService.create reads getEffectiveBudget, adds the amendment amount, sets revisedBudget, saves the project, and calls recomputePrevuMontants. Delete does the opposite subtraction and calls the same recompute. That recompute asks only for rows with status PREVU, so FACTURE and PAYE are never touched. Accounting that is closed stays closed. The whole KPI side follows on its own, because margin, consumption rate, CA production and PPR all read getBudgetTnd, and getBudgetTnd reads getEffectiveBudget. Two honest limits. First, the Avenant entity has workloadDays, the impact on sold man-days. I store it, but I do not add it to the project's soldWorkloadDays. Only a project update writes that field. So between the amendment and the project update, the drift indicator is out of date. I could close that gap in three lines. I left it because I wanted exactly one writer per field. Second, delete takes the amount away from the current effective budget instead of replaying the history of amendments. That lands correctly as long as amendments are separate additions, and for ST2i they are.

POINT TO backend/src/main/java/com/pms/billing/service/AvenantService.java – create and delete ·
backend/src/main/java/com/pms/billing/service/JalonService.java – recomputePrevuMontants ·
backend/src/main/java/com/pms/billing/entity/Avenant.java – workloadDays ·
backend/src/main/java/com/pms/project/service/ProjectService.java – setSoldWorkloadDays ·
report/chap_06.tex – Sprint 6, "Amendments"

THEN THEY ASK

So your drift indicator can be wrong? — Yes, between the amendment and the project update. It is one field, I know about it, and it is the kind of thing I would rather name myself than let you find.

DO NOT SAY

Do not answer in general words about the amendment adjusting the project. They will ask which fields exactly, and being vague there sounds like not knowing.

SAY THIS

By checking them by hand at the end of each financial sprint, not by an automated test. One project was entered in both, the indicators were read side by side, and where they differed the workbook was treated as right and my code was fixed. I have no automated test that checks an EAC figure, and I will not pretend I have one.

IF THEY PUSH

To be concrete: in the sprint 6 review, the billing progress sheet was entered again milestone by milestone, and the invoiced totals were compared line by line. In the sprint 8 review, one project was entered on the platform and the indicators were read twice, once in the workbook and once on the dashboard. Both reviews are written up in chapter 6. What the test suite does cover is the business rules that the workbook states but cannot apply: the hundred percent cap, the fact that an invoiced milestone cannot be changed, and the fact that only an amendment moves the revised budget. That is twenty-three tests across the billing module. What it does not cover is a golden-dataset test that feeds known man-days and rates into buildKpi and checks the EAC that comes out. KpiControllerTest is six tests, covering the shape of the response, the empty-project case, the duplicate-snapshot conflict and the permissions. Across the whole backend the figure is forty-one percent of lines and twenty-four percent of branches, including the demonstration seeder, with the kpi module at seventy-nine percent of lines but only thirty-six percent of branches. Those branch numbers are the honest weak point, because the branches that are not covered are exactly the null cases: no budget, no EV, no rate. If I had one more week, that golden-dataset test is the first thing I would write, and the worked example I gave you earlier is exactly the fixture it would use.

POINT TO report/chap_06.tex – Sprint 6 and Sprint 8 "Sprint review" sections · report/chap_07.tex – Table tab:coverage, run of 13 September 2026, and Fig. fig:cap-coverage · backend/src/test/java/com/pms/kpi/KpiControllerTest.java · backend/src/test/java/com/pms/billing/service/JalonServiceTest.java · backend/src/test/java/com/pms/billing/AvenantServiceTest.java

THEN THEY ASK

Then how can the company trust these figures in production? — They should run one month in parallel, workbook and platform side by side, before they switch. That is the deployment recommendation, and it is the same check I did by hand at each sprint review.

DO NOT SAY

Do not blow up the sprint reviews into "validated against the client's real production data". The real workbook figures are confidential and hidden in the report, and overselling this is the one thing that could cost me the defence.

Database and data integrity

When a jury asks about the database, they are not testing whether you know what an index is. They are testing whether you made choices on purpose and can name what each one costs. A student who copied a tutorial cannot explain why a constraint exists. A student who designed a schema can. Answer with the concrete thing every time: the migration number, the constraint name, the column type. Where the design has a real hole, say it in one sentence, then say what you would add and why it is small. That sounds like engineering judgement. A defensive answer sounds like a student who does not know his own system.



The projects table and its neighbours (as built, V1 to V28) — Every arrow here is a real foreign key in the migrations. projects points only at users, and fourteen tables point back at it. That is why nearly every index in the schema starts with project_id, and why deleting a project can only ever be logical.

SAY THIS

I chose PostgreSQL 17 because this whole project is money arithmetic, and PostgreSQL gives me an exact decimal type, check constraints and partial indexes inside the engine itself. It is free, ST2i can keep it running after I leave, and the same version runs on my machine, in CI and in the container.

IF THEY PUSH

Three features decided it, and I use all three. First, NUMERIC, an exact decimal type, which a margin calculation needs. Second, partial indexes, which are indexes that cover only part of a table. I have seven unique indexes written WHERE deleted = FALSE, and without them my soft delete would be broken. MySQL cannot do that. Third, check constraints: I have thirty-one of them, and they enforce business rules inside the engine. On top of that, PostgreSQL runs DDL inside transactions, so a migration that fails halfway rolls back instead of leaving half a schema behind. The version is pinned as postgres:17-alpine in docker-compose.yml, so my laptop, the CI runner and the deployed stack all run the same engine. The price I accept is that I am now tied to features only PostgreSQL has. Moving to another engine would mean writing the partial indexes again in Java code. I made that trade on purpose.

POINT TO docker-compose.yml (postgres:17-alpine) ·
backend/src/main/resources/db/migration/V18__soft_delete_unique_indexes.sql ·
backend/src/main/resources/db/migration/V5__schema_project.sql · DECISIONS.md, ADR-019 ·
Report chapter 3, section "Database and persistence"

THEN THEY ASK

Would MySQL have worked? — For most of the schema yes, but not for the partial unique indexes that make my soft delete safe. I would have had to enforce that uniqueness in Java and lose the guarantee.

DO NOT SAY

Do not say "because it is the best open-source database". Name the three features you really use, or the answer sounds like a slogan.

SAY THIS

Because a float cannot hold 0.10 exactly. Add a thousand invoice lines in floats and the total drifts by a few cents. A few cents on a margin means a wrong management decision. Every amount in my schema is NUMERIC, and every amount in Java is BigDecimal.

IF THEY PUSH

There is not one FLOAT, REAL or DOUBLE PRECISION column anywhere in the twenty-eight migrations. Amounts are NUMERIC(15,2), and twenty columns use it. Daily cost rates are NUMERIC(10,2), ratios are NUMERIC(7,4) or NUMERIC(5,4), and the exchange rate is NUMERIC(15,6). The scales are different on purpose. Money needs two decimals because the millime is the smallest unit. A margin of 44.12 percent must be stored as 0.4412, which needs four. A conversion rate to the dinar needs six. On the Java side everything maps to BigDecimal, and no entity has a double or a float field. I also write the divisions out instead of leaving them to the machine. KpiService computes the consumption rate with `divide(budgetTnd, 4, RoundingMode.HALF_UP)`, so the rounding rule is written down and anyone can check it. The simple demonstration is this: 0.1 + 0.2 in a double gives 0.30000000000000004, while in NUMERIC it gives exactly 0.3.

POINT TO `backend/src/main/resources/db/migration/` (20 columns NUMERIC(15,2), zero float columns) ·
`backend/src/main/java/com/pms/project/entity/Project.java` ·
`backend/src/main/java/com/pms/kpi/service/KpiService.java` · DECISIONS.md, ADR-020

THEN THEY ASK

Why NUMERIC(15,2) and not something bigger? — Fifteen digits holds up to about ten thousand billion, far above any ST2i contract, and two decimals is the millime, so more precision would only store noise.

DO NOT SAY

Do not say floats are "less precise". They are not exact. Say that, and give the 0.1 plus 0.2 example.

66

You use transactions. What does a transaction really guarantee?

EASY

SAY THIS

That a group of writes is all or nothing. My clearest example is a contract amendment. Recording one touches three tables, and if any part fails the database goes back to the state before. There is never a project carrying a new budget with no amendment to explain it.

IF THEY PUSH

`AvenantService.create` is marked `@Transactional`. In one call it adds the amendment amount to `projects.revised_budget`, computes again the amount of every milestone still in PREVU status in `jalons_facturation`, and inserts the row in `avenants`. Three tables, one unit of work. Without the transaction, a crash between the first write and the third leaves a budget nobody can explain, and the margin this whole tool exists to compute becomes wrong. The four guarantees are simple. Atomicity: all or nothing. Consistency: the constraints hold at commit. Isolation: somebody reading at the same time does not see half the change. Durability: once it is committed, it survives a crash. There are 121 `@Transactional` annotations in the codebase, and read paths are marked `@Transactional(readOnly = true)` so the database can skip dirty-checking. I also set `open-in-view: false` in `application.yml`, so no lazy query can quietly run outside a transaction while a response is being written out.

POINT TO

```
backend/src/main/java/com/pms/billing/service/AvenantService.java ·  
backend/src/main/java/com/pms/billing/service/JalonService.java (recomputePrevuMontants)  
· backend/src/main/resources/application.yml (open-in-view: false)
```

THEN THEY ASK

What isolation level do you run? — PostgreSQL's default, READ COMMITTED. The honest limit is that AvenantService reads the budget and writes it back, so two amendments recorded on the same project in the same second could lose one update. I have no @Version optimistic lock, and adding one is a single column and one migration.

DO NOT SAY

Do not recite A-C-I-D as four words first. Give the amendment example, then name the four properties. Otherwise it sounds learned by heart.

67

You use Flyway. Why? And why can you never edit a migration once it has run?

MEDIUM

SAY THIS

Flyway applies numbered SQL scripts in order and stores a checksum, a kind of fingerprint, of each one. If I edited V5 after it had run, the fingerprint would no longer match and the application would refuse to start. That is exactly what I want, because that database already holds the old V5 and cannot un-run it.

IF THEY PUSH

The schema is twenty-eight scripts, V1 to V28. Flyway keeps a table called flyway_schema_history. For each script it holds the version, the checksum and the time it ran, and at startup it applies only what is missing. Editing a script that already ran does not travel back in time. My machine has the old shape, a colleague who clones fresh gets the new one, and the two databases drift apart in silence. The checksum turns that silent drift into a loud failure at startup. The right move is always a new script, and I did it twice for real. V17 fixes a check constraint added in V5 that refused a one-day project. It drops the old constraint and adds the corrected one, instead of rewriting V5. V18 replaces the UNIQUE constraints created in V1 and V5 with partial unique indexes, again with a new script. About the configuration: validate-on-migrate is true in the production profile, because a container starts from a clean volume and any drift must fail loudly. It is false in the base and dev profiles, because a local database drifts when I try things by hand, and I did not want that to block my daily work.

POINT TO backend/src/main/resources/db/migration/V17__fix_project_date_check.sql ·
backend/src/main/resources/db/migration/V18__soft_delete_unique_indexes.sql ·
backend/src/main/resources/application-prod.yml (validate-on-migrate: true) ·
DECISIONS.md, ADR-019 · Report chapter 4, "Schema management through Flyway"

THEN THEY ASK

What if a script has a typo that breaks a fresh install? — I fix it only while it has never left my machine. The moment it is pushed and applied somewhere else it is frozen, and the fix becomes a new script.

That is exactly what V17 is.

DO NOT SAY

Do not say "because it is good practice". Say the checksum catches it when a database and its written history disagree, and name V17 to show you have lived it.

68

Hibernate can create the schema for you. Why did you not just use `ddl-auto=update`?

MEDIUM

SAY THIS

Because update only adds. It never drops a column, never renames one, never writes a check constraint, and never tells you what it did. I run `ddl-auto=validate` instead.

Hibernate compares the entities with the schema and refuses to start if they do not agree.

IF THEY PUSH

Three concrete reasons. First, update only adds things. If I rename a field, Hibernate creates a new empty column and leaves the old one holding the data, without a word, and nobody finds out until a report is wrong. Second, it cannot express what my schema depends on: the thirty-one check constraints, the seven partial unique indexes written `WHERE deleted = FALSE`, and the data migrations. V20 deletes two permissions that were seeded but never enforced. V23 creates the `MANAGE_DI` permission and then raises `token_version` so Director sessions reload their rights. Hibernate knows nothing about any of that. Third, there is no review and no history. With Flyway a schema change is a file in a pull request that a human reads, and the twenty-eight files are the living documentation of how the database got here. The choice is written down as ADR-019 and stated in chapter 4. The price of my choice is that I write the SQL by hand, and the entity can drift away from the table. That is exactly why validate is on. If they drift, the application fails at startup, not at three in the morning with a user on it.

POINT TO `backend/src/main/resources/application.yml` (`ddl-auto: validate`) ·
`backend/src/main/resources/db/migration/V20__descope_unused_permissions.sql` ·
`backend/src/main/resources/db/migration/V23__devis_interne.sql` · DECISIONS.md, ADR-019 ·
Report chapter 4, lines on `ddl-auto validate`

THEN THEY ASK

Then why does your test profile use `create-drop`? — Because the unit tests run on an in-memory H2 with Flyway switched off, for speed. That is a choice about the test suite, never about a real database.

DO NOT SAY

Do not attack `ddl-auto` in general. Say it never drops and cannot write a check constraint, then say `validate` is what you really run, and why.



What is an index? Which ones did you create, and on what basis?

MEDIUM

SAY THIS

An index is a sorted structure the database keeps next to a table, so it can jump straight to the rows instead of reading all of them. I have thirty-nine: thirty-two plain and seven unique. Almost all are partial. They index only the rows where deleted is false, because those are the only rows I ever ask for.

IF THEY PUSH

Without an index, finding every workload row for project twelve means reading the whole table. With one, the database goes straight to them. I added indexes for two reasons. The first is every foreign key that a screen filters on: `plan_charges(project_id)`, `charges_reelles(project_id)`, `jalons_facturation(project_id)`, `backlog_items(sprint_id)` and, since V28, `backlog_items(assignee_id)`. Every child list in the interface is opened for one project, or one sprint, or one developer. The second reason is uniqueness that carries a business meaning. `uk_pc_active` covers (`project_id`, `user_id`, `period`) and makes it impossible to hold two planned-workload rows for the same person, the same project and the same month. That is the Excel cell, and a cell holds one value. The same shape guards `charges_reelles`, `snapshot_kpis` on (`project_id`, `snapshot_date`), and `tcc_annuels` on (`resource_id`, `annee`). The `WHERE deleted = FALSE` clause does two jobs. It keeps the index smaller, and it makes uniqueness apply only among live rows. The cost is honest: every index slows inserts down and takes disk space. So I did not add them freely, only on the foreign keys used as filters and on the columns that must be unique.

POINT TO `backend/src/main/resources/db/migration/V7__schema_workload.sql` ·
`backend/src/main/resources/db/migration/V8__schema_kpi.sql` ·
`backend/src/main/resources/db/migration/V21__tcc_annuel.sql` ·
`backend/src/main/resources/db/migration/V28__backlog_item_assignee.sql` ·
`backend/src/main/resources/db/migration/V5__schema_project.sql`

THEN THEY ASK

Did you measure a query before and after adding an index? — No. I ran no `EXPLAIN ANALYZE` benchmarks. I chose the indexes by thinking about how each screen reaches its data, and with the demo dataset of fifty projects the difference would not be measurable yet.

DO NOT SAY

Do not invent a speed-up figure. Saying you reasoned about how each screen reaches its data, and did not benchmark, is a stronger answer than a number you cannot defend.

SAY THIS

A check constraint is enforced by the database itself, so it holds even when the write does not come through my Java code. It could be a script, a psql session, a future import tool, or a bug in my own service. Java validation protects the user interface. The constraint protects the data.

IF THEY PUSH

I have thirty-one of them. `chk_pc_days` says `planned_days` must be greater than zero and at most thirty-one, so nobody can plan forty days inside one month. `chk_jf_pourcentage` keeps a billing milestone between zero and one hundred percent. `chk_status` limits `projects.status` to the five allowed values. `chk_project_dates` requires `end_date` to be on or after `start_date`. The rule I used to decide where something lives is simple. A rule that is true of one single row, always, belongs in the database. A rule that has to look at several rows, or to know who is asking, stays in the service. The clearest example of the second kind is `validatePourcentageSum` in `JalonService`: the percentages of the active milestones of a project must not add up to more than one hundred. A check constraint sees one row at a time and cannot say that, so it lives in Java inside the transaction. The honest cost of database constraints is the error message. When one fires, the user gets a technical error unless I translate it, so most rules are checked in Java as well. The constraint is the last line of defence, not the first.

POINT TO `backend/src/main/resources/db/migration/V7__schema_workload.sql (chk_pc_days, chk_cr_days) · backend/src/main/resources/db/migration/V9__schema_billing.sql (chk_jf_pourcentage) · backend/src/main/resources/db/migration/V5__schema_project.sql (chk_status, chk_initial_budget) · backend/src/main/java/com/pms/billing/service/JalonService.java`

THEN THEY ASK

Give me a rule that could not be a check constraint — The sum of the billing milestone percentages per project. A check constraint sees one row, and that rule needs every row of the project, so it lives in `JalonService` inside the transaction.

DO NOT SAY

Do not say the database "double-checks" Java. Say that Java gives the message and the database gives the guarantee. They are not the same job.

71

Two developers on your team each write a migration numbered V29 at the same time. What happens?

MEDIUM

SAY THIS

Nothing breaks in silence. Git shows the clash the moment the second one merges, because two files both start with V29. Whoever merges second renames his to V30 and reads it again, in case it assumed it ran first. If it ever slipped through, Flyway refuses to start with two scripts at the same version.

IF THEY PUSH

The problem shows up in the pull request, not in the database. Flyway sees a duplicate version and stops instead of picking one, so the worst case is a failed startup, not corruption. What matters is the discipline that keeps it from becoming corruption. In my configuration `out-of-order` is false in both the base and the dev profile. So if V29 has already run on a database and a V28b turns up afterwards, Flyway refuses instead of quietly slipping it in behind. It forces the history to stay in one straight line. On a small team the working rule is to take the next free number when you merge, not when you start writing, and to merge often. That is how I worked. On a bigger team the standard fix is a version built from a timestamp, for example V20260917103000, so two people physically cannot collide.

POINT TO `backend/src/main/resources/application.yml (out-of-order: false) ·`
`backend/src/main/resources/application-dev.yml (out-of-order: false) ·`
`backend/src/main/resources/db/migration/ (linear V1 to V28)`

THEN THEY ASK

And if both versions had already run on two different databases? — Then the two databases hold different content under the same version number, and they must be rebuilt from one history. That is the real reason the number is taken at merge time, not when the work starts.

DO NOT SAY

Do not say "Flyway handles it automatically". Flyway refuses, and a human renames the file. Claiming it is solved on its own shows you have never hit it.

72

Why soft delete rather than really deleting the row, and what does it cost you?

HARD

SAY THIS

I never physically delete a business row. A delete sets `deleted` to true. The reason is that the financial indicators are computed from history, so erasing a month of workload would change a margin that has already been reported. The cost is that unique constraints stop working, and V18 is where that bit me.

IF THEY PUSH

Every entity extends BaseEntity, which carries the deleted flag, and a delete method simply sets it and saves. The problem V18 fixes is concrete. V1 declared UNIQUE on users.email and V5 declared UNIQUE on projects.code. A soft-deleted row is still physically there, so once project S2I-014 was deleted, nobody could ever create S2I-014 again. A dead row was holding the code hostage. V18 drops both constraints and puts partial unique indexes in their place, CREATE UNIQUE INDEX ... WHERE deleted = FALSE, so uniqueness applies only among live rows, which is what the business rule always meant. The same pattern was used from the start on team_assignments, plan_charges, charges_reelles, snapshot_kpis and tcc_annuels. I own the other costs too. The tables grow for ever, and there is no purge job. And every query has to carry deleted = false by hand, so ProjectRepository writes that condition into each @Query. If I forget one, deleted rows come back into a list. Hibernate's @SQLRestriction would apply the filter on its own at entity level, and I did not use it. That is the first improvement I would make, and it is one annotation per entity, not a redesign.

POINT TO backend/src/main/resources/db/migration/V18__soft_delete_unique_indexes.sql ·
backend/src/main/resources/db/migration/V1__schema_auth.sql ·
backend/src/main/java/com/pms/shared/entity/BaseEntity.java ·
backend/src/main/java/com/pms/project/repository/ProjectRepository.java ·
backend/src/main/java/com/pms/project/service/ProjectService.java (delete)

THEN THEY ASK

What if the law made you really erase a person's data? — Soft delete alone would not be enough. I would need an anonymisation routine that overwrites the personal columns while keeping the financial rows. That is a hole in the current design, and the deleted flag cannot fix it.

DO NOT SAY

Do not present soft delete as free. If you skip the unique-constraint problem and the hand-written filter, the examiner will bring them up for you.

73

You are live at ST2i with real data. How do you change the schema without losing any of it?

HARD

SAY THIS

The same way every deployment already works. The container starts, Flyway reads the history table, applies only the missing scripts inside transactions, and the application takes traffic only once that has worked. For a risky change I add a backup first and split it into two deployments, so the old code never breaks.

IF THEY PUSH

The mechanism is already proven on every push. PostgreSQL runs DDL inside transactions, so a script that fails halfway rolls back instead of leaving half a schema. In production, validate-on-migrate is true, so a changed script refuses to run at all. The container health probe holds traffic back, and the smoke job in the CI workflow brings the whole stack up from an empty volume, waits for the backend to report healthy, then logs in through the nginx proxy. That is all twenty-eight migrations running for real on PostgreSQL 17 on every push. For a dangerous change, like renaming or splitting a column, I would not

do it in one step. Expand first: add the new column, fill the existing rows with an UPDATE in the same migration, and write to both. Then deploy the code that reads the new one. Contract later: a separate migration drops the old column once no running instance needs it. The one honest gap is that I have never rehearsed a restore. A backup you have never restored is a hope, not a backup. So `pg_dump` before migrate, plus a restore procedure I have tested, is the first thing I would add before a real production rollout.

POINT TO `.github/workflows/ci.yml` (smoke job: docker compose up, health wait, login through nginx)
• `docker-compose.yml` • `backend/src/main/resources/application-prod.yml` (`validate-on-migrate: true`) • `docs/DEPLOYMENT.md` • Report chapter 7, on startup idempotence

THEN THEY ASK

How do you roll a migration back? — You do not. You roll forward with a new migration. The only real rollback is the backup, and that is exactly why the backup step is the one I would add first.

DO NOT SAY

Do not promise zero downtime or a rehearsed restore. Claiming a disaster-recovery procedure you never ran is the kind of thing a jury will test with one more question.

74

Twenty-eight migrations, thirty-one check constraints, seven partial indexes. What proves that any of it really works?

HARD

SAY THIS

The unit tests do not prove it. They run on an in-memory H2 with Flyway switched off. What proves it is the end-to-end job in CI. It starts a real PostgreSQL 17 from an empty volume, applies all twenty-eight migrations, and passes only if a real login works through the proxy.

IF THEY PUSH

I will be exact about what runs where, because the difference matters. `application-test.yml` uses `jdbc:h2:mem` with `MODE=PostgreSQL`, `ddl-auto` set to `create-drop` and `flyway.enabled` set to `false`. That makes the test suite fast, but it also means H2 builds the schema from the entities. So my check constraints, my partial unique indexes and my data migrations are not exercised by a single unit test. That is a real gap and I will not dress it up. What covers it is the smoke job in `.github/workflows/ci.yml`. It runs `docker compose up` on the full stack, waits up to five minutes for the backend health probe, then posts to `/api/auth/login` through nginx and checks that an access token comes back. A broken migration means an unhealthy container and a red build. But it is coarse. It proves the schema builds and the application runs. It does not prove that one particular constraint rejects one particular bad row. The proper fix is Testcontainers: run the integration tests against a throwaway PostgreSQL 17 with Flyway enabled. That is a dependency and a base test class, not a redesign, and it is what I would add first. To be complete, coverage on the run of 13 September 2026 is 41 percent of lines and 24 percent of branches across everything, including the seeder.

POINT TO

```
backend/src/test/resources/application-test.yml (H2, ddl-auto: create-drop,  
flyway.enabled: false) · .github/workflows/ci.yml (smoke job) · backend/pom.xml  
(h2database, no testcontainers) · Report chapter 7, section Testing, Table of coverage  
and Figure cap_coverage
```

THEN THEY ASK

So could a bad constraint reach production? — A constraint that blocks a legitimate write would fail the smoke job and never ship. A constraint that is only too permissive would pass, and only Testcontainers would catch that.

DO NOT SAY

Do not suggest the migrations are unit-tested. Name H2, name the smoke job, name Testcontainers as the fix. The honesty is what earns the point here.

75

Chapter 3 of your report says amounts are numeric(18,3) with a currency column, and that soft delete uses an active flag and a deleted_at timestamp. I opened your migrations and I see NUMERIC(15,2) and a boolean. Which one is your database?

HARD

SAY THIS

The migrations are. Chapter 3 is the design chapter, and it carries the conventions I wrote before I built anything. The database that exists uses NUMERIC(15,2), TIMESTAMP and one boolean called deleted, and chapter 4 describes that correctly. Where the two disagree, Flyway is the truth.

IF THEY PUSH

I can list the differences instead of waving them away. Money: designed as numeric(18,3), built as NUMERIC(15,2), because ST2i works in Tunisian dinars, where two decimals is the millime, and no contract comes near ten thousand billion. Timestamps: designed as timestamptz, built as TIMESTAMP with the application pinned to UTC through the Jackson configuration. That is the weaker of the two choices, and timestamptz is what I would use again. Soft delete: designed as an active flag plus a deleted_at timestamp, built as one boolean deleted on BaseEntity, which loses the date of the deletion. Table names: the design listed currencies, exchange_rates, workload_plan, actual_workload, kpi_snapshots and project_manager_assignments. What exists is plan_charges, charges_reelles and snapshot_kpis. The currency and its rate are carried on the project row itself, and project_manager_assignments became the direct foreign key projects.chef_projet_id. None of this is hidden: docs/DATABASE_DESIGN.md carries a dated as-built warning from 5 July 2026 that names exactly which design tables were never built and says the Flyway migrations win. What I should have done is carry that correction back into the convention table of chapter 3 before depositing. It is documentation debt, not a defect in the schema.

POINT TO report/chap_03.tex, "Modelling conventions" and "Auditability and logical deletion" · report/chap_04.tex, "Base entity and automatic auditing" · docs/DATABASE_DESIGN.md, sections 1 and 2 (as-built warning, 2026-07-05) ·

```
backend/src/main/java/com/pms/shared/entity/BaseEntity.java ·  
backend/src/main/resources/db/migration/V18__soft_delete_unique_indexes.sql
```

THEN THEY ASK

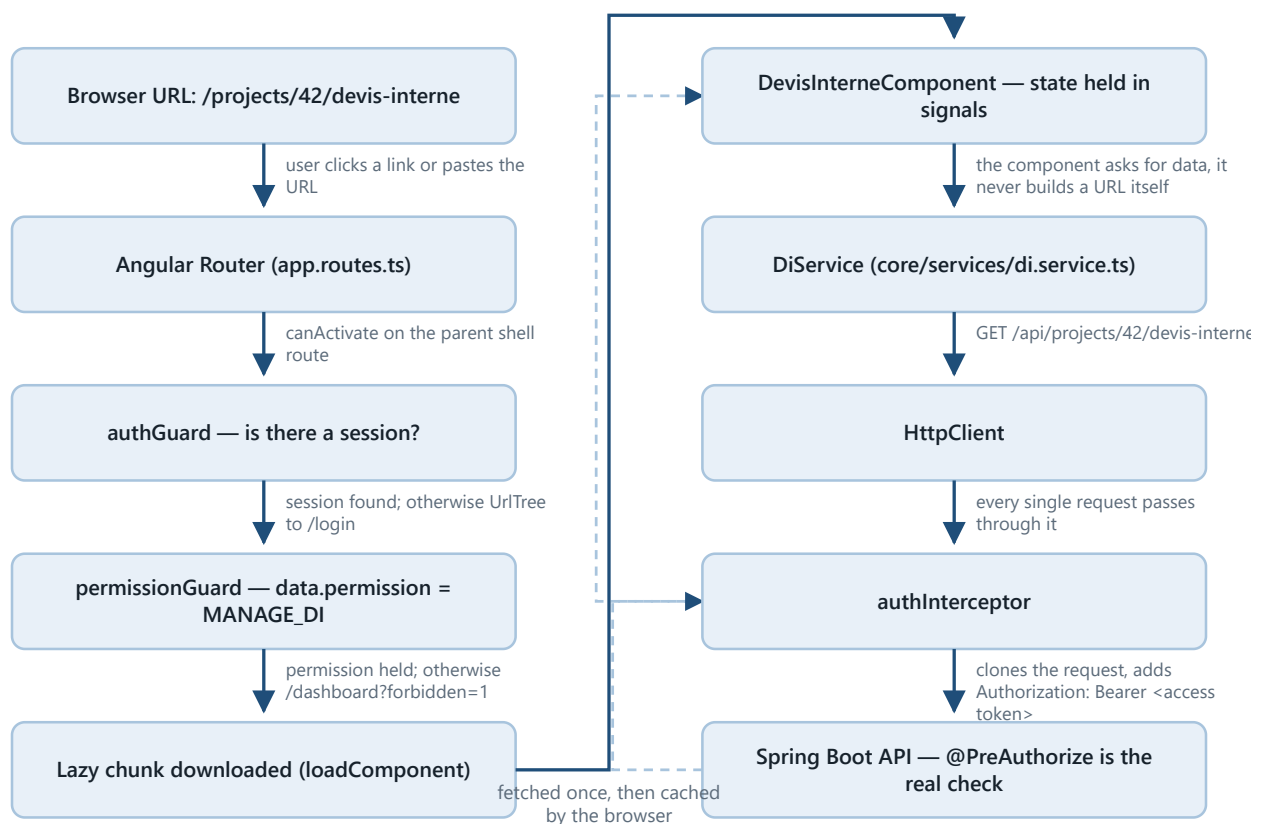
So which document should the jury trust? — The migrations first, then chapter 4, then the as-built section of DATABASE_DESIGN.md. The convention table in chapter 3 is design intent, and I would rather say so than defend it.

DO NOT SAY

Do not argue that the two descriptions mean the same thing. Admit the list item by item, then say where the truth lives. An examiner who has the file open will not accept anything else.

The Angular application and the interface decisions

When a jury asks about the frontend, they are rarely asking "do you know Angular". They are checking two things. Can you explain a choice made in the browser without hiding behind framework words? And do you know where the browser stops being trustworthy? So answer every question with the concrete file and the concrete number. Be the first person in the room to name the bundle overrun and the missing frontend tests. A limit you disclose reads as engineering judgement. A limit they find for themselves reads as a blind spot.



One click, from URL to API — Drawing this proves the point the jury is testing. The two guards sit before the component and only shape navigation. The one arrow that really carries authority is the interceptor's token going into the server's @PreAuthorize check.

SAY THIS

I chose Angular because it ships the whole application layer in one versioned box: router, forms, HTTP client, dependency injection. I was one developer with ten sprints. Every hour I did not spend choosing and gluing libraries went into the financial engine instead.

IF THEY PUSH

The report gives this reason directly in chapter 3. Angular is opinionated, so a single developer spends his time on the domain instead of on architecture decisions that are already made for him. There is a second reason that mattered day to day. The backend is Java 21 and strongly typed, and TypeScript gives me the same discipline on the client. The eleven model files in `core/models` mirror the backend DTOs, so when a field is renamed on the server, the frontend build fails instead of quietly showing undefined. I am running Angular 21.2 with TypeScript 5.9. The honest trade-off is weight. Angular's baseline is heavier than React or Svelte, and that shows in my initial bundle, which I disclose in Figure 7.5. I accepted that cost because this is an internal desktop tool on a company network, not a public site competing on first paint.

POINT TO `frontend/pms-frontend/package.json` · `report/chap_03.tex`, section 'Frontend stack' (Angular 21) · `frontend/pms-frontend/src/app/core/models/` (11 model files)

THEN THEY ASK

Why not React? — React is a view library. I would have had to pick a router, a forms library, an HTTP client and a dependency injection story myself, and then defend four separate choices instead of one.

DO NOT SAY

Do not say Angular is 'better' than React. You cannot prove it and the jury will make you try. Say it fitted a solo developer on a fixed deadline.

SAY THIS

A standalone component declares its own imports at the top of its own file. There is not one NgModule in this project. Each component says exactly what it needs, and that is the whole wiring.

IF THEY PUSH

Concretely: `ProjectPickerComponent` declares imports of `CommonModule`, `FormsModule`, `PaginationComponent` and `TranslocoModule`, and that is all anyone has to read to know what it depends on. The application starts with `bootstrapApplication(App, appConfig)` in `main.ts`, not with a

root module. What used to sit in an NgModule's providers array now sits in app.config.ts, which is eighteen lines: provideRouter with the routes, provideHttpClient with the auth interceptor, provideI18n for Transloco, and a LOCALE_ID factory. There are 66 TypeScript files in src and zero NgModules. The real gain is not elegance, it is bundling. Because each lazily loaded component pulls in only the imports it declares, the build tool can split the code per route instead of per module. That is what makes the 18 lazy routes really produce 37 separate chunks instead of one blob.

POINT TO frontend/pms-frontend/src/main.ts · frontend/pms-frontend/src/app/app.config.ts · frontend/pms-frontend/src/app/shared/project-picker/project-picker.component.ts · frontend/pms-frontend/src/app/shared/pagination/pagination.component.ts

THEN THEY ASK

So what replaced the module providers? — app.config.ts: provideRouter, provideHttpClient(withInterceptors([authInterceptor])), provideI18n and LOCALE_ID, all in one small file.

DO NOT SAY

Do not describe it as 'the new way of writing components'. That says nothing. Say the component owns its own list of dependencies, and that this is what makes code splitting per route possible.

78

What does lazy loading give you here, and how do you know it works?

EASY

SAY THIS

Eighteen of my routes use loadComponent, so the code of a screen is downloaded the first time somebody opens it. The file every user pays for is 22 kilobytes. The heaviest screen is 205 kilobytes, and only the people who open it ever download it.

IF THEY PUSH

In app.routes.ts every feature route is declared with loadComponent and a dynamic import, so the router fetches the chunk on the first navigation and keeps it after that. I know it works because the production build prints it: the output directory holds 37 JavaScript chunks, main.js is 22.3 kB, and the biggest lazy chunk is 205 kB. That matters because the screens are not small. project-detail alone is 64 kB of source, governance 41 kB, agile 40 kB. A developer who only opens the dashboard and the workload plan never downloads governance, billing, KPIs or the Devis Interne. I must be exact about what I measured. I measured transferred bytes from the build report, not page-load milliseconds. I never ran a throttled-network test or Lighthouse, so I will not put a number in seconds on it.

POINT TO frontend/pms-frontend/src/app/app.routes.ts (18 loadComponent routes) · frontend/pms-frontend/dist/pms-frontend/browser/ (37 chunks, main 22.3 kB, largest chunk 205 kB) · report/chap_07.tex, Figure 7.5

THEN THEY ASK

Did you measure the time a user actually saves? — No. I measured bytes from the build report. A measurement on a slowed-down network is what I would add before claiming a number the user can feel.

DO NOT SAY

Do not invent a load-time gain in seconds. You measured bytes, and the jury can ask to see the measurement.

79

You say you use signals. What is a signal, and why did it replace the older change detection?

MEDIUM

SAY THIS

A signal is a value that remembers who reads it. When it changes, Angular redraws only the parts of the screen that read it. I run with no Zone.js at all. The library is not even installed in this project.

IF THEY PUSH

The old mechanism worked by suspicion. Zone.js patched `setTimeout`, `addEventListener` and `XHR`, and after any of them fired it told Angular 'something happened somewhere, go and check every binding in the tree'. Signals turn that around. In `AuthService`, `_context` is a signal, and `isLoggedIn` and `permissions` are computed values built from it, so Angular knows exactly which views are out of date when a user logs in. The clearest example is the projects list. `filtered`, `sorted` and `paged` are three computed signals chained together, so typing one letter in the search box recomputes the table body and nothing else. Not the sidebar, not the KPI cards. Signals also replace lifecycle glue: `ThemeService` uses an effect that writes the `data-theme` attribute and `localStorage` whenever the theme signal changes, and there is no subscription to clean up. I should be careful with my claim here. I did not profile render time before and after, so what I can defend is the change in structure, not a performance number.

POINT TO `frontend/pms-frontend/src/app/core/services/auth.service.ts` (signal, computed) · `frontend/pms-frontend/src/app/features/projects/project-list/project-list.component.ts` (filtered/sorted/paged) · `frontend/pms-frontend/src/app/core/services/theme.service.ts` (effect) · `frontend/pms-frontend/package.json` (no zone.js dependency)

THEN THEY ASK

Are signals just RxJS under another name? — No. RxJS models a stream over time, and I still use it for HTTP. A signal models a current value and is read straight away, which is exactly what a template needs.

DO NOT SAY

Do not claim signals made the application 'faster'. You never profiled it. Describe what changed in the structure instead.

80

How does the sidebar know not to show a menu entry the user cannot open?

MEDIUM

SAY THIS

Every menu item carries a permission code. One computed signal filters the menu against the permission list the login response returned, and a whole section disappears when none of its items survive. Nothing is hidden with CSS.

IF THEY PUSH

In `sidebar.component.ts` the navigation is a plain array of sections, and each item can carry a permission such as `VIEW_BILLING` or `MANAGE_ROLES`. The computed `visibleSections` filters the items against `auth.permissions()`, then drops the sections that end up empty, so a developer does not see an empty 'Administration' heading. Thirteen different permission codes drive the menu. The codes are not invented on the client. The login response carries the user's permission list, the same codes appear in the route data of `app.routes.ts`, and the same codes appear in the 110 `@PreAuthorize` annotations on the backend controllers. Chapter 4 calls this the dynamic menu: the interface follows the authorization policy on its own, and no authorization rule is duplicated on the client. The practical result is that the same build of the application looks different for an administrator, a director and a developer, which is what the three screenshots in chapter 4 show.

POINT TO `frontend/pms-frontend/src/app/layout/sidebar/sidebar.component.ts` (`visibleSections`) · `frontend/pms-frontend/src/app/core/services/auth.service.ts` (`permissions` computed) · `report/chap_04.tex`, 'Dynamic menu' and Figures `ui-perm-catalogue` to `ui-dev-dashboard`

THEN THEY ASK

What if I edit my permissions in localStorage and reload? — The menu shows more entries, and every one of them gets a 403, because the server reads the signed token, not my browser storage.

DO NOT SAY

Do not call the hidden menu a security feature. It is there for comfort. Say the codes are shared with the server, and the server is what enforces them.

81

You have route guards. Are those your security?

MEDIUM

SAY THIS

No. A guard only decides if the router shows a screen. It runs on the user's own machine, so the user can break it. The real check is on the server: 110 `@PreAuthorize` annotations across 18 controllers. The guard is there to be helpful. It is not a lock.

IF THEY PUSH

authGuard looks if a session exists. If there is none, it returns a UrlTree that sends the user to /login. permissionGuard reads the code the route needs from the route data, then it calls auth.hasPermission. If the check fails, it sends the user to /dashboard with a forbidden flag. The dashboard then shows a banner that tells the user why he was sent back, so he is not left confused. All of that is JavaScript that I sent to the browser. Anyone can open the developer tools, set the permission array to whatever he likes, and open the route. But he still cannot get the data. The JwtAuthenticationFilter takes the permission codes inside the signed token and turns them into Spring authorities, and every endpoint has an annotation. So the job of the guard is to stop honest users from hitting dead ends. The job of the server is to stop dishonest ones. I also have a third guard that has nothing to do with security. It is unsavedChangesGuard, and it asks the user to confirm before he leaves a project form with unsaved changes.

POINT TO frontend/pms-frontend/src/app/core/guards/auth.guard.ts · frontend/pms-frontend/src/app/core/guards/permission.guard.ts · frontend/pms-frontend/src/app/core/guards/unsaved-changes.guard.ts · frontend/pms-frontend/src/app/features/dashboard/dashboard.component.ts (forbidden banner) · backend/src/main/java - 110 @PreAuthorize annotations over 18 controllers

THEN THEY ASK

Then why write guards at all? — Without them, a developer who clicks /billing gets a screen full of failed requests and error pop-ups, instead of a clean redirect that tells him the reason.

DO NOT SAY

Never say 'the guard protects the route'. That is exactly the phrase the jury is waiting for, and it is wrong.

82

Why did you build a search pop-up with pages to pick a project, instead of a simple dropdown?

MEDIUM

SAY THIS

A dropdown shows one line at a time, and it shows nothing but a name. My demo database already holds 50 projects, and the real archive of ST2i is bigger. So I built a pop-up with search, filters, favourites, recent items and a table of twelve rows per page.

IF THEY PUSH

The picker does not open as a browser, and that is on purpose. When nothing is chosen, it shows up to twelve shortcut cards, sorted by how useful they are: first the projects I manage, then recent ones, then favourites, then active ones. When a project is chosen, it shows a context card. That card has the code, the status, the manager, the team size, the budget and the period. So the user can see he is on the right project without opening anything. The full browser only opens when he asks for it. Its search looks at the code, the name, the client and the manager at the same time, and there are two filter chips: 'assigned to me' and 'favourites'. Favourites and recent items are kept in localStorage. The page size is twelve, and the pagination bar is the same shared component that the projects list uses. Here is the honest limit. The list is loaded once with size=1000 and then filtered inside the browser. That is right

with 50 projects, and it is still right with a few hundred, because one trip to the server for every key press would feel slower. Past a few thousand it stops being right, and ProjectService already offers a paged list computed on the server for that day.

POINT TO `frontend/pms-frontend/src/app/shared/project-picker/project-picker.component.ts` · `frontend/pms-frontend/src/app/shared/pagination/pagination.component.ts` · `frontend/pms-frontend/src/app/core/services/project.service.ts` (`listAll` uses `size=1000`; `list(page,size,sort)` exists) · `backend/src/main/resources/db/migration/V26__seed_demo_data.sql` (50 seeded projects)

THEN THEY ASK

Why filter in the browser rather than on the server? — At this size, filtering in the browser is faster than one request for every key press. The paged endpoint on the server already exists, and I would move to it when the data is big enough to be worth the extra trip.

DO NOT SAY

Do not just say 'a dropdown does not scale'. Name what the picker gives that a dropdown cannot: search over four fields, favourites, recent items and a context card.

83

Walk me through what happens to an HTTP request. How is the token added, and what happens when it has expired?

MEDIUM

SAY THIS

One function sits between every request and the network. It copies the request and adds the Authorization Bearer header. If the answer is 401, it refreshes the token one time and sends the first request again with the new token.

IF THEY PUSH

The interceptor is registered one time, in `app.config.ts`, through `provideHttpClient` with `withInterceptors`. In Angular a request cannot be changed, so the interceptor makes a copy of the request and sets the header on the copy. When a 401 comes back, it first checks that the failed call was not `/auth/refresh` or `/auth/logout` itself. Without that check it would loop for ever. Then it calls `handleTokenRefresh` and uses `switchMap` to send the first request again with the fresh token. The part I am most careful about is what happens when several calls fail together. The dashboard fires several requests at the same time, so when the access token expires they all get a 401 at once. `AuthService` keeps an `isRefreshing` flag and a `BehaviorSubject`, so exactly one `/auth/refresh` call leaves the browser. The other callers wait on that subject and reuse its result. If the refresh itself fails, `expireSession` clears the storage and sends the user to `/login` with an 'expired' flag, so he is told why instead of being dropped on a login form with no explanation. There is one more case. The server answers 403 with the code `FIRST_LOGIN_REQUIRED`, and then the interceptor opens the change-password screen. The access token lives fifteen minutes in production. The refresh token is a seven-day `HttpOnly` cookie, and that is why those calls use `withCredentials`.

POINT TO

```
frontend/pms-frontend/src/app/core/interceptors/auth.interceptor.ts · frontend/pms-frontend/src/app/core/services/auth.service.ts (handleTokenRefresh, single-flight) · frontend/pms-frontend/src/app/app.config.ts · backend/src/main/resources/application-prod.yml (access 900s, refresh 604800s)
```

THEN THEY ASK

Why is the access token in localStorage if an injected script can read it? — Because it dies after fifteen minutes, and the long-lived refresh token is HttpOnly. The stricter design keeps both tokens away from JavaScript, and that is the change I would make next.

DO NOT SAY

Do not skip the part about several calls at once. If you only say 'it tries again on 401', the next question is what happens when five calls fail together, and the single-flight refresh is your best answer here.

84

The company works in French. How do you handle French and English in the interface?

MEDIUM

SAY THIS

I use Transloco, and the word lists are static JSON files: 977 keys for each language, split into seven files, so a screen only downloads its own words. When you switch language, the screen is drawn again at once, with no page reload.

IF THEY PUSH

The setup says that French is both the default language and the fallback language, and useFallbackTranslation is on. So a key that is missing in English shows the French text instead of a raw key on the screen. The loader fetches /i18n/fr.json for the main word list, and paths like /i18n/project/fr.json for the six scopes: admin, agile, auth, dashboard, project and resources. LanguageService chooses the language one time at startup, through an application initializer. It looks at localStorage first, then at the browser language, then at French. There is one detail that is easy to get wrong, and I handled it on purpose. The date pipe and the number pipe of Angular read LOCALE_ID, not Transloco. So app.config.ts sets LOCALE_ID from the same storage key before the first screen is drawn, and main.ts registers the French locale data. Without that, the text would be French while the dates and the thousand separators stayed American. To add a third language, you drop in the JSON files and add one entry to availableLangs. No component has to change. One thing I would fix: prodMode is still false in the Transloco config, so missing keys are still written to the log in production.

POINT TO frontend/pms-frontend/src/app/core/i18n/transloco.providers.ts · frontend/pms-frontend/src/app/core/i18n/language.service.ts · frontend/pms-frontend/src/app/core/i18n/transloco-loader.ts · frontend/pms-frontend/public/i18n/(fr.json and en.json, 977 keys each, plus 6 scoped folders) · frontend/pms-frontend/src/app/app.config.ts (LOCALE_ID factory)

THEN THEY ASK

Why is the language not stored on the user's profile in the database? — It is a preference of the device, it has to work on the login screen before anyone has signed in, and keeping it in the browser means the backend does not need to know anything about language.

DO NOT SAY

Do not say 'I used a translation library' and then stop. The `LOCALE_ID` point is the one that shows you understood that translating text and formatting numbers are two different problems.

85

Is the interface easy to use for people with disabilities, and does it work on a phone?

HARD

SAY THIS

I did the basic things on purpose, and I did not run an audit. Buttons that show only an icon carry aria-labels, the command palette works fully with the keyboard, and I respect the reduced-motion setting. I never used a screen reader and never ran an automatic audit, so I will not claim that the app follows WCAG.

IF THEY PUSH

What exists is precise. Every icon-only control in the sidebar, in the pagination bar and in the picker carries an aria-label. The Ctrl or Command plus K palette answers to arrow up, arrow down, Enter and Escape, so a user can reach any page and any project without a mouse. The picker pop-up closes on Escape and puts the cursor in its search field when it opens. There is a focus-visible outline rule in the stylesheet, and there are reduced-motion media queries in the picker styles and the pagination styles that switch the hover animations off. The language service sets the lang attribute on the document, so a screen reader would say the words in the right language. Here is what does not exist: no automatic accessibility run, no test of the focus trap inside the pop-up, no contrast check of the dark theme, and only one focus-visible rule instead of a full plan for focus. About screen sizes, the stylesheet has two breakpoints, plus Bootstrap column classes that hide the less important table columns below the large and extra-large widths, so tables stay readable on a laptop or a tablet. This is a desktop-first internal tool, for people sitting at a workstation and replacing an Excel workbook. I would draw that line in the same place again. But I will not call it accessible until it has been tested.

POINT TO `frontend/pms-frontend/src/app/layout/sidebar/sidebar.component.ts` (onKey, aria-label, Ctrl+K) · `frontend/pms-frontend/src/app/shared/project-picker/project-picker.component.ts` (keydown.escape, prefers-reduced-motion) · `frontend/pms-frontend/src/app/shared/pagination/pagination.component.ts` (aria-label on every control) · `frontend/pms-frontend/src/styles.scss` (focus-visible rule; breakpoints at 768px and 769-1024px) · `frontend/pms-frontend/src/app/core/i18n/language.service.ts` (document.documentElement.lang)

THEN THEY ASK

If it is not audited, how can you say the keyboard works? — Because I can show it live: open the palette with Ctrl+K, press arrow down to a project, press Enter, and never touch the mouse.

DO NOT SAY

Do not say 'it is accessible' and do not name a WCAG level. You have no audit, and a claim like that invites them to ask for the report you do not have.

86

Your build says the bundle is almost 200 kilobytes over its own budget. Explain that.

HARD

SAY THIS

The first bundle is 694.76 kilobytes against a budget of 500 kilobytes, so it is 194.76 over. It is in the report as Figure 7.5. I did not hide it. And it is not JavaScript: 333 kilobytes of it is CSS, because I import all of Bootstrap and all of Bootstrap Icons.

IF THEY PUSH

You can check the split in the build output folder: the stylesheet alone is 333.5 kB, while the main JavaScript file is 22.3 kB. The cause is two lines at the top of styles.scss. One imports the whole Bootstrap SCSS, which pulls in every component, even the ones I never draw. The other imports the whole Bootstrap Icons stylesheet, for a set of about forty icons that I really use. In angular.json I set the budget as a warning and not as an error, and I did that on purpose, so the number shows up on every single build instead of blocking the pipeline over something that is only cosmetic. The fix is known and small: import only the Bootstrap parts the application uses, and cut the icon font down to the icons I need. I expect that removes most of the extra weight. I did not do it because Sprint 10 had to deliver a working pipeline, two container images and a stack that answers, and trimming CSS does not change whether the platform computes a margin correctly. I would draw that line in the same place again, because extra weight that is measured and said openly is a backlog item, while extra weight that is never measured is a defect nobody knows about.

POINT TO report/chap_07.tex, Figure 7.5 (fig:cap-frontend-build), 694.76 kB and 194.76 kB over budget · frontend/pms-frontend/angular.json (initial budget: 500kB warning, 1MB error) · frontend/pms-frontend/src/styles.scss lines 6-7 (full Bootstrap SCSS + Bootstrap Icons) · frontend/pms-frontend/dist/pms-frontend/browser/styles-QRGBQRPY.css (333.5 kB) vs main.js (22.3 kB)

THEN THEY ASK

Why not raise the budget to 700 kB so the warning goes away? — Because then nobody ever sees the number again. A budget is only useful while it still hurts a little.

DO NOT SAY

Do not blame Angular for the size. Your JavaScript is small and your CSS is not, and anyone can check that in ten seconds.

87

You report 97 tests and 41 per cent coverage. How many of them cover the Angular application?

HARD

SAY THIS

None. There is not one test file in the Angular project. The 97 tests and the 41 per cent of lines are backend only. I tested the frontend by using it, and that is the weakest part of what I delivered.

IF THEY PUSH

The choice is visible in the configuration, not only in the missing files: `angular.json` sets `skipTests` to `true` on every schematic, so it was decided at the start of the project. What I did instead is not nothing. The CI pipeline runs the production build on every push, which catches type errors and template errors. It also caught one real defect, which I describe in chapter 7: a production bundle that was still pointing at the development API address. Using the app by hand on my own machine had hidden that completely. TypeScript removes another family of bugs, because the model files copy the backend DTOs, so a renamed field on the server breaks the build. And the logic whose failure would cost money, the margin and the EVM calculation, lives on the server. That is where the 97 tests, the 41 per cent of lines and the 24 per cent of branches really are. What is truly untested is the guard redirect logic, the single-flight refresh in the interceptor, and the ordering rules of the picker. All three are pure functions with no DOM, so they are the cheapest tests that exist, and they are the first thing I would write.

POINT TO `frontend/pms-frontend/src` – no `.spec.ts` files exist • `frontend/pms-frontend/angular.json` (`skipTests: true` on all schematics) • `report/chap_07.tex`, Table `tab:coverage` and Figure 7.2 (41 per cent lines, 24 per cent branches, backend) • `report/chap_07.tex` (production bundle pointing at the development API, caught by the build job)

THEN THEY ASK

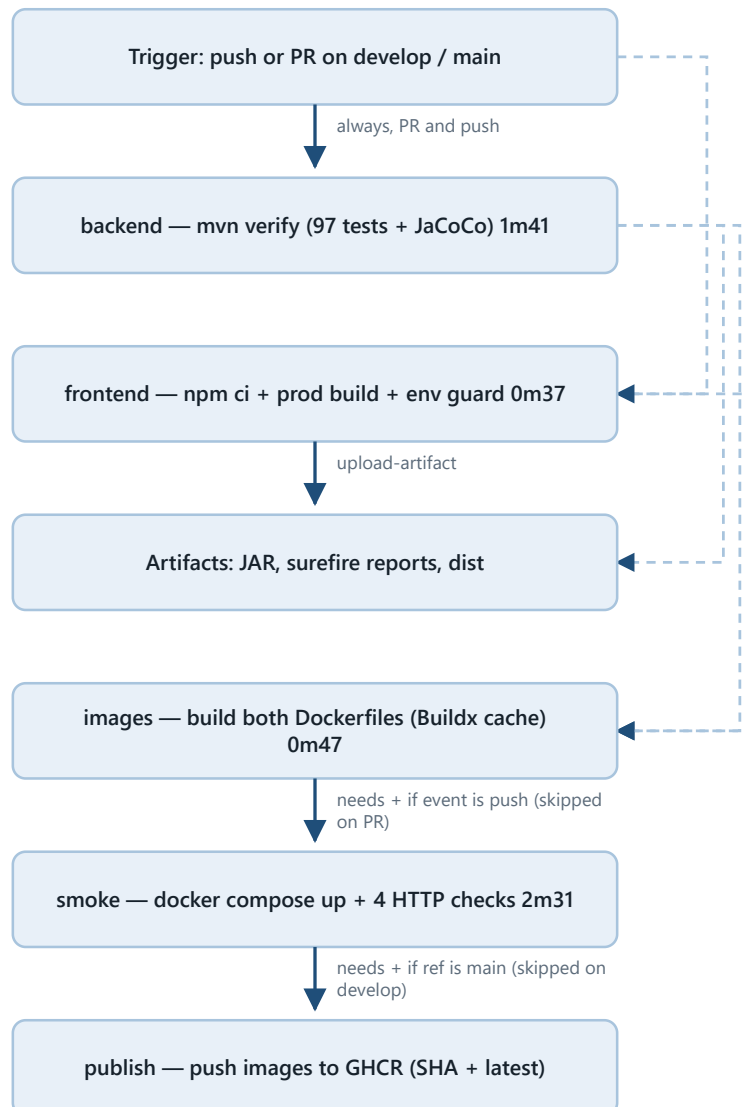
With one more sprint, what would you test first? — The 401 path in the interceptor, because it is the only piece of frontend logic whose failure logs out every user at the same time.

DO NOT SAY

Do not present using the app by hand as a way of testing. Name it as a gap, then name the three tests you would write and say why the interceptor comes first.

Testing, CI/CD and Deployment — how he proves it works and how it ships

This is where the jury stops asking what he built and starts asking whether it can be trusted. They are testing engineering maturity, not the number of tests. Do you know the difference between a test and a number? Do you know your own weak spots? Can you name them before they do? The posture is full honesty with a clear boundary: say the real figure first, then explain why that was the right place to stop. A defensive answer about 41 percent coverage costs more marks than the 41 percent itself.



The five CI jobs and what gates what — Drawing this shows the jury that the quality gates run on every change, while the slow end-to-end job and the publication step only run under a condition. That is why a full run costs five minutes thirty-nine and not fifteen.

88

Can you explain the difference between a unit test and an integration test, and tell us which kind you wrote?

EASY

SAY THIS

A unit test checks one class on its own, with everything around it replaced by fakes. An integration test starts the real application and goes through HTTP, the security filter and the database. I wrote both: five unit classes and eight integration classes, 97 tests in total.

IF THEY PUSH

The five unit classes use Mockito. `JalonServiceTest`, for example, replaces the repositories with mocks and checks the billing-milestone rules on their own: all 34 unit tests finish in 2.8 seconds. The eight integration classes carry `@SpringBootTest`, `@AutoConfigureMockMvc`, `@ActiveProfiles("test")` and `@Transactional`. Each one creates a test user, calls `POST /api/auth/login` to get a real JWT (a signed ticket that proves who you are), then calls the endpoint with that token. Those 63 tests take 59 seconds, because every class starts a Spring context. I split them on purpose. I write a unit test when the rule is arithmetic or a guard clause. I write an integration test when the thing I must prove is the permission check, because a permission check only exists when the whole filter chain really runs. `@Transactional` rolls each test back, so the in-memory database stays clean between tests.

POINT TO `backend/src/test/java/com/pms/billing/service/JalonServiceTest.java` ·
`backend/src/test/java/com/pms/project/ProjectControllerTest.java` ·
`backend/src/test/java/com/pms/TestFixtures.java` · `report/chap_07.tex` – Sprint 9,
 subsection Testing, Figure `fig:cap-test-run`

THEN THEY ASK

Why not write integration tests for everything, since they prove more? — Because each one costs a Spring context, which takes seconds, and it hides which layer broke. My 34 unit tests run in under three seconds and they point straight at the rule that failed.

DO NOT SAY

Do not call the controller tests "unit tests of the controller". They start the whole application, and a jury member who knows Spring will catch that at once.

89

How many tests do you have, and what do they actually check?

EASY

SAY THIS

Ninety-seven tests in thirteen classes, all green, one minute and seven seconds. I wrote them to check two things: that an endpoint refuses a caller who does not hold the right permission, and that the business rules the Excel workbook states but cannot enforce. I did not write them to reach a coverage number.

IF THEY PUSH

The clearest permission test is in `ProjectControllerTest`: a user who holds only `VIEW_PROJECT` gets 200 on the project list and 403 on the project creation, inside the same test. Across the whole suite there are eleven assertions of 401 and three of 403. The business rules are the other half.

`ChargeReelleServiceTest` covers rule BR-033: a developer may submit his own workload but not another person's, and a validator may submit for someone else. `JalonServiceTest` refuses a billing milestone that would push the sum of the percentages over 100. `AuthControllerTest` checks refresh-token rotation: after one refresh the first token is dead, and after logout the next refresh returns 401. These are exactly the rules a spreadsheet writes in a comment cell and then cannot stop anyone from breaking.

POINT TO `backend/src/test/java/com/pms/auth/AuthControllerTest.java` ·
`backend/src/test/java/com/pms/workload/ChargeReelleServiceTest.java` ·
`backend/src/test/java/com/pms/billing/service/JalonServiceTest.java` · `report/chap_07.tex`
– Table `tab:coverage` (tests and coverage per module)

THEN THEY ASK

Do the tests cover every one of your endpoints? — No. There are 108 endpoint mappings across 18 controllers, and my suite reaches 11 of those controllers. Six have zero covered lines, and I can name them.

DO NOT SAY

Do not say the suite covers "all endpoints". There are 108 mappings and six controllers are untouched, so anyone can check that claim and find it false.

90

You mention JaCoCo in your report. What does it measure, and what is your figure?

EASY

SAY THIS

JaCoCo attaches an agent to the test JVM and records which lines and branches really ran. My figure for the whole backend is 41 percent of lines, 32 percent of instructions and 24 percent of branches. The modules that carry money and permissions sit between 51 and 81 percent. Everything else pulls the average down.

IF THEY PUSH

The exact counts are 1661 of 4036 lines and 348 of 1419 branches. Module by module: `auth` is at 81 percent of lines, `kpi` 79, `team` 77, `agile` 75, `billing` 75, `project` 72, `workload` 62 and `governance` 51. Then it drops off a cliff: `shared` is at 10 percent, `user` at 4.6 and `mission` at 3.9. The plugin is `jacoco-maven-plugin` 0.8.12, wired with `prepare-agent` and a report goal tied to the test phase. So the same `mvn verify` that the CI runs also produces the report, and I never make it by hand. My thesis prints this table and the raw JaCoCo screenshot side by side, exactly so the reader can check my arithmetic against the tool.

POINT TO `backend/pom.xml` – `jacoco-maven-plugin`, `jacoco.version 0.8.12` ·
`backend/target/site/jacoco/jacoco.csv` · `report/chap_07.tex` – Table `tab:coverage` and
Figure `fig:cap-coverage`

THEN THEY ASK

Which is the more worrying number, 41 percent lines or 24 percent branches? — Branches. It tells me that even where my tests do run the code, they often take only one side of the condition. That is the gap I would close first.

DO NOT SAY

Do not round 41 up to "around half". Giving the exact figure is what buys you credit for the rest of the answer.

91

Your tests run against H2, not PostgreSQL. Is that not a problem?

MEDIUM

SAY THIS

It is a real cost and I made the trade with my eyes open. H2 in memory, in PostgreSQL mode, means the CI job needs no database service and the suite finishes in about a minute. The price is that Flyway is switched off in the test profile, so my 28 production migrations are never run by JUnit.

IF THEY PUSH

application-test.yml sets flyway.enabled to false and ddl-auto to create-drop, so Hibernate builds the test schema straight from the entity classes. That means a broken migration would pass the whole test suite and would only fail when a real container starts. What catches that instead is the smoke job in the pipeline. It runs docker compose up against a real postgres:17-alpine, which applies all 28 migrations, including the 1098-line demo seed, and the job fails if the backend never reports healthy. So the migration path is tested, by the pipeline and not by JUnit. If I built this again I would use Testcontainers, which starts a throwaway real PostgreSQL for the test run. I did not use it, because it makes Docker a hard requirement on every developer machine and it roughly doubles the time the suite takes, and the pipeline already gave me the same signal one layer up.

POINT TO backend/src/test/resources/application-test.yml ·
backend/src/main/resources/db/migration/ - 28 .sql files, V1 to V28 ·
.github/workflows/ci.yml - job smoke · .github/workflows/ci.yml - backend job comment on
@ActiveProfiles("test") and H2

THEN THEY ASK

So an H2 test could pass on SQL that PostgreSQL rejects? — Yes, and that is exactly why the pipeline has a job that starts the real database and refuses to go green until the backend is healthy on it.

DO NOT SAY

Do not say that H2 in PostgreSQL mode is the same as PostgreSQL. It is a compatibility mode, not the same engine, and if you overstate it you invite a follow-up you cannot win.

92

Forty-one percent coverage. Does that mean your application is forty-one percent correct?

MEDIUM

SAY THIS

No, and the two numbers have nothing to do with each other. Coverage says a line ran. It does not say anything was checked. I could reach ninety percent tomorrow by calling every getter with no assertion at all, and the application would be exactly as correct as it is today.

IF THEY PUSH

There is a concrete example in my own figures. Of the 2375 lines that are not covered, 1119 — that is 47 percent of everything not covered — sit in four demonstration data seeder classes.

EnterpriseDataSeeder on its own is 533 lines. Those classes exist to fill the database with the 50 projects and 36 users you see in my screenshots. They run once at startup and there is nothing in them worth asserting. If I take them out, the figure becomes 56.5 percent. I did not take them out in the report, because setting up JaCoCo to hide my worst numbers is exactly the move that makes a coverage figure worthless. The number I actually act on is the branch figure, 24.5 percent against 41.2 percent of lines. That gap says my tests walk through the code but often only down the happy path, and that is a statement about quality, which line coverage is not.

POINT TO backend/target/site/jacoco/jacoco.csv — EnterpriseDataSeeder 533 missed lines, DemoDataSeeder 415, AgileDemoSeeder 136, DataInitializer 35 · backend/src/main/resources/db/migration/V26__seed_demo_data.sql — 1098 lines, 50 projects, 36 users · report/chap_07.tex — Sprint 9 Testing: "It was not written to reach a percentage"

THEN THEY ASK

Then what would you use instead of coverage? — Mutation testing, with a tool like PIT. It changes the code on purpose and fails if no test notices, so it measures whether the assertions are real, not whether the lines ran.

DO NOT SAY

Do not attack coverage as an idea to get away from a low number. Say your number, then explain exactly what it proves and what it does not.

93

Walk us through your continuous integration pipeline, job by job.

MEDIUM

SAY THIS

Five jobs. backend and frontend run at the same time on every change. images builds both containers once those two are green. smoke starts the whole stack and signs in for real. publish pushes the images to the registry, and only from main. A full run takes five minutes and thirty-nine seconds.

IF THEY PUSH

backend runs `mvn verify` on Java 21 with the Maven cache. It compiles, runs the 97 tests, writes the JaCoCo report, then uploads the JAR and the surefire reports as artifacts: one minute forty-one. frontend runs `npm ci` and a production Angular build in thirty-seven seconds, plus a guard step. images uses Buildx with a GitHub Actions layer cache to build both Dockerfiles, forty-seven seconds. It declares needs on backend and frontend, so a failing test never produces an image. smoke only runs on a push, because it is too slow for every pull request. It creates a throwaway `JWT_SECRET` and database password, runs `docker compose up`, polls container health for up to five minutes, then runs four HTTP checks through nginx: a real login, a deep Angular route and the translation catalogue. That takes two minutes thirty-one. publish only runs on the main branch, and it pushes both images to GHCR tagged with the commit SHA and with latest. There is also a concurrency group with cancel-in-progress, so pushing twice does not leave two runs racing each other.

POINT TO `.github/workflows/ci.yml` · `docs/DEPLOYMENT.md` – section 9, job matrix by branch · `report/chap_07.tex` – Figure `fig:ci-pipeline` and Figure `fig:cap-ci-run` (run 40, four jobs green, the fifth skipped)

THEN THEY ASK

In your screenshot one job is grey, not green — did something fail? — No. That is publish, which only runs on the main branch. The run was on develop, so it skipped exactly as declared, and a job that skips when it should is the pipeline working correctly.

DO NOT SAY

Do not say the jobs run one after another. backend and frontend run at the same time, and getting the shape of your own pipeline wrong ruins the whole answer.

94

You used multi-stage Docker builds. What do you really gain from that?

MEDIUM

SAY THIS

The build tools never reach the image I ship. Stage one has Maven, the sources and the dependency cache. Stage two has a JRE and the JAR, and nothing else. The frontend works the same way: Node builds the bundle, nginx serves it, and there is no Node in the final image at all.

IF THEY PUSH

In practice that means a smaller image and a much smaller attack surface. A compiler and a package manager sitting in a production container are two more things that can be attacked, and you need neither of them to run a JAR. I also split the copy step so that `pom.xml` and `dependency:go-offline` form their own layer. That layer is rebuilt only when the dependencies change, not on every edit of the source. The runtime stage creates a system user with uid 1001 and runs as that user instead of root. It sets `MaxRAMPercentage` to 75, so the JVM sizes itself to the container limit and not to the memory of the host. It uses an exec-form entrypoint, so the JVM becomes PID 1 and gets `SIGTERM` properly on

docker stop. The one thing I add on top of the slim runtime is curl, because the health probe needs it and the Temurin image does not ship it.

POINT TO backend/Dockerfile – maven:3.9-eclipse-temurin-21 AS build, eclipse-temurin:21-jre-jammy AS runtime · frontend/pms-frontend/Dockerfile – node:22-alpine AS build, nginx:1.27-alpine AS runtime · frontend/pms-frontend/.dockerignore – excludes node_modules, ~500 MB of build context

THEN THEY ASK

You skip the tests inside the image build with -DskipTests. Isn't that dangerous? — No, because the images job declares needs on backend. The pipeline never reaches image construction unless mvn verify has already passed, and running the suite twice doubles the build time and gives no new signal.

DO NOT SAY

Do not give an image size in megabytes. You have not measured it, and a made-up number is the one thing a jury can prove wrong on the spot.

95

What is a container healthcheck, and why does your backend need a ninety-second start period?

MEDIUM

SAY THIS

A healthcheck is a command Docker runs inside the container on a timer, to decide whether it is really ready and not just started. My backend needs ninety seconds because its very first boot applies 28 Flyway migrations, and one of them inserts about eleven hundred rows of demonstration data. That takes sixty to ninety seconds.

IF THEY PUSH

All three services declare one. The database uses pg_isready with -h 127.0.0.1, and that flag is not decoration. initdb runs a temporary server that listens only on a Unix socket, so without forcing a TCP connection the check would go green before the database accepts any real client. The backend curls /actuator/health, which I opened with permitAll in SecurityConfig and set with show-details: never, so a call with no login proves the application is up without leaking disk or database state. nginx answers a static /healthz that I kept on purpose outside the Angular routing logic, so it still replies even if the build is missing. The start period matters because failures inside that window do not count against the twelve retries: without it, the container would be called unhealthy while it is still migrating. Then depends_on with condition: service_healthy chains them together. The backend waits for a healthy database, and nginx waits for a healthy backend, because nginx resolves the name "backend" once at startup and dies with "host not found in upstream" if the container is not there yet.

POINT TO docker-compose.yml – three healthcheck blocks, start_period 20s / 90s / 10s · backend/src/main/resources/application.yml – exposure include health,info; show-details: never · backend/src/main/java/com/pms/shared/config/SecurityConfig.java – permitAll on /actuator/health · backend/src/main/resources/db/migration/V26__seed_demo_data.sql – 1098 lines

THEN THEY ASK

Is ninety seconds not just hiding a slow application? — It is the first boot only. Later starts skip the migrations and the container is healthy in seconds, and the pipeline proves it because it polls the health state instead of sleeping for a fixed time.

DO NOT SAY

Do not say the backend "waits" for the database by itself. It is Compose's `depends_on` with condition: `service_healthy` that forces the order, not application code.

96

Why do your frontend and your API share a single origin?

MEDIUM

SAY THIS

nginx serves the Angular build and forwards `/api` to the backend container, so the browser only ever talks to one address, port 8081. That removes CORS completely. And the refresh cookie — `HttpOnly`, `SameSite=Strict`, scoped to `/api/auth/refresh` — works with no special exception made for it.

IF THEY PUSH

In development the two are separate: `ng` serve on 4200 calls the backend on 8090, which really is cross-origin and needs a CORS configuration. In containers, both sit behind the same nginx, so that configuration becomes dead weight. I am happy to say plainly that the deployed setup is simpler and safer than my own development loop. The nginx config holds a few decisions worth naming. The `/api/` location uses the `^~` prefix, so it wins against the regular-expression location that handles static files. Deep links fall back to `index.html` through `try_files`, so reloading `/projects/1` works. Assets with a fingerprint in the name get a one-year immutable cache, while `index.html` is no-store, because it is the file that names those fingerprints. And the translation catalogues are explicitly no-cache, because Angular copies them through without a hash in the filename, and a long cache would freeze the translations after a redeploy.

POINT TO `frontend/pms-frontend/docker/nginx.conf` · `docker-compose.yml` — header comment: single entry point `http://localhost:8081` · `backend/src/main/resources/application-prod.yml` — `cors.allowed-origins` defaults to `http://localhost:8081` · `report/chap_03.tex` — Figure `fig:deploy`, the deployment diagram

THEN THEY ASK

Does a single origin not make the nginx container a single point of failure? — It does, and in a real deployment it would sit behind the company's own reverse proxy handling TLS, which is where you would add redundancy. For an internal tool on one host, one entry point is the right shape.

DO NOT SAY

Do not say CORS is "disabled". It is not needed, which is a different and stronger thing to say, and saying you disabled CORS sounds like you removed a protection.

SAY THIS

It caught the worst defect of the whole project. The Angular production configuration was not swapping in the production environment file, so the production bundle still pointed at localhost:8090. On my machine everything looked fine, because that backend was running. On any other host the application would have been completely broken.

IF THEY PUSH

The fix was in angular.json, and I added a guard so it can never come back: the frontend job greps the built bundle for localhost:8090 and fails the job with a clear error if it finds it. Automating the delivery chain brought two more defects to the surface in the same week. A Maven wrapper was written up in the project files but had never been committed. And a gitignore rule was quietly excluding .env.example, which is exactly the file the deployment procedure tells you to copy. None of the three was visible while I ran the application by hand, and that is the strongest argument I can make for the pipeline. The same job also reports the bundle budget: the initial bundle is 694.76 kB, which is 194.76 kB over the 500 kB warning threshold set in angular.json. It is a warning, not an error, so the build passes. I put that in the report instead of leaving it out, because it is the next thing the interface should fix.

POINT TO .github/workflows/ci.yml – step "Vérifier la substitution d'environnement" •
frontend/pms-frontend/angular.json – budgets: maximumWarning 500kB, maximumError 1MB •
report/chap_07.tex – Sprint 10 Implementation, "Defects revealed by the exercise"; Figure
fig:cap-frontend-build • backend/mvnw

THEN THEY ASK

Why not make the bundle budget a hard error? — Because it would block every merge over a performance worry and not a correctness one. I would first split up the two components that are too big, then tighten the threshold behind them.

DO NOT SAY

Do not present the environment-substitution bug as something you found by reading the code. The whole value of the story is that only the automation found it.

SAY THIS

That is correct and I will not soften it. The user module is at 4.6 percent of lines and mission at 3.9 percent. Six controllers have literally zero covered lines. I ran out of sprint, and I chose to spend the last testing days on the modules that carry money and permissions.

IF THEY PUSH

The exact figures are 18 of 392 lines in user and 8 of 207 in mission. The six controllers with nothing at all are *PartiePrenante*, *Mission*, *DevisInterne*, *Permission*, *Resource* and *Role*. *UserController* has one covered line out of nineteen. Given one more day, I would start with *UserController*, because that is the module that creates accounts and gives out roles, which makes it the untested code with the highest risk in the repository. Now the boundary. What is untested there is the CRUD around the permission model, not the enforcement of it. The filter chain, the token and the 403 path are all exercised by *AuthControllerTest* and *ProjectControllerTest*, and those apply to every endpoint, including the ones I did not test one by one. *Mission* is the thinnest business module in the system: a register with no computation in it. Since something had to be left out, I would draw the line the same way again. I would rather leave a register untested than the margin calculation, because a wrong register is a data-entry problem and a wrong margin is a decision made on a false number.

POINT TO backend/target/site/jacoco/jacoco.csv – user 18/392 lines, mission 8/207 ·
backend/src/main/java/com/pms/user/controller/UserController.java ·
backend/src/test/java/com/pms/project/ProjectControllerTest.java – the 200-then-403 RBAC test · report/chap_07.tex – Sprint 9 Testing: "user and mission have no test class of their own"

THEN THEY ASK

So an admin could break the permission system and no test would notice? — They could damage the data through the untested CRUD, yes. What they cannot do is get around the check itself, because that path is tested and all 108 endpoints share it.

DO NOT SAY

Do not say "I did not have time" and then stop. The missing half of the answer is which modules you protected instead, and why that order was the right one.

99

Your backend has ninety-seven tests. Your frontend has none. How is that acceptable?

HARD

SAY THIS

It is not acceptable, and I wrote it down as a gap, not as a choice. There is no karma, no jasmine and no spec file anywhere in the Angular project. The CI has no npm test step, and there is a comment in the workflow that says exactly why.

IF THEY PUSH

package.json still declares a test script, but angular.json has no test target and the testing dependencies are not installed, so adding npm test to the pipeline would make the CI red forever. I preferred an open gap with a written reason over a green tick that proves nothing. It is logged in my backlog as FE-1 and as risk T-3, and my own internal audit calls it the biggest credibility gap in the project. I am not discovering it here in front of you. The first thing I would write is the AuthService: login, refresh, logout and the HTTP interceptor, about twelve to fifteen specs. That flow already broke once during development, and it is the one place where a silent frontend bug logs everybody out. What the frontend does have is a production build that must compile on every change, plus the guard that stops the one bug it has already shipped.

POINT TO frontend/pms-frontend/package.json – no karma, no jasmine in devDependencies · frontend/pms-frontend/angular.json – no test target declared · .github/workflows/ci.yml – frontend job, NOTE comment explaining the absent npm test step · docs/ENHANCEMENTS.md – FE-1 and risk T-3

THEN THEY ASK

Would the backend tests catch a frontend bug? — No. They stop the browser from doing something it is not allowed to do, but they cannot tell me that a button is wired to the wrong endpoint.

DO NOT SAY

Do not argue that a TypeScript build which compiles is a replacement for tests. The compiler checks types, not behaviour, and the jury will push on exactly that.

100

Suppose ST2i wants to run this next month. How would you deploy it, and what is missing before it is production?

HARD

SAY THIS

One Linux host inside the ST2i network, docker compose up with a real .env file, and the company's own reverse proxy in front to terminate TLS. Four things are missing first: TLS with COOKIE_SECURE set to true, the demonstration data removed, scheduled backups, and branch protection so the pipeline is a gate and not a suggestion.

IF THEY PUSH

The mechanics are already there. main pushes both images to GHCR tagged by commit SHA, so the host pulls a known tag instead of building, and the stack needs about three gigabytes of disk and three free ports. Then the four gaps, in order of danger. First, the demonstration data.

V26__seed_demo_data.sql lives in classpath:db/migration, and the production profile loads that location. So fifty demo projects and thirty-six accounts, with passwords published in my documentation, are created under the production profile too. And the PMS_DEMO_SEED_USERS flag only gates the Java seeder, not the SQL migration. That must be excluded before a single real record exists. Second, TLS. I put it out of scope on purpose, to be terminated upstream. The moment it is in place, COOKIE_SECURE must go to true, or the refresh cookie travels without the Secure flag. Third, backups. Today they are a written pg_dump procedure that somebody has to remember to run, and that needs a scheduled job and a copy kept off the host. Fourth, branch protection is not configured in GitHub today, so a red pipeline does not really block a merge, and my own deployment document says so in writing. Beyond those four I would want log aggregation and metrics, since I expose only health and info on purpose. And I would point out that Flyway here goes forward only, with no rollback script.

POINT TO docs/DEPLOYMENT.md – section 10 Hors périmètre; section 9 branch-protection note · backend/src/main/resources/application-prod.yml – flyway.locations comment, cookie-secure: \${COOKIE_SECURE:false} · docker-compose.yml – PMS_DEMO_SEED_USERS default true, with the comment saying to disable it for real use · DECISIONS.md – ADR-026, containerised deployment, superseding ADR-014

THEN THEY ASK

So it is not production ready today? — No. It is ready for a demonstration and it can be deployed again and again the same way. The distance to production is those four items, and I would rather name them myself than let someone find them after the first real project is entered.

DO NOT SAY

Do not claim it is production ready. The demo accounts alone make that false, and being the one who says so first is worth more than the claim.

PART TWO

The 60 questions on the list you were given

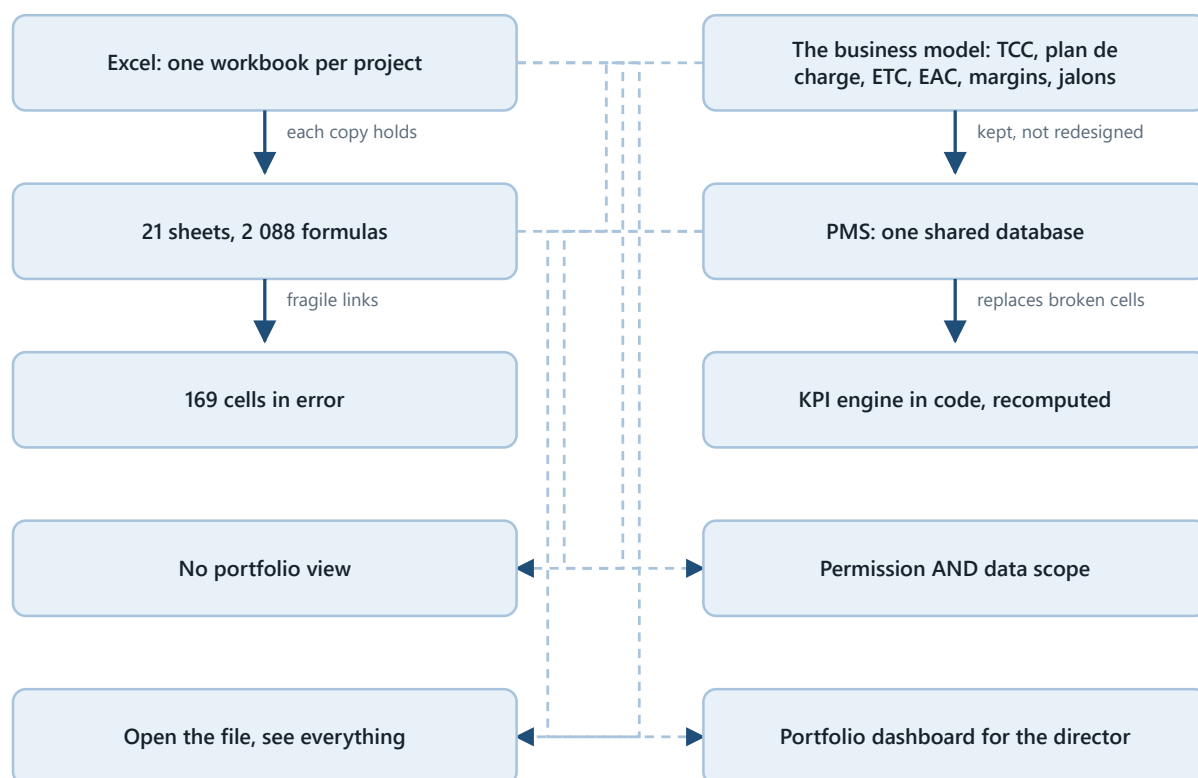
These are the questions somebody prepared for you, in the same order and with the same letters: A to E, then the ten they marked as the most dangerous. Every answer here was written after reading your real code and your real report, exactly like the first 100.

6 of them assume something about your project that is not true. Those cards open with a red box telling you what is wrong, so you can correct the examiner politely instead of agreeing with a mistake.

Some of them ask again, in different words, about something in Part one. That is on purpose: hearing the same idea asked two ways is the best way to find out whether you really know it.

A. Problem, context and business analysis

This examiner wants to know if you understood the business before you wrote code. He is testing one thing above all: can you explain why this software had to exist, in the company's own words, with the company's own numbers. Stay concrete — say "21 sheets", "2 088 formulas", "169 broken cells", "one copy per project" — and always separate the business model, which was good, from the medium, Excel, which was not.



The Excel world and the PMS world — The business model crosses from left to right unchanged; only the medium and its three faults are replaced.

101

Why did ST2i need PMS instead of continuing with Excel?

EASY

SAY THIS

The financial model in the Excel file was good. The file itself was the problem. ST2i keeps one workbook per project, so nothing adds up at portfolio level, the formula chain breaks, and anyone who opens the file sees the margins and the cost rates. PMS keeps the model and changes the medium.

IF THEY PUSH

ST2i tracks every project in a workbook called the "fiche de revue projet", reference F-AFF-13 version 03. It has 21 sheets and 2 088 formulas, and there is one copy per project. That model works: it is how the company decides whether a project is still profitable. The medium is what fails. There is no single source of truth, because each copy drifts from the others. There is no consolidation, so the director cannot read the real cost and the margin across the whole portfolio. The formula chain between sheets is fragile, and the real file the company gave me contains 169 cells in error, #REF! and #VALUE!. A spreadsheet also cannot partition information: it cannot stop a developer from reading the margin or the cost rates of his colleagues, and it does not keep a reliable history of who changed what. So I set four objectives in the report: one shared source instead of scattered copies, a faithful copy of the financial model rather than an approximation of it, access partitioned per user, and an authorization policy that can change without a new software release.

POINT TO report/chap_01.tex (sections "Project objectives", "Study of the existing system", "Critique of the existing system") · docs/BUSINESS_ANALYSIS.md §1 · docs/DEV_Fiche_revue_projet-Real.xlsx

THEN THEY ASK

Then why not just lock the file and put it on SharePoint? — That fixes sharing, not the three real faults: it still gives no portfolio consolidation, it still cannot hide one cell from one person, and it still leaves 2 088 formulas per project with no test that proves they are right.

DO NOT SAY

Do not say Excel is a bad tool. The examiner may use that workbook himself. The fault is the medium for this job, not the tool.

102

What exactly was wrong with the existing Excel process? Give me three concrete problems.

EASY

SAY THIS

First, dispersion: one workbook per project, so no consolidated view and copies that drift apart. Second, broken calculation: 169 cells of the real file are in error out of 2 088 formulas. Third, no access control: opening the file means seeing the margins and every colleague's cost rate.

IF THEY PUSH

Problem one is dispersion. Each project gets its own copy of a 21-sheet workbook, so there is no single source of truth and no way to read the actual cost and the margin across the portfolio. Two copies of the same template can drift apart without anyone noticing. Problem two is calculation reliability. The whole model relies on links between sheets: I counted 1 426 cross-sheet references, 771 of them pointing at the TCC sheet alone. One moved column breaks the chain, and the production file the company gave me already carries 169 cells showing #REF! or #VALUE!. Problem three is access control. A spreadsheet protects a file, not a field. The TCC sheet holds the daily loaded cost of each person, which

is close to their salary, and the Dashboard holds the margin. A developer who needs to write his man-days has to open a file that shows both. There are two further problems I list in the report: no consolidation at portfolio level, which makes a director's review slow and manual, and weak traceability, because the history of changes and decisions is not reliably kept.

POINT TO report/chap_01.tex ("Critique of the existing system") · docs/BUSINESS_ANALYSIS.md §9 and §10 (formula census, 169 #REF!/#VALUE!, 1 426 cross-sheet references)

THEN THEY ASK

How do you know there are exactly 169 broken cells? — I parsed the real workbook when I did the business analysis and classified all 2 088 formulas into nine categories; the broken ones are category F9 in docs/BUSINESS_ANALYSIS.md §10.

DO NOT SAY

Do not claim Excel loses data or crashes. Stick to what you measured in the file you were given.

103

Why do you say that the Excel workbook was effectively the real business specification?

MEDIUM

SAY THIS

Because it is the document the company actually decides with. I was given written specifications too, but where they disagreed with the workbook, the workbook was right, because that is what the project managers use every month. One sheet, the Glossaire, even writes down the calculation method of every indicator.

IF THEY PUSH

I received five written documents: a cahier des charges, a business rules specification with 65 rules, a software requirements specification, a use case specification and a project plan. They describe the system in general terms and give simplified formulas, for example margin equals budget minus estimate at completion. The workbook holds the real model, with three different margins, the accrued income (FAE, revenue earned but not yet invoiced), the drift in man-days, and a cost rate that depends on the year. One sheet makes this explicit: the Glossaire defines every indicator, its unit and its calculation method, so it reads like a specification chapter written by the business. So I wrote a governance rule at the start of the project, in DECISIONS.md: when sources conflict, the order is my recorded decisions, then the latest client instruction, then the business reality taken from the Excel, and only then the BRS, the use cases, the SRS and the project plan. The workbook also gave me something no document gave me: an acceptance test. At the end of each increment I reproduced on the platform a calculation the workbook already did, and compared the two numbers.

POINT TO DECISIONS.md (Source-of-Truth Priority; ADR-002; ADR-011) · report/chap_01.tex ("Analysis of the reference workbook"; the "Glossary of indicators" sheet; section on adapting Scrum) · docs/BUSINESS_ANALYSIS.md (authoritative inputs header)

THEN THEY ASK

Is it not risky to put a spreadsheet above a signed specification? — I never ignored a document silently; every conflict is written up as a decision record with both positions, so the choice is traceable, and the client supervisor validated each one.

DO NOT SAY

Do not say the written specifications were useless. They gave the 65 business rules, the actors and the use cases; they were just weaker on the financial model.

104

What did you discover by analyzing the workbook that was not explicitly written in the original requirements documents?

MEDIUM

SAY THIS

Six things. The cost rate depends on the year, not only on the person. Five whole registers existed that no document listed. Multi-currency was a fact, not an option. Several constants were business settings, not code constants. The earned value is typed in, not computed. And the person who costs is not always a person who logs in.

IF THEY PUSH

First, the TCC (taux de coût chargé, the daily loaded cost of a resource) is recorded per year, and the number of working days used to divide it changes: 22 in 2024, 20 in 2025. So the same person does not cost the same in January 2024 and January 2025, which the documents never said. Second, the workbook carries five registers the 16 documented phases did not mention at all: risks, deliverables, stakeholders, changes and avenants (contract amendments). The avenants matter most, because an avenant changes the budget and therefore changes every margin. Third, the money is in three currencies at once: the contract in FCFA, the reporting in TND, the missions in EUR with an exchange rate. Fourth, several values that look like technical constants are really business settings: the 1.7 overhead coefficient, the working days per year, the risk provision PPR at 5 % of the budget, the conversion of hours into man-days by dividing by 8, and the rule that the sum of billing milestone percentages must not go over 100 %. I made them configurable instead of compiling them in. Fifth, the Glossaire says the earned value (EV) is an estimate made by the project manager from the project plan, not something the sheet measures — so in PMS the project manager types it at each review instead of the engine deriving it. Sixth, the TCC and workload lists contain people who have no account, which is why the platform separates a User, who logs in, from a Resource, who costs money.

POINT TO DECISIONS.md (ADR-004 five extra modules; ADR-007 multi-currency; ADR-008 configurable parameters; ADR-022 User vs Resource) · report/chap_01.tex (TCC sheet; Glossary sheet; Parameters sheet) · backend/src/main/resources/db/migration/V21_tcc_annuel.sql

THEN THEY ASK

Which of those six cost you the most work? — The five extra registers, because each one needed its own screens, permissions and controlled vocabularies; but the per-year cost rate is the one that changed the shape of the KPI engine.

DO NOT SAY

Do not present these as things the client hid from you. They were simply obvious to people who use the file every day, which is exactly why reading the file mattered.

105

Why did you decide to reproduce the Excel financial model instead of redesigning it completely?

MEDIUM

SAY THIS

Because that model is what the company decides with, and it works. If I had invented a better one, every past project review would stop being comparable, and nobody at ST2i would trust my numbers. Redesigning a firm's margin model is not an intern's call.

IF THEY PUSH

This was the first decision I recorded, as ADR-002. The written specification gave a simplified set of formulas, and the workbook gave the rich one used in production: three margins (sold, current, at completion), the estimate to complete (ETC), the estimate at completion (EAC), the revenue produced, the accrued income (FAE), the remaining work (RAF) and the drift. I chose the rich one for three reasons. It is the model the project managers already read, so they can check my screen against their sheet line by line. It keeps old reviews comparable with new ones, which a new model would break. And it gave me an acceptance criterion for every sprint: reproduce a figure the workbook already produced, and compare. To be exact, I reproduced the model, not the file. I did not copy the 169 broken cells; I recompute cleanly from normalised data. I did not copy the sheet duplication either: Dashboard and Statistiques mostly read other sheets through lookups, so in PMS they became a read view, not stored data. And one formula is re-expressed rather than copied: instead of storing the annual loaded cost and dividing by the working days of the year, PMS stores a daily rate and an overhead rate per resource and per year, and the administrator does that division once when entering the rate. The result is the same cost, but the number of working days per year is not kept as a parameter in the platform today. That is a small honest gap I can close by adding one parameter.

POINT TO DECISIONS.md (ADR-002) · backend/src/main/java/com/pms/kpi/service/KpiService.java (method `chargeCost: days x dailyRate x (1 + tccRate)`) · backend/src/main/resources/db/migration/V21__tcc_annuel.sql · docs/BUSINESS_ANALYSIS.md §7

THEN THEY ASK

So you copied the company's mistakes as well? — No. I copied the rules and dropped the accidents: the broken cells are gone, the duplicated presentation sheets are gone, and the controlled vocabularies became real database constraints instead of conventions.

DO NOT SAY

Do not say the Excel model was perfect. Say it was proven and owned by the business, which is a different and stronger claim.

SAY THIS

The idea of "remaining work". In Excel, remaining means the cells below the current month — it is a position on a sheet. A database has no below. I had to turn a spatial idea into a rule, and that rule drives the ETC, the EAC and therefore the margin.

IF THEY PUSH

In the Coût Prévisionnel sheet the estimate to complete, ETC, is a running remainder read down a column: everything under the month you are looking at is still to come. A relational database has no up or down, so I had to define remaining as a condition. In PMS, a planned cell is a triple: resource, month, planned man-days. A cell counts as remaining when there is no validated actual entry for that same resource and that same month. The ETC is then the sum of those remaining days multiplied by the cost rate of the year of that month, the EAC is the actual cost plus the ETC, and the margin is the effective budget minus the EAC. You can read that in KpiService: I build a set of keys "user:period" from the validated actuals, filter the plan against it, and sum. There is a real limit I should state. Because the test is per cell, a month where a developer records 1 day out of 10 planned counts as done, and the 9 remaining days leave the ETC. So on partly filled months the ETC is slightly optimistic. The fix is small and I know where it goes: take the difference plan minus actual per cell, floored at zero, instead of excluding the whole cell. The second hardest rule was the cost rate that depends on the year, because the cost of a man-day has to be picked using the month being costed, not using today's rate.

POINT TO backend/src/main/java/com/pms/kpi/service/KpiService.java (buildKpi: actualKeys, remainingPlan, etcCost, eac; chargeCost) · report/chap_01.tex (Planned workload sheet; Glossary of indicators) · backend/src/main/resources/db/migration/V22__evm_indicators.sql

THEN THEY ASK

How did you check your ETC was right? — I ran the same project through both: I rebuilt a real review in PMS and compared the ETC, EAC and margin against the workbook's own numbers for the same month.

DO NOT SAY

Do not pick a rule that is only arithmetic, like a mission total. The examiner is looking for a rule where the change of medium, not the maths, was the difficulty.

How did you decide which Excel functionality was actually necessary for PMS and which functionality should not be reproduced?

MEDIUM

SAY THIS

I used one test: does this sheet produce a decision, or feed an indicator? If yes it became a module. If it only presented, duplicated or repaired something, it did not. I still analysed all 21 sheets, so nothing was dropped by accident.

IF THEY PUSH

I mapped every one of the 21 sheets to an owner, a set of rules and a target module, and I classified all 2088 formulas into nine categories, so the coverage is provable rather than a feeling. Then I applied the test. Kept: everything that feeds a number defined in the Glossaire — identification, TCC, planned workload, actual workload, billing milestones, missions, deliverables, risks and the parameter vocabularies. Not reproduced, and each for a stated reason. The 169 broken cells: they are damage, not logic, so PMS recomputes instead. The Dashboard and Statistiques sheets: they mostly read other sheets through lookups, so in PMS they are a read view served by the KPI engine, not stored data. The three near-duplicate billing recap sheets became one billing module, because they are three presentations of one set of milestones. The Fiche Doc sheet, which is document control with zero formulas, became audit fields on the records instead of a screen. The action list did not become its own register: a risk in PMS carries a mitigation plan field, which covers the need without a second workflow, and I accept that as a simplification. One exclusion was not mine: the DI sheet, the devis interne, holds commercially sensitive figures, so on the company's instruction PMS ships the DI structure as an empty template and no real value is seeded anywhere.

POINT TO docs/BUSINESS_ANALYSIS.md §9 (21/21 sheets mapped), §10 (2 088 formulas in nine categories), §16 (DI exclusion rule) · report/chap_01.tex (Table: Traceability from the Excel workbook to the PMS modules) · backend/src/main/java/com/pms/project/entity/LigneDi.java

THEN THEY ASK

Who validated that those exclusions were safe? — The industrial supervisor acted as product owner and accepted each increment against the workbook; the DI exclusion in particular is a written management decision, dated in the business analysis.

DO NOT SAY

Do not say you kept everything. Claiming full coverage of a 21-sheet workbook in one internship is not credible, and the exclusions are defensible.

SAY THIS

Because before building anything you have to answer one question: can we just buy this? I could not skip it. And I made the comparison mean something by scoring the tools against the requirements of the workbook, not against a generic feature list.

IF THEY PUSH

Building software a company could buy is waste, so the build-or-buy question had to be answered in writing. The trick was choosing the criteria. A generic comparison of project tools proves nothing, so I took the criteria straight from the reference workbook: a loaded cost rate per resource and per year, billing milestones expressed as a percentage of the contract and capped at 100 %, revenue earned but not yet invoiced, earned-value indicators, currency conversion at the business date, mission costs broken down by component, and an authorization policy that can change without redeploying. Read that way the answer is clear. MS Project attaches a cost to a resource, but not a cost to a resource for a given year, and it has no notion of a billing milestone as a share of a contract. Jira has no financial model of its own; costing needs a marketplace add-on such as Tempo, and even then it is time-based, not contract-based. monday.com has formula columns, but they are evaluated per board and do not compose across boards, which reproduces the exact fragility we are escaping. Odoo Project comes closest, because invoicing and timesheets are in the same suite, but its cost model is one hourly cost per employee and its margin is Odoo's own, so adopting it means either changing the company's numbers or customising a whole ERP. That asymmetry is the justification for building.

POINT TO report/chap_01.tex ("Existing market solutions" and "Comparative synthesis", Table 1.3)

THEN THEY ASK

Did you actually test these four tools? — I studied their documented capabilities and their cost and permission models against my criteria; I did not run a pilot of each, and the comparison is presented on that basis.

DO NOT SAY

Do not attack the four tools. Each is stronger than PMS in its own field, and the report says so; the argument is about fit, not quality.

SAY THIS

Jira steers the work. PMS steers the economics of a contract. In Jira the unit is a ticket; in PMS the unit is a man-day that carries a price, and that price flows into the actual cost, the estimate at completion and the margin.

IF THEY PUSH

In Jira a task moves across a board and the value produced is the software. In PMS a day of work is a financial object: it is multiplied by the daily loaded cost of that person for that year, it becomes the actual cost, the actual cost plus the estimate to complete gives the estimate at completion, and the budget minus that gives the margin the director reads at the monthly review. The second difference follows from the first: confidentiality has to be per field, not per project. A developer in PMS types his man-days and never sees the margin of the project he works on. I enforce that with two things together, a permission such as `VIEW_FINANCIALS` and a data scope that limits which projects a user's queries can even return. Jira's permission schemes work on issues and projects, so expressing "this person supplies the input but must never see the output" is awkward there. The third difference is what PMS does not try to be. It does have a backlog and sprints, because the ST2i subject asked for agile planning, but deliberately small: two entities, no epics, no story points. Items are estimated in man-days, because the plan de charge, the cost rates and the indicators all reason in man-days, and a second unit with no conversion to money would be useless here. On agile richness Jira wins, and I say so in the report.

POINT TO `report/chap_01.tex` (Jira paragraph; Table 1.3) · `DECISIONS.md` (ADR-021 permission and scope; ADR-027 agile module kept small, estimates in man-days) · `backend/src/main/java/com/pms/kpi/service/KpiService.java`

THEN THEY ASK

Could ST2i use both, Jira for the work and PMS for the money? — Yes, and that is the honest position: PMS only needs the consumed man-days per person per month, which is exactly what a time-tracking tool exports.

DO NOT SAY

Do not say PMS replaces Jira. It does not, and claiming it invites a comparison you would lose.

CAREFUL — THE QUESTION IS NOT QUITE RIGHT

The question assumes I framed this as a decision not to compete on scheduling. I did not. Table 1.3 compares the tools against the requirements of the reference workbook, the workbook itself has no scheduling, and the ST2i subject asked for agile planning through sprints and a backlog without ever mentioning scheduling or critical path.

SAY THIS

It was not a choice to step back from a fight. Table 1.3 scores every tool against the requirements of the reference workbook, and the workbook has no scheduling either. The ST2i subject asked for agile planning through sprints and a backlog, and never mentions scheduling or critical path.

IF THEY PUSH

The comparison table in chapter 1 does not ask "which tool is best". It asks "which tool carries the requirements this workbook already encodes", and those requirements are the per-year cost rate, the milestone as a percentage of contract, the accrued income, the earned-value indicators, the dated currency conversion, the mission cost components and the configurable authorization. Scheduling is in the table for honesty, as the row where the market tools score well and PMS scores a dash. I state that openly in the text: the existing tools are strong exactly where the workbook is weak, and absent exactly where it is strong. The subject the company assigned lists complete financial monitoring, operational monitoring of progress, delays, milestones and payments, agile planning through sprints and a backlog, and dynamic dashboards. Task dependencies and critical path appear nowhere in it. So building a scheduler would have been work the client did not ask for, taken from a financial model they did ask for and which nobody else provides. There is no scheduling code in the platform, and I would rather say that plainly than half-build a Gantt chart. If ST2i wants it later it is an addition, not a redesign: the sprints, deliverables and milestones that a dependency graph would need are already stored.

POINT TO `report/chap_01.tex` ("The assigned subject"; "Comparative synthesis", Table 1.3 row "Scheduling and critical path", and the paragraph beginning "The table should be read in both directions") · `report/chap_02.tex` (release plan: Sprint 5 "Agile planning: backlog and iterations") · `DECISIONS.md` (ADR-027)

THEN THEY ASK

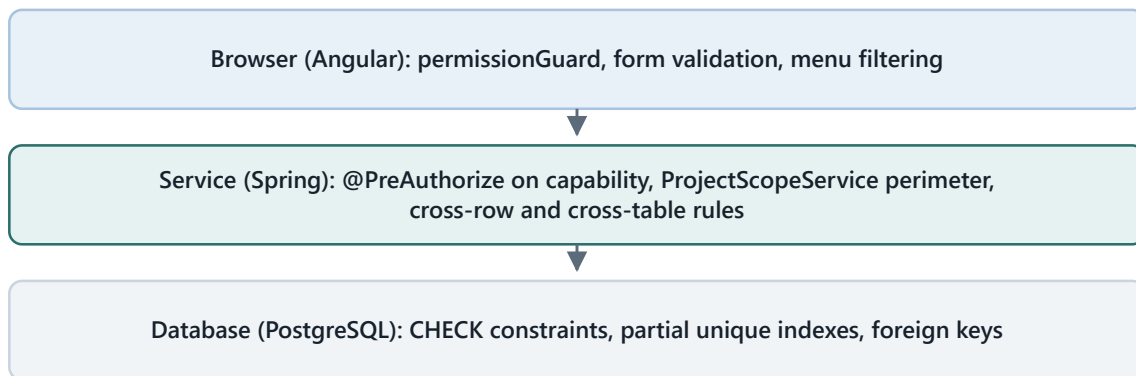
Is a project tool without a critical path really complete? — For this company's problem, yes: their steering question each month is "is this project still profitable", and that is answered by cost, earned value and margin, not by a network of task dependencies.

DO NOT SAY

Do not invent a scheduling roadmap or hint that a Gantt chart is half done. There is no scheduling code in the repository.

B. Requirements and business rules

This examiner wants to know if you really did analysis, or if you just coded. He is testing whether you can tell a function apart from a rule, and whether you know which layer of the system holds each rule and why. Stay concrete: name the Excel sheet, name the BR number, name the SQL constraint, name the Java method. Never say "it is handled" without saying where.



Where a rule is enforced, and what each layer catches — A rule the browser alone enforces is not enforced; the service catches anyone calling the API directly, and the database catches anyone bypassing the application entirely.

111

How did you transform the Excel analysis into functional requirements?

EASY

SAY THIS

I read the workbook sheet by sheet for six weeks before writing any code. For each sheet I wrote down three things: what it represents, what it calculates, and what the platform must therefore do. That third column became the functional requirements, and I kept a traceability table so every module points back to the sheet it came from.

IF THEY PUSH

The reference workbook is F-AFF-13 version 03. It has twenty-one linked sheets and more than two thousand formulas. I treated it as the real specification, not as a nice-to-have. The preparation phase ran from 5 January to 13 February 2026, six weeks, before Sprint 1. For each of the eleven most important sheets I wrote the same three paragraphs in chapter 1: what it represents, the key information and calculations, and the derived PMS functionality. That last paragraph is the bridge. For example the 'TCC' sheet holds a loaded cost per resource per year, so the platform needs a resource module with one cost rate per resource and per year, which became Sprint 2. The 'Avancement_Facturation' sheet holds milestones expressed as a percentage of the contract, so the

platform needs a jalon (billing milestone) module, which became the billing sprint. Two sheets were special. The 'Glossaire' sheet gave the definition and the calculation method of every indicator, so it is the direct specification of the KPI engine. The 'Parametres' sheet holds the controlled lists the company uses for risk probability, severity and treatment, so those became database enumerations instead of free text. I closed the analysis with the traceability table in chapter 1, one row per sheet, giving the business logic captured and the PMS module and sprint that carries it. From those modules I wrote the requirements document, FR-001 to FR-053, and the use cases UC-001 to UC-020, and then the thirty-two user stories US-01 to US-32 in the backlog. The honest limit is that I did the analysis alone. There was no requirements workshop with several people. My product owner, the industrial supervisor, checked each increment against the workbook at the sprint review, which caught the mistakes late rather than early.

POINT TO report/chap_01.tex (section 'Analysis of the reference workbook'; Table tab:excel-traceability) · report/chap_02.tex (Table tab:besoins-fonctionnels; Table tab:backlog, US-01 to US-32) · docs/BUSINESS_ANALYSIS.md · docs/DEV_Fiche_revue_projet-Real.xlsx

THEN THEY ASK

Did you ever find a formula in the workbook you could not reproduce? — Yes, a few cells in 'Statistiques' referenced values typed by hand with no formula behind them, so I turned them into fields the user enters on the internal quote rather than pretending to compute them.

DO NOT SAY

Do not say you 'converted the formulas into code'. That skips the analysis step and makes it sound mechanical. The examiner wants the reasoning path from sheet to requirement.

112

What is the difference between a functional requirement and a business rule in your project?

EASY

SAY THIS

A functional requirement says what a user can do. A business rule says what must always be true, no matter who does it or from which screen. In my documents they even have separate numbering: FR-001 to FR-053 for the functions, BR-001 to BR-065 for the rules.

IF THEY PUSH

A functional requirement is an action offered to an actor. For instance, the project manager can create a billing milestone with a percentage, an amount and dates. It maps to a screen, an endpoint and a use case. A business rule is a condition the data must respect whatever happens. For instance, the sum of the milestone percentages of one project must never go over one hundred percent. That rule holds when you create a milestone, when you edit one, and it would still hold if tomorrow I added an import from a file. The relation is many-to-many. One requirement is usually constrained by several rules: creating a milestone is constrained by the positive percentage rule, by the at-most-one-hundred rule, and by the rule that says a milestone belongs to exactly one project. And one rule constrains several requirements: the rule that a developer only enters charge (workload) on projects they are assigned to

constrains both the submit function and the edit function. There is a practical consequence too. Requirements move a lot during a project, because the company changes its mind about screens. Rules move much less, because they come from how ST2i actually runs a contract. So I put the rules where they are hard to bypass, in the service layer and in the database, and I let the screens change on top of them.

POINT TO `report/chap_02.tex` (Table `tab:besoins-fonctionnels` vs Table `tab:regles-gestion`) ·
`backend/src/main/java/com/pms/billing/service/JalonService.java` (`validatePourcentageSum`)
· `docs/BUSINESS_ANALYSIS.md`

THEN THEY ASK

And a non-functional requirement? — That is a quality of the service rather than a function, for example a response under two seconds for routine operations, or mandatory JWT authentication with hashed passwords.

DO NOT SAY

Do not define a business rule as 'a validation'. Validation is one way to enforce a rule; the rule exists even when nothing validates it yet.

113

Why are business rules necessary if they are already implemented in the code?

MEDIUM

SAY THIS

Because the code is only one expression of the rule, written in a language my client cannot read. The rule is the agreement with ST2i. It also tells me where to enforce it, it gives my tests something to test against, and it gives me one place to look when the company changes its mind.

IF THEY PUSH

There are four concrete reasons. First, ownership. The rules came from the workbook and from the company, not from me. Writing them as BR-001 to BR-065 in plain language let my product owner read them and say yes or no. He cannot review a Java method. Second, placement. A rule stated on its own lets me choose the layer. Once I read 'the sum of milestone percentages must not exceed one hundred percent', I can see it spans several rows and so cannot be a simple column CHECK in SQL, and it has to live in the service. That decision is not visible from the code alone. Third, testing. The rule is the oracle. `ChargeReelleServiceTest` and `JalonServiceTest` exist to prove specific rules, not to prove that a method returns something. Fourth, change. When ST2i changes a rule, I look up the identifier and find the code. In my services the rules are quoted as comments where they are enforced, for example 'BR-033' above the ownership check in `ChargeReelleService` and 'BR-050' above the financial masking in `ProjectService`. The weakness I will admit is that this link is a comment, not tooling. Nothing automatically proves that all sixty-five rules are still enforced and covered by a test. With overall line coverage at forty-one point two percent, some of those rules are covered only by reading, which is why I also compared platform figures against the workbook figures at every sprint review.

POINT TO backend/src/main/java/com/pms/workload/service/ChargeReelleService.java (comments 'BR-033', 'BR-062...064', 'H-8') ·
backend/src/main/java/com/pms/project/service/ProjectService.java (comment 'BR-050' on toResponse) · backend/src/test/java/com/pms/workload/ChargeReelleServiceTest.java ·
backend/src/test/java/com/pms/billing/service/JalonServiceTest.java

THEN THEY ASK

So how do you know the code still matches BR-033 today? — Through the test in ChargeReelleServiceTest that asserts a developer without VALIDATE_WORKLOAD gets an AccessDeniedException when submitting for another user; if someone removes the check, that test fails.

DO NOT SAY

Do not say 'for documentation'. That sounds like paperwork. Give the four working uses: agreement, placement, testing, change.

114

Give me one example of a business rule that the database enforces.

EASY

SAY THIS

A billing milestone percentage must be greater than zero and at most one hundred. It is a CHECK constraint named chk_jf_pourcentage, in migration V9, on the jalons_facturation table. Even if someone ran an UPDATE by hand in psql, PostgreSQL would refuse it.

IF THEY PUSH

The exact line is CONSTRAINT chk_jf_pourcentage CHECK (pourcentage > 0 AND pourcentage <= 100), in V9__schema_billing.sql on the table jalons_facturation, which holds the jalons de facturation, the billing milestones. I chose the database for this one because it is a rule about a single column of a single row, so the database can check it cheaply and can never be bypassed. There are others of the same shape. chk_project_dates on projects says the end date is null or on or after the start date. I had to fix it in V17 because I first wrote strictly greater than, which blocked a legitimate one-day project. chk_pc_days on plan_charges says planned days are greater than zero and at most thirty-one, and chk_cr_days on charges_reelles says actual days are at least zero and at most thirty-one, because a month cannot hold more. chk_tcc_rate on resources says the TCC rate, the taux de cout charge or loaded cost multiplier, is at least zero and under ten. The status values are guarded too: chk_status on projects restricts the status to DRAFT, ACTIVE, ON_HOLD, COMPLETED or CANCELLED. When one of these fires, Spring throws a DataIntegrityViolationException and my GlobalExceptionHandler turns it into a clean error response rather than a stack trace.

POINT TO backend/src/main/resources/db/migration/V9__schema_billing.sql (chk_jf_pourcentage) ·
backend/src/main/resources/db/migration/V17__fix_project_date_check.sql
(chk_project_dates) · backend/src/main/resources/db/migration/V7__schema_workload.sql
(chk_pc_days, chk_cr_days) ·
backend/src/main/java/com/pms/shared/exception/GlobalExceptionHandler.java (line 96,
DataIntegrityViolationException)

THEN THEY ASK

Is the same check also in the service? — The single-value ones are covered by bean validation on the request DTO so the user gets a friendly message, and the database constraint is the last line that nothing can go around.

DO NOT SAY

Do not invent a constraint name. If you blank on `chk_jf_pourcentage`, describe the rule and say it is in `V9` rather than guessing a name.

115

Give me one business rule that must be enforced in the service layer rather than only in the frontend.

MEDIUM

SAY THIS

The sum of all billing milestone percentages on a project must not go over one hundred percent. That one spans several rows, so it cannot be a column CHECK, and the browser cannot be trusted with it. It lives in `JalonService`, in a method called `validatePourcentageSum`.

IF THEY PUSH

The method reads the current total for the project from the repository, subtracts the old percentage when we are editing an existing milestone, adds the new one, and throws a `BusinessRuleException` if the result is over one hundred. It runs on both create and update, so there is no path around it. Two reasons it must sit there. First, it is a rule across rows of a table, and a SQL CHECK only sees the row being written, so the database cannot express it without a trigger, which I did not want. Second, the frontend cannot be the guard because anyone can call the API directly with curl or Postman; the Angular check is only there to give a fast message. There are other service-layer rules of the same family in the project. A developer can only enter charge on a project where they are an active team member, in `ChargeReelleService`, method `assertTeamMembership`. A project status can only move along legal transitions, held by the `ProjectStatus` enum and checked in `changeStatus`, so `ACTIVE` can go to `ON_HOLD`, `COMPLETED` or `CANCELLED` but `COMPLETED` can go nowhere. Only a finished project can be archived. A charge that is already validated can no longer be edited or deleted. One honest limit on the percentage rule: it reads then writes without a database lock, so two people creating milestones at the very same instant could both pass the check. In practice one project manager manages the billing of one project, so the window never opens, but the correct fix would be a pessimistic lock on the project row.

POINT TO `backend/src/main/java/com/pms/billing/service/JalonService.java` (`validatePourcentageSum`, lines 138-145) · `backend/src/main/java/com/pms/workload/service/ChargeReelleService.java` (`assertTeamMembership`) · `backend/src/main/java/com/pms/project/entity/ProjectStatus.java` (`canTransitionTo`) · `backend/src/test/java/com/pms/billing/service/JalonServiceTest.java`

THEN THEY ASK

Why not a database trigger then? — A trigger hides business logic in a place no reviewer reads, splits the rules across two languages, and is much harder to unit test than a Java method; the billing package sits at seventy-five percent line coverage precisely because the rule is testable Java.

DO NOT SAY

Do not claim the rule is enforced in the database as well. It is not, and the examiner may ask you to show the constraint.

116

Why is project creation reserved for the Director?

MEDIUM

SAY THIS

Because creating a project means opening a contract: the code, the client, the budget, the engagement type. That is a commitment the company makes, so it belongs to the person who governs the portfolio. In the code it is a permission, `CREATE_PROJECT`, and the Director role is the only one that holds it by default.

IF THEY PUSH

Let me be precise about how it is enforced, because the wording matters. Nowhere in my code do I test whether someone is a Director. The method `ProjectService.create` carries `@PreAuthorize("hasAuthority('CREATE_PROJECT')")`. The link between the role and that permission lives in the database, in migration `V12__fix_rbac_matrix.sql`, which grants `CREATE_PROJECT` to `DIRECTEUR` and to nobody else. The business reason is that project creation is where money enters the system. The creation form holds the fiche d'identification, the project identity card taken straight from the Excel sheet: client, contract reference, funder, currency, sold workload in man-days, penalty provision, sold net margin. Those are contract terms. A project manager steers the work inside that envelope; they do not decide the envelope. There is also something worth pointing out. The Administrator does not have `CREATE_PROJECT` either. V12 deliberately cut the Administrator down to five permissions: manage users, manage roles, view the audit log, and the two resource permissions. That was a real correction; the first version of my matrix gave the Administrator everything, which broke the separation between running the platform and running the business. Finally, because the matrix is data, an administrator can move `CREATE_PROJECT` to another role from the Roles admin page and the change takes effect without a redeployment. The token version is bumped so open sessions reload their permissions.

POINT TO `backend/src/main/resources/db/migration/V12__fix_rbac_matrix.sql` (DIRECTEUR block includes `CREATE_PROJECT`; ADMIN block has 5 permissions only) · `backend/src/main/java/com/pms/project/service/ProjectService.java` (line 100, `@PreAuthorize hasAuthority CREATE_PROJECT`) · `backend/src/main/resources/db/migration/V25__rbac_admin_module.sql` (role/permission admin, `MANAGE_ROLES` granted to ADMIN) · `docs/AUTHORIZATION_MATRIX.md`

THEN THEY ASK

So an administrator could grant himself CREATE_PROJECT? — Yes, an administrator with MANAGE_ROLES can change the matrix; that is the price of a dynamic RBAC, and the mitigation is that the change is a data change recorded with created_by and updated_by rather than a silent code path.

DO NOT SAY

Do not say 'because the Director is the boss'. And do not say the check is on the role name, because the whole architecture argument of my report is that it is not.

117

Why can a Project Manager manage a project without being allowed to create every project?

MEDIUM

SAY THIS

Because managing and creating are two different capabilities, and on top of that a project manager only reaches the projects in their own perimeter. They hold EDIT_PROJECT but not CREATE_PROJECT, and even EDIT_PROJECT only bites on projects where they are the chef de projet or a team member.

IF THEY PUSH

There are two independent mechanisms and they answer two different halves of the question. The first is the capability. In V12 the CHEF_PROJET role gets EDIT_PROJECT, ASSIGN_DEVELOPER, VIEW_TEAM, VALIDATE_WORKLOAD, MANAGE_BILLING, MANAGE_MISSION, MANAGE_GOVERNANCE and the view permissions, but not CREATE_PROJECT, not DELETE_PROJECT and not ASSIGN_CHEF_PROJET. That is separation of duties: the person who spends the budget is not the person who sets it. The second is the perimeter, which I call scope. ProjectScopeService answers the question 'which projects may this user touch'. If the user holds VIEW_ALL_PROJECTS, which the Director has and the project manager does not, there is no filter at all. Otherwise the accessible projects come from relations: projects where the user is the chef de projet, and projects where the user has an active team assignment. Any other project id gives an AccessDeniedException, a 403. So the real rule is permission AND perimeter, not permission alone. I also closed two ways a project manager could have widened their own power through the edit form. In ProjectService.update, the director field is only applied if the caller holds CREATE_PROJECT, and the chef de projet field is only applied if the caller holds ASSIGN_CHEF_PROJET. Without those two guards a project manager with EDIT_PROJECT could have reassigned the project to themselves or swapped the director.

POINT TO backend/src/main/resources/db/migration/V12__fix_rbac_matrix.sql (CHEF_PROJET block, 14 permissions, no CREATE_PROJECT) ·
backend/src/main/java/com/pms/project/service/ProjectScopeService.java ·
backend/src/main/java/com/pms/project/service/ProjectService.java (lines 150-158, guards on directorId and chefProjetId) ·
backend/src/test/java/com/pms/project/ProjectScopeServiceTest.java

THEN THEY ASK

How does the perimeter perform with many projects? — `accessibleProjectIds` runs a single SQL query with an EXISTS subquery; I merged three separate queries into one during the optimisation pass, and the paged list applies the filter as an id IN clause rather than filtering in memory.

DO NOT SAY

Do not answer only with permissions. If you skip the perimeter, the examiner will ask why a project manager cannot edit another manager's project, and you will have no answer.

118

Why does a project have exactly one active Project Manager?

MEDIUM

CAREFUL — THE QUESTION IS NOT QUITE RIGHT

The report says 'exactly one active project manager per project'. In the schema it is at most one: `chef_projet_id` is nullable, so a DRAFT project can have none until the Director assigns one.

SAY THIS

Because one person has to be accountable for the numbers. The project manager validates workload, sets the billing milestones and answers for the margin. The database holds it as a single column, `chef_projet_id` on the projects table, so two at once is structurally impossible.

IF THEY PUSH

The column is `chef_projet_id` BIGINT with a foreign key `fk_projects_chef` to `users(id)`, declared in `V5__schema_project.sql`. It is one column, not a join table, so the schema itself forbids two simultaneous project managers. That was a deliberate modelling choice taken from the workbook: the Fiche identification sheet has one line for the chef de projet, because at ST2i one named person answers for the project review. It also matters operationally. Workload validation, billing milestone management and the mission register all belong to whoever holds that seat, and my `ProjectScopeService` derives the perimeter partly from this relation. If two people held it, 'who validated this charge and who owes an explanation for the drift' would have no single answer. I should correct the wording slightly, to be exact. It is at most one, not exactly one. The column is nullable, so a project can sit in DRAFT with no project manager until the Director assigns one, which is realistic because the contract is often signed before the manager is named. Reassigning is a single call, `assignChefProjet`, gated by `ASSIGN_CHEF_PROJET`. And here is the honest weakness: there is no history table of project manager changes. When the Director reassigns, the previous value is overwritten. I only keep `updated_by` and `updated_at` from the audit columns added in V19. If ST2i needs to know who steered the project in March, the schema cannot answer that today, and the fix would be the same soft-delete pattern I used for team assignments.

POINT TO `backend/src/main/resources/db/migration/V5__schema_project.sql` (`chef_projet_id`, `fk_projects_chef`) • `backend/src/main/java/com/pms/project/service/ProjectService.java` (`assignChefProjet`, `@PreAuthorize ASSIGN_CHEF_PROJET`) • `backend/src/main/resources/db/migration/V19__audit_columns.sql` (`created_by` / `updated_by` on projects) • `report/chap_02.tex` (Table `tab:regles-gestion`, Projects row)

THEN THEY ASK

What if a project manager leaves the company mid-project? — The Director reassigns with `assignChefProject`; the departing user is deactivated rather than deleted, so every charge, milestone and assignment they created still points at a valid user row.

DO NOT SAY

Do not claim the database guarantees 'exactly one'. The column is nullable, and if you overstate it the examiner only has to ask whether a DRAFT project can exist without a manager.

119

Why did you preserve assignment history instead of deleting old team assignments?

EASY

SAY THIS

Because an old assignment is the reason an old cost exists. If I delete the row that says someone worked on a project last March, the March cost stops having a justification. So removing a member is a logical delete, not a real one.

IF THEY PUSH

Every table in the schema has a `deleted` boolean, and removing a team member sets that flag rather than issuing a `DELETE`. In `TeamAssignmentService.remove` the code sets `deleted` to `true` and saves. Three reasons. First, the financial chain. A charge reelle, the actual workload record, points at a project and a user for a month; it is multiplied by the TCC, the loaded daily cost, to produce the actual cost of that month, which feeds the earned value indicators. The assignment is what made that entry legitimate. Erase it and the audit trail breaks. Second, auditability is an explicit non-functional requirement of the project, NFR-008, history preservation and logical deletion, and the workbook itself keeps everything: nobody at ST2i deletes a line of the project review. Third, re-assignment has to stay possible. That is why the uniqueness is a partial unique index, `uk_ta_project_user_active` on `team_assignments(project_id, user_id) WHERE deleted = FALSE`. It says one active assignment per person per project, while still allowing the same person to be re-assigned later, which a plain `UNIQUE` constraint would have blocked forever. I used the same pattern in V18 for `users.email` and `projects.code`, replacing the absolute `UNIQUE` constraints with partial unique indexes so a code can be reused after a soft delete. The cost of all this is that every query has to remember the filter, which is why my repositories expose named methods like `findActiveByProjectId` rather than plain `findAll`.

POINT TO `backend/src/main/java/com/pms/team/service/TeamAssignmentService.java` (remove sets `deleted = true`) · `backend/src/main/resources/db/migration/V6__schema_team.sql` (`uk_ta_project_user_active` partial unique index) · `backend/src/main/resources/db/migration/V18__soft_delete_unique_indexes.sql` · `report/chap_02.tex` (non-functional requirements, NFR-008 auditability)

THEN THEY ASK

What is the risk of soft delete? — Forgetting the `deleted = FALSE` filter in one query silently shows deleted rows, so I centralised it in named repository methods and every service loads through helpers like

loadProject and findActiveById rather than through generic finders.

DO NOT SAY

Do not say 'for history' and stop. Name the partial unique index, because that is the part that shows you thought about re-assignment.

120

Why can't a developer enter workload for a project to which they are not assigned?

MEDIUM

SAY THIS

Because a charge is a cost. If a developer could post days onto any project, they could inflate that project's real cost and wreck its margin. So the service checks two things on every submission: that you are entering for yourself, and that you are an active member of that project's team.

IF THEY PUSH

Both checks are in ChargeReelleService.submit and they are separate. The first is assertOwnership, which covers BR-033: if the caller does not hold VALIDATE_WORKLOAD, the target user id must equal their own id, otherwise an AccessDeniedException. That stops a developer submitting in someone else's name. The project manager holds VALIDATE_WORKLOAD, so they can post on behalf of a team member who is away. The second is assertTeamMembership: the target user must have an active team assignment on that project, or be the project's chef de projet, who counts as a member implicitly. If neither is true it throws a BusinessException. The same ownership check runs again on update, so you cannot submit a valid entry and then edit it into someone else's name. A validated charge is frozen: edit and delete both refuse once validatedAt is set. Reading is filtered as well. In findByProject, a caller who holds neither VALIDATE_WORKLOAD nor VIEW_ALL_PROJECTS only sees their own lines, so one developer cannot read a colleague's declared effort. Why service and not browser: the Angular screen only offers the projects the user is assigned to, but anyone can call POST on the endpoint directly. The browser is convenience, the service is the rule. The database cannot do this one, because it would need to read the team_assignments table while writing charges_reelles, which is a trigger, not a CHECK. This is covered by ChargeReelleServiceTest; the workload package sits at sixty-two point three percent line coverage.

POINT TO backend/src/main/java/com/pms/workload/service/ChargeReelleService.java (submit, assertOwnership, assertTeamMembership, canSeeAllWorkload) ·
backend/src/test/java/com/pms/workload/ChargeReelleServiceTest.java ·
backend/src/main/resources/db/migration/V7__schema_workload.sql (uk_cr_active: one active entry per project, user and month) · frontend/pms-frontend/src/app/core/guards/permission.guard.ts

THEN THEY ASK

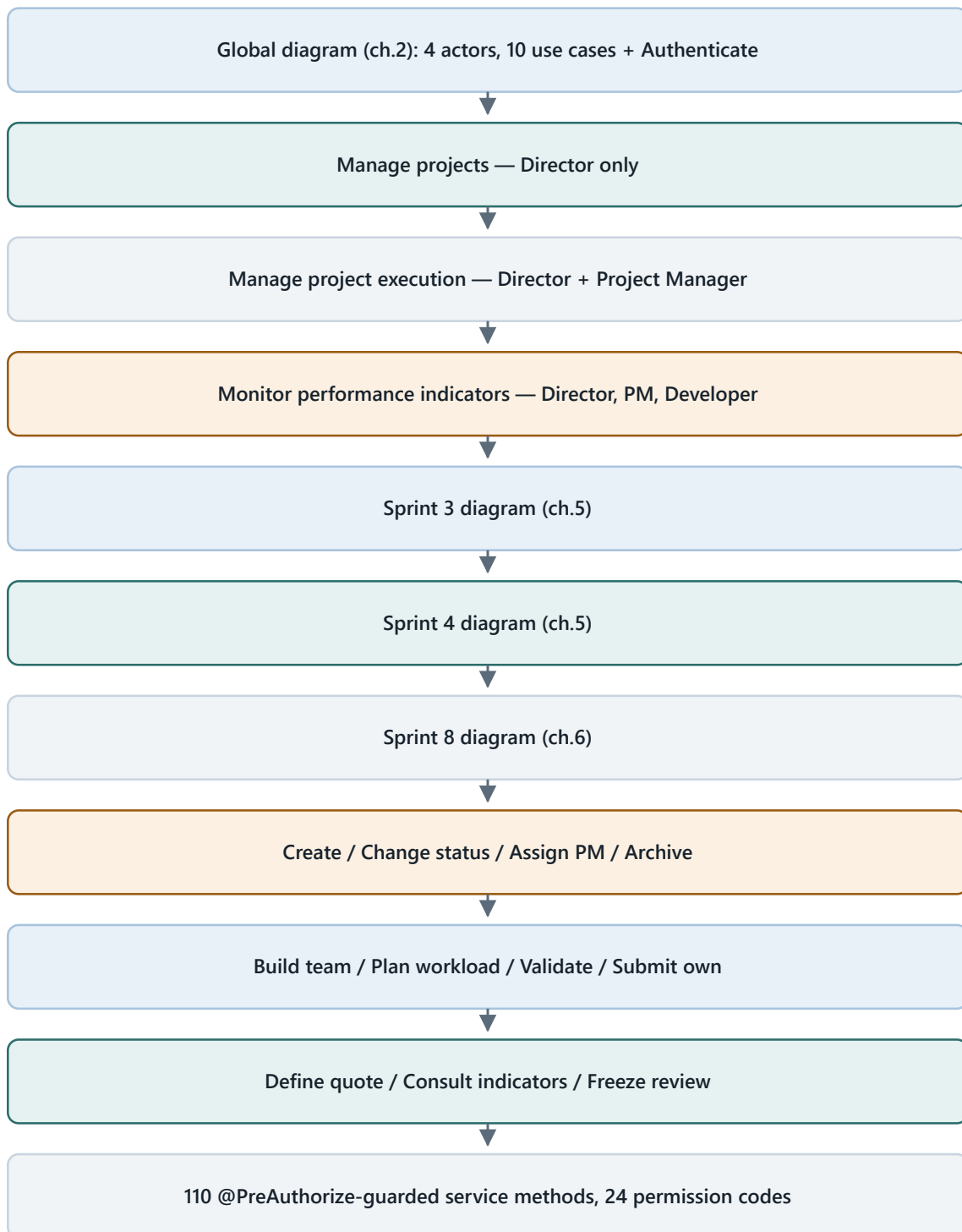
What stops the same developer entering the same month twice? — The service checks existsByProjectIdAndUserIdAndPeriodAndDeletedFalse before saving, and the partial unique index uk_cr_active on (project_id, user_id, period) WHERE deleted = FALSE backs it up in the database.

DO NOT SAY

Do not say the frontend prevents it. That is the answer the examiner is fishing for, and it is the wrong one.

C. UML and analysis / design

This examiner is not testing whether you know UML notation. He is testing whether you chose your level of abstraction on purpose, and whether you can defend that choice with the real content of your figures. Answer like an architect who made decisions, not like a student reciting definitions: say what the diagram shows, say what it deliberately does not show, and say where the missing detail went. Three of these ten questions carry a wrong assumption, so read each one twice before you agree with it.



Two levels of use case: the global map and the sprint detail — Nothing is hidden by the global diagram: each of its ellipses opens into a sprint diagram, and each sprint use case lands on real guarded code.

121

Why is your global use-case diagram intentionally high-level?

EASY

SAY THIS

Because that diagram answers one question only: what is this platform for, and who is allowed to use which part of it. So I drew ten business use cases and four actors. Every elementary operation is drawn too, but one level down, in the eight sprint diagrams of chapters 4 to 7.

IF THEY PUSH

The global diagram in chapter 2 has eleven ellipses: ten business use cases plus Authenticate. Each ellipse is a functional domain, not a method. "Manage projects" covers creation, configuration, the assignment of a project manager, status change and archiving. "Manage project execution" covers the team, the plan de charge (the monthly workload plan) and its validation, and the iterations. If I had drawn one ellipse per operation I would have had about a hundred: the backend has 110 service methods guarded by a @PreAuthorize permission check, and 24 distinct permission codes. A diagram of a hundred ellipses no longer shows what the system does, it shows how it is built, and that belongs in the design chapters. The real job of this figure is to make the actor boundaries visible at a glance, and they are: the administrator touches no project use case, and the developer touches no financial use case. Those two facts are the security posture of the whole platform, and they are readable in two seconds on this figure.

POINT TO report/chap_02.tex – section "Global use case diagram" · report/img/use_case.png · docs/figures/gen_drawio_usecase.py – SPECS['use_case'] · report/chap_02.tex – Table 2.1, actors and their boundaries

THEN THEY ASK

So where does an examiner see the detail? — In the eight sprint use-case diagrams, figures uc_sprint1 to uc_sprint8, each drawn inside the sprint that delivered it, in chapters 4 to 7.

DO NOT SAY

Do not say "to keep it simple" and stop there; that sounds like you ran out of time rather than made a choice.

122

Why did you group several operations into a use case such as "Manage Projects"?

MEDIUM

SAY THIS

Because a use case is a goal an actor has, not a method the system offers. The Director's goal is to put a project under control and keep it there. Create, configure, assign the chef de projet and archive are steps inside that one goal, not four separate goals.

IF THEY PUSH

The test I applied is simple: does the actor stop and feel finished? Nobody logs in wanting only to "update a project". The Director logs in wanting the project to exist, to have a budget, and to have a project manager on it. That is one goal, so it is one ellipse. Behind that ellipse the code really is several operations: ProjectService has eleven methods guarded by @PreAuthorize, ProjectController exposes ten endpoints, and the PROJET permission module holds six codes — VIEW_PROJECT, CREATE_PROJECT, EDIT_PROJECT, DELETE_PROJECT, ASSIGN_CHEF_PROJET and VIEW_ALL_PROJECTS. Those six codes are how the right is actually granted, and they appear in chapter 4 where the dynamic RBAC is built. Grouping them in the global figure also keeps the figure honest about scope: the ellipse is the unit the company negotiated with me, and the permission code is the unit the software enforces. They are two different levels, and I drew each at its own level.

POINT TO backend/src/main/java/com/pms/project/service/ProjectService.java — 11 @PreAuthorize-guarded methods · backend/src/main/java/com/pms/project/controller/ProjectController.java — 10 endpoints · backend/src/main/resources/db/migration/V2__seed_rbac.sql — PROJET module permission codes · backend/src/main/resources/db/migration/V13__add_view_all_projects.sql

THEN THEY ASK

Is "Manage Projects" then just CRUD with a nicer name? — No: it also carries the controlled lifecycle (DRAFT, ACTIVE, ON_HOLD, COMPLETED, CANCELLED) and the assignment of the chef de projet, which are business rules, not CRUD.

DO NOT SAY

Do not call the grouped operations "CRUD"; that invites him to say your use case is just a database table with a label on it.

123

Why didn't you put Create Project, Update Project, Delete Project, Assign Project Manager, etc. all separately in the global diagram?

MEDIUM

SAY THIS

I did put them separately — just not on that figure. They are all drawn on the Sprint 3 use-case diagram in chapter 5. Nothing is hidden; the detail is moved one level down so that each figure answers one question.

IF THEY PUSH

The Sprint 3 diagram shows exactly four elementary use cases: create and configure a project, change the project status, assign the project manager, and archive a completed project. So the examiner can see every operation he named, with the right actor on each one — and he can see that archiving is the one the chef de projet also reaches, while the other three are the Director's alone. Putting those four back on the global figure would have forced me to do the same for every other domain, and the count is the argument: 110 guarded service methods across 22 services. The global figure would have become a list of endpoints. There is also a modelling reason. Create, update and delete would all hang off the same

actor with the same association, so the figure would repeat the same information four times and add nothing. The rule I followed for the whole report is one figure, one question: the global figure answers "what is the scope and who owns what", and the sprint figure answers "what exactly did this sprint deliver".

POINT TO docs/figures/gen_drawio_usecase.py – SPECS['uc_sprint3'] · report/chap_05.tex – figure uc_sprint3.png, label fig:uc-s3 · report/chap_02.tex – "Decomposing each of these into its elementary operations would produce a diagram of some forty ellipses"

THEN THEY ASK

Why forty in the text and a hundred now? — Forty is the count of documented business operations across the ten domains; a hundred is the count of guarded methods in code, which includes reads and sub-resources — the text quotes the modelling figure, not the code figure.

DO NOT SAY

Do not say the operations are "implied" by the global diagram; say plainly that they are drawn, and name the figure where.

124

What is the difference between a global use-case diagram and a sprint-level use-case diagram in your report?

EASY

SAY THIS

The global one is a single figure that describes the whole system and never changes. The sprint ones are eight figures, one per sprint, each describing only what that sprint delivered. Different granularity, different lifetime, different purpose.

IF THEY PUSH

There are nine use-case diagrams in the report: one global in chapter 2, and eight sprint diagrams across chapters 4 to 7. The global one has four actors and ten business use cases plus Authenticate, and it is written once during analysis; it is the contract for the whole platform. A sprint diagram is much narrower: Sprint 3 has four use cases and two actors, Sprint 6 has five use cases and two actors, Sprint 8 has four and two. Each sprint figure names elementary operations — "Record a payment", "Validate the team workload", "Freeze the monthly review" — and each carries a short note stating the business rule that the sprint had to enforce, for example that an amendment (avenant) is reserved to the Director because it changes the contract, not its execution. There is one more difference that matters: only actors involved in that sprint appear. Sprint 2 shows the administrator alone, Sprint 4 shows the Director, the chef de projet and the developer. So the sprint figure is also a scoping document, and reading the eight of them in order shows the platform being built.

POINT TO docs/figures/gen_drawio_usecase.py – nine SPECS entries · docs/figures/README.md – "The nine use case diagrams" · report/chap_04.tex – fig:uc-s1, fig:uc-s2 · report/chap_06.tex – fig:uc-s6, fig:uc-s7, fig:uc-s8

THEN THEY ASK

Does every sprint use case map back to exactly one global use case? — Yes, except Authenticate, which is a use case in Sprint 1 and an included use case everywhere after that.

DO NOT SAY

Do not describe the sprint diagrams as "zoomed-in versions" only; say they also carry the sprint's business rule as a note, which the global one cannot.

125

Why can Director and Project Manager share the same high-level "Manage Projects" use case?

HARD

CAREFUL — THE QUESTION IS NOT QUITE RIGHT

The question assumes the Director and the Project Manager both connect to "Manage projects" in the global diagram. They do not: only the Director does. The two use cases they genuinely share are "Manage project execution" and "Monitor performance indicators".

SAY THIS

They do not share it. On my global diagram "Manage projects" is attached to the Director alone. What the two of them share is "Manage project execution" and "Monitor performance indicators", and they share those because the use case is the same goal — only the perimeter differs.

IF THEY PUSH

Looking at the figure: the Director's four lines go to "Manage projects", "Define the internal quote", "Manage project execution" and "Monitor performance indicators". The chef de projet's four lines go to "Manage project execution", "Manage financial tracking", "Manage project governance" and "Monitor performance indicators". So "Manage projects" is the Director's, and the seeded permission matrix agrees: only DIRECTEUR holds CREATE_PROJECT and ASSIGN_CHEF_PROJET. Now the real question behind yours is a good one — is it legitimate for two actors to sit on one ellipse at all? Yes, and here is why. A use-case diagram cannot draw scope, and scope is what separates these two roles. The Director holds VIEW_ALL_PROJECTS, which lifts the perimeter filter; the chef de projet does not, so ProjectScopeService derives his perimeter from the relations — the projects he manages and the projects he is assigned to. Same goal, same ellipse, different set of rows. I should be honest about one point: the chef de projet does hold EDIT_PROJECT in the matrix, so he can edit a project he manages. I folded that under "Manage project execution" rather than drawing a second line to "Manage projects", because his goal is to run the project, not to define it.

POINT TO `report/img/use_case.png · docs/figures/gen_drawio_usecase.py` — actors: `dir -> u3,u4,u5,u9`
; `pm -> u5,u7,u8,u9 · backend/src/main/resources/db/migration/V2__seed_rbac.sql` —
DIRECTEUR vs CHEF_PROJET permission lists ·
`backend/src/main/java/com/pms/project/service/ProjectScopeService.java`

THEN THEY ASK

Then how does the reader know the Director sees more? — Not from this figure: the scope rule is stated in chapter 2's actor table and enforced by ProjectScopeService, which chapter 5 describes.

DO NOT SAY

Do not agree that the two actors share "Manage Projects" — the figure clearly shows one line, from the Director. Agreeing would make every later answer about actor boundaries look unreliable.

126

Why did you use `<<include>>` with Authenticate? Explain exactly what that relationship means.

MEDIUM

SAY THIS

Include means the base use case always runs the included one, every single time, as a mandatory part of its flow. The arrow goes from the base to Authenticate. On my global diagram nine use cases point at Authenticate that way, so I write the rule once instead of nine times.

IF THEY PUSH

The precise meaning matters, so let me state it. In an include relationship the base use case is incomplete on its own: its behaviour is not fully described until the included use case runs, and it runs unconditionally, not "if needed". Reading one arrow on my figure: "Manage projects --<<include>>--> Authenticate" says that every time someone manages projects, authentication has already happened as part of that flow. I used it because it is literally true of the running system, not because it looks tidy. The Spring Security filter chain runs a JWT filter on every request, and every business service method carries a `@PreAuthorize` permission check — 110 of them across 22 services. There is no path into the platform that skips it, which is exactly the condition for include rather than extend. I will also be honest about the standard objection: many people prefer to model authentication as a precondition, because Authenticate is a technical concern and not a business goal. I know that argument and I accept it is defensible. I chose include because chapter 4 delivers Authenticate as a real use case in Sprint 1, so it exists as a use case in my model and can properly be included by the later ones. One detail people notice: "Freeze the monthly review" has no include arrow of its own. That is deliberate. It is an extension, so it runs inside the flow of "Monitor performance indicators", which already includes Authenticate.

POINT TO `report/img/use_case.png` — nine «include» arrows to Authenticate ·
`docs/figures/gen_drawio_usecase.py` — `deps: [('u1'..'u9', 'auth', INCLUDE)]` ·
`report/chap_02.tex` — "points to the ``Authenticate'' use case through an include relation" · `report/chap_04.tex` — Sprint 1, fig:uc-s1, Authenticate as a delivered use case

THEN THEY ASK

Which direction does the include arrow point? — From the base use case to Authenticate: the base depends on the included one, so the dashed arrow leaves the base and arrives at Authenticate.

DO NOT SAY

Do not say include means "reuse" or "shared code"; that is an implementation reading and he will correct you. Say it means mandatory, unconditional part of the base flow.

127

Why did you use <<extend>> for freezing the monthly review?

MEDIUM

SAY THIS

Because consulting the indicators is complete on its own, and freezing them is optional behaviour that happens only at the monthly review. That is precisely what extend means: a conditional addition to a base use case that does not need it.

IF THEY PUSH

On my global diagram the arrow goes from "Freeze the monthly review" to "Monitor performance indicators", labelled «extend», and the same pair appears again on the Sprint 8 diagram. The direction is the opposite of include: the extension points at the base, because the base does not know about it. The business behaviour matches exactly. A chef de projet opens the indicators any day of the month, as often as he wants, and that is a finished interaction. Once a month, at the revue mensuelle, he freezes the state instead of just reading it. The code shows the same split: KpiService.computeLive recalculates every indicator on the fly for a read, while KpiService.createSnapshot writes a SnapshotKpi row and refuses a second snapshot for the same project on the same date. So one is a pure read that always works, and the other is a rare write with its own rule. One honest point: createSnapshot is guarded by EDIT_PROJECT rather than by a dedicated freeze permission, so a Director technically holds the right too, while my Sprint 8 figure attaches the freeze to the chef de projet only. The figure shows the intended owner of the monthly review, not the only holder of the permission.

POINT TO report/img/use_case.png – «extend» from Freeze the monthly review to Monitor performance indicators · docs/figures/gen_drawio_usecase.py – SPECS['use_case'] deps ('u10', 'u9', EXTEND) and SPECS['uc_sprint8'] deps ('u3', 'u2', EXTEND) · backend/src/main/java/com/pms/kpi/service/KpiService.java – computeLive and createSnapshot · report/chap_06.tex – Sprint 8, fig:uc-s8

THEN THEY ASK

What is the extension point exactly? — The moment the user is looking at the computed indicators for a project: from there he may stop, or he may freeze that state as the month's official snapshot.

DO NOT SAY

Do not say extend means "an extra feature"; say it means optional and conditional, and that the base use case is complete without it.

SAY THIS

It is the one-page map of everything the platform stores, and it settles two things before any sprint starts: what belongs to a project, and how authorization is modelled. Fifteen classes, drawn once in chapter 2, then refined by six detailed diagrams in the sprints.

IF THEY PUSH

The figure has fifteen classes with Project in the middle, because every business class reaches Project. Its first job is to show that this domain is one connected whole — computing a single indicator reads workload, billing, cost rates and the *devis interne* (the internal quote) at once — and that connectedness is the argument chapter 3 uses to justify a modular monolith instead of microservices. Its second job is to settle the role question, which is easy to get wrong. The four roles are not subclasses of User; they are rows of the Role class. A user holds a role, a role grants permissions, so the authorization policy is data an administrator edits, not a hierarchy compiled into the application. Modelling the roles as types would have made every policy change a new release. Its third job is to be the contract the sprint diagrams refine: projects and teams in Sprint 3, agile planning in Sprint 5, financial management in Sprint 6, governance in Sprint 7, the indicator engine in Sprint 8, and the authorization model in Sprint 2. It is not the database schema — the schema is chapter 3, built by 28 Flyway migrations, and it has more tables than this figure has classes.

POINT TO docs/figures/class_specs.py — CLASS_GLOBAL, 15 classes · report/chap_02.tex — section "Class diagram", fig:class-global · report/chap_03.tex — monolith justification, "transactional consistency" · backend/src/main/resources/db/migration/ — 28 migrations V1 to V28

THEN THEY ASK

Why are the roles not subclasses of User? — Because a subclass is compiled in: changing who may do what would need a code change and a redeploy, and the whole point of the dynamic RBAC is that it is a data change.

DO NOT SAY

Do not present the global class diagram as the database model; he will then ask why table counts do not match and you will be defending a claim you never made.

CAREFUL — THE QUESTION IS NOT QUITE RIGHT

The report does say "the project is the aggregation root", so the premise holds. One wording precision is worth making yourself before he does: the relations drawn are UML compositions (filled diamond), not aggregations (hollow diamond). "Aggregate root" there is the domain-driven-design term, not the UML one.

SAY THIS

Because ten classes in the model cannot exist without a project and have no meaning outside one, so Project owns them. In the figure those ten are drawn with a filled diamond on the Project side, and in the database fourteen tables carry a mandatory project_id.

IF THEY PUSH

The ten owned classes are TeamAssignment, WorkloadPlan, ActualWorkload, BillingMilestone, KpiSnapshot, BacklogItem, Mission, Deliverable, Risk and Sprint. Each is drawn 1 on the Project side and 0..* on its own side, because a project starts with no sprints and an empty backlog, and that zero is a fact worth writing. The test I used is the one my supervisor gave me: a composition means the part cannot exist without the whole. A jalon de facturation (a billing milestone) with no project is meaningless; so is a line of plan de charge. Fourteen tables in the schema carry a NOT NULL project_id foreign key to projects(id), which is the same statement in SQL. This is also what makes the indicator engine possible: one project id is enough to reach workload, billing, cost rates, the devis interne and the deliverables inside a single transaction, with no distributed read. Two things are deliberately outside the root. User, Role, Permission and Resource form a reference core that stands on its own, so Project reaches a user only through plain associations — "directed by" and "managed by" — never by owning him. And the parties prenantes (stakeholders) are an association too, not a composition, because they are people and companies that outlive the project. One honest bound: composition normally implies the part dies with the whole, and PMS never hard-deletes — every business table has a deleted flag and a deletion is a state change, for auditability. So the ownership is real, but the cascade is not exercised.

POINT TO docs/figures/class_specs.py — CLASS_GLOBAL 'rel', ten relations marked 'comp' ·
 report/chap_03.tex — "the project is the aggregation root" ·
 backend/src/main/resources/db/migration/ — V6 to V11, V23, V27: project_id BIGINT NOT
 NULL REFERENCES projects(id) · report/chap_03.tex — "Auditability and logical deletion"

THEN THEY ASK

Then why does only team_assignments have ON DELETE CASCADE? — Because it was the first child table written, before the logical-deletion rule was fixed; once no row is ever physically removed the cascade never fires, so the mandatory foreign key is what actually enforces the ownership.

DO NOT SAY

Do not claim the parts are deleted with the project — they are not, nothing is hard-deleted, and he may well check.

130

Why did you separate User and Resource instead of making them the same entity?

MEDIUM

SAY THIS

Because they answer two different questions. User answers "who may log in and with what rights". Resource answers "what does this person's working day cost". Not every user is a resource, and the two are administered by different people under different permissions.

IF THEY PUSH

In the global class diagram the link reads Resource 0..1 to User 1: a resource always belongs to exactly one user, but a user has zero or one resource. That zero is the whole point — the administrator and the Director log in every day and are never staffed on a project, so they have no daily rate. If I had merged the two I would carry null rates on those rows and check for null in every cost computation. The attributes are different too: User holds email, passwordHash, active, firstLogin, tokenVersion and role; Resource holds dailyRate, tccRate, staffingStart, staffingEnd and computes the annual cost as dailyRate times (1 + tccRate) times 218 working days. The strongest argument is the permission matrix. Account administration is `MANAGE_USERS` in the `ADMIN` module; the cost rate is `MANAGE_RESOURCES` in the `RESSOURCE` module. The Director holds `MANAGE_RESOURCES` but not `MANAGE_USERS` — he may set a TCC and may not create an account. With one entity that separation could not be expressed at all. There is also a lifecycle reason: the TCC (taux de coût chargé, the loaded cost rate) is held per year and the historical value must stay immutable, while a user account has no yearly version. The one-to-one is enforced by a unique non-null foreign key, so the join costs nothing.

POINT TO backend/src/main/java/com/pms/user/entity/Resource.java – @OneToOne
@JoinColumn(name="user_id", nullable=false, unique=true), getAnnualCost() ·
backend/src/main/java/com/pms/user/entity/User.java ·
backend/src/main/resources/db/migration/V2__seed_rbac.sql – DIRECTEUR holds
MANAGE_RESOURCES, not MANAGE_USERS ·
backend/src/main/resources/db/migration/V4__add_unique_resource_user.sql ·
backend/src/main/resources/db/migration/V21__tcc_annuel.sql

THEN THEY ASK

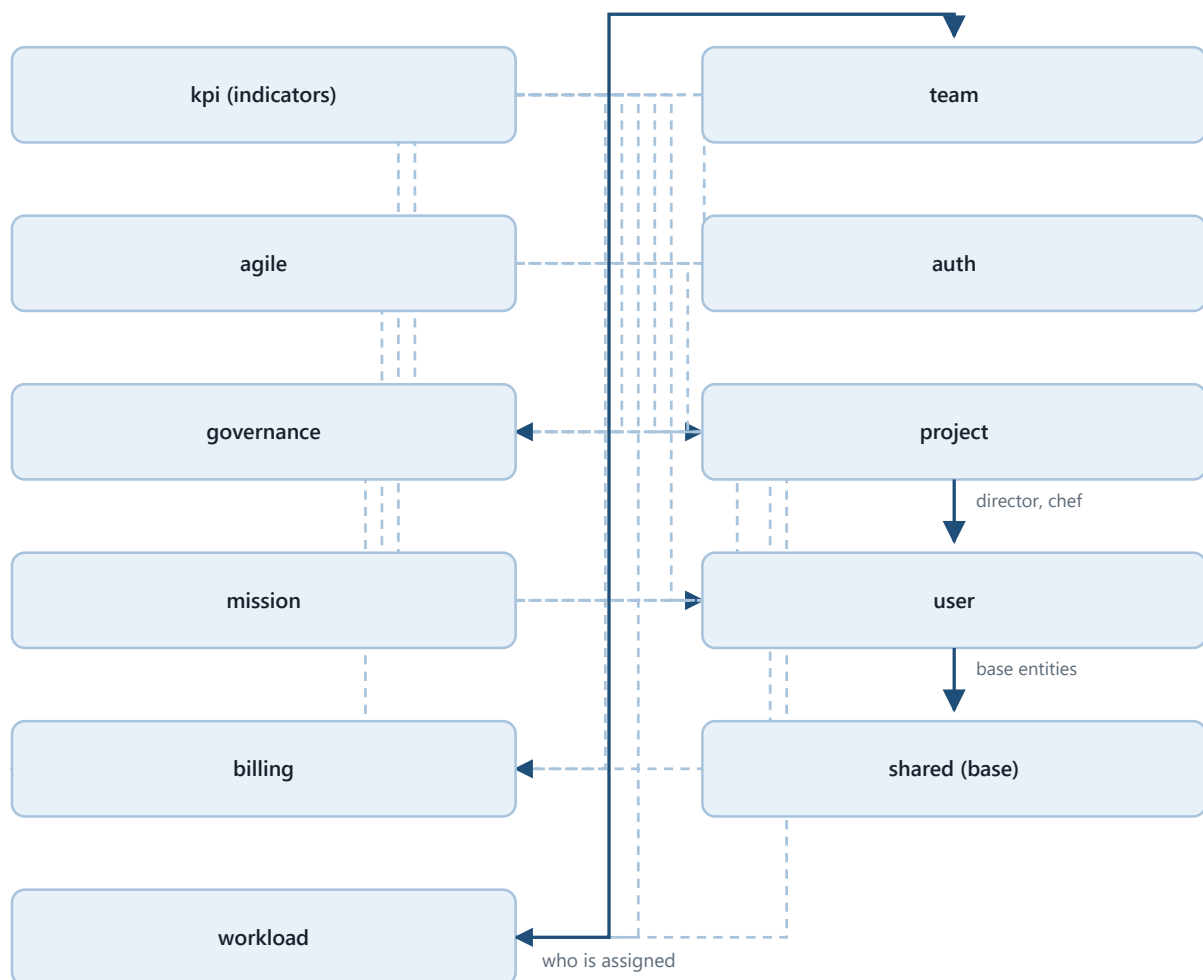
Is a one-to-one table not over-normalisation? — It would be if the two sides had the same owner and the same lifecycle; here they have different permissions, different yearly versioning and different sensitivity, and the unique foreign key makes the join free.

DO NOT SAY

Do not justify it only as "clean design"; give the permission argument, because that is the one he cannot refuse.

D. Architecture

This examiner is checking whether you made architecture choices on purpose or copied a tutorial. He wants a reason tied to your real situation: one team, one company, one database, money that must add up. The posture to take: be calm, give the reason before the name of the pattern, and where the code does not fully match the rule you wrote down, say so first and say what the fix is. A jury trusts a candidate who audits his own repo more than one who recites his own documentation.



PMS module dependency graph — KPI on top, shared at the bottom — Arrows point from the module that depends to the module it depends on: kpi only reads and nothing reads kpi, shared is a pure sink, and the single red edge project -> billing is the one real cycle I found when auditing the imports.

SAY THIS

Because the domain is tightly joined and the deployment is tiny. One KPI reads workload, billing, cost rates and the devis interne in one go. Splitting that into services would have brought me distributed transactions and no functional gain, for one company and one team.

IF THEY PUSH

I wrote the reason down in ADR-016 before I built anything. First reason: transactional consistency. To compute one margin, KpiService reads the plan de charge (the planned workload), the charge réelle (the actual workload), the TCC (the yearly cost rate per resource), the jalons de facturation (the billing milestones) and the devis interne (the internal quote) inside a single read transaction. If those lived in five services I would need distributed transactions or eventual consistency, and a margin that is eventually correct is not correct. Second reason: the scale. PMS is used inside one consulting company, ST2i, on an internal network; the operating cost of a distributed system would never be paid back by the load. Third reason: I was one developer with eight months, so every hour spent on service discovery, inter-service contracts and distributed tracing was an hour not spent on the Excel financial rules, which are where the real value is. ADR-026 later added Docker Compose with three containers, and I want to be clear that three containers are not three services: one application, its database, and an nginx serving the Angular build.

POINT TO DECISIONS.md – ADR-016 "Modular monolith, layered, feature-based" · DECISIONS.md – ADR-026 ("ADR-016 is unaffected – three containers are not three services") · report/chap_03.tex – Section "Architectural style", the three arguments · backend/src/main/java/com/pms/kpi/service/KpiService.java

THEN THEY ASK

So you would never use microservices? — I would, when a part needs its own scaling or its own release rhythm and can live with delayed consistency; none of my eleven modules has that need today, and my ENHANCEMENTS decision log records the split as considered and rejected, not as unknown.

DO NOT SAY

Do not say microservices are modern and monoliths are old. That frames it as fashion rather than as a fit to the problem, and it invites him to ask which version you would pick if fashion changed.

132

What is the difference between a monolith and a modular monolith?

EASY

SAY THIS

Both ship as one artefact. The difference is inside. A plain monolith lets any class call any other class. A modular monolith cuts the code into named modules with a rule about who may call whom, and that rule is written down and meant to be checked.

IF THEY PUSH

A monolith is a packaging fact: one build, one deployable, one database. That says nothing about the inside. What people call a big ball of mud is a monolith where the inside has no boundaries, so a change anywhere can break anything. A modular monolith keeps the same packaging but adds structure: my backend is cut into eleven Java packages under `com.pms` — `auth`, `user`, `project`, `team`, `workload`, `billing`, `mission`, `governance`, `agile`, `kpi`, plus `shared` — and the Angular frontend mirrors the same feature folders. ADR-016 states the boundary rules: a module never injects another module's repository, cross-module work goes through the other module's published service, each module owns its own entities, and there are no circular dependencies. The point is that if I ever wanted to pull one module out into its own service, the seam is already drawn. I will be honest about the enforcement: those rules are written but nothing in the build fails when they are broken, and I will come back to that.

POINT TO `backend/src/main/java/com/pms/` — the eleven packages · `frontend/pms-`
`frontend/src/app/features/` — twelve mirrored feature folders · `DECISIONS.md` — ADR-016,
module boundary rules 1 to 5 · `docs/ARCHITECTURE.md` — section 2

THEN THEY ASK

How do you keep a modular monolith from turning into a plain monolith over time? — Only a test can do it; package discipline and code review drift, so the honest answer is an ArchUnit rule in the build, which I have specified but not yet written.

DO NOT SAY

Do not answer only with the definition and stop. The examiner is really asking whether YOUR monolith is modular, so bring your own package list in as proof.

133

What happens if the number of projects and users becomes much larger than expected?

MEDIUM

SAY THIS

The design holds better than you might expect, because almost every query is scoped to one project, not to the whole table. What breaks first is not the database, it is the KPI computation on read and the list screens I did not paginate. Both have known, cheap fixes.

IF THEY PUSH

Nearly all the heavy reading is per project. Every data call in KpiService is filtered by projectId — findActiveByProjectId on plan de charge, charge réelle, livrables and jalons — so the cost of one dashboard depends on the size of one project, not on how many projects exist. Growth in the number of projects therefore costs more rows in the tables, which is what indexes are for: my 28 Flyway migrations create 32 indexes, and all the monthly data is normalised as one row per project, resource and month rather than as twenty-five month columns. Pagination exists on the three endpoints that list the most rows — projects, users and workload — and the rest of the lists are per project and naturally small. The first real limit is the KPI engine: it recomputes on read, so a dashboard showing many projects at once grows linearly with the number of projects; the fix is to serve those rollups from the kpi_snapshots table, which already exists, instead of recomputing. The second limit is that I run a single backend container, so beyond that I would put two instances behind nginx, which works because authentication is a stateless JWT and only a small per-user permission cache is in memory. I have not load-tested any of this, so I am giving you the shape of the bottleneck, not a measured number.

POINT TO backend/src/main/java/com/pms/kpi/service/KpiService.java — every repository call filtered by projectId · backend/src/main/resources/db/migration/ — 28 migrations, 32 CREATE INDEX statements · backend/src/main/java/com/pms/project/repository/ProjectRepository.java and workload repositories — Pageable · DECISIONS.md — ADR-015 (kpi_snapshots per project and per month)

THEN THEY ASK

Have you measured it? — No, I have not run a load test; my non-functional target was a two second response on realistic company data, and I verified that by hand in the demo dataset, which is why I am describing where it would break rather than claiming a number.

DO NOT SAY

Do not say "PostgreSQL scales, so it is fine". That dodges the question. Also do not invent a request-per-second figure; you never ran a benchmark and he may ask for the tool you used.

134

Why do you claim that your architecture is reversible?

HARD

SAY THIS

Reversible means I kept the option open, not that I proved it. The code is already cut along the lines I would cut a service, and the vertical layering inside each module is clean. I will be straight with you: some modules still reach into another module's repository, so an extraction today would need that cleaned first.

IF THEY PUSH

The claim in chapter 3 is about optionality: partitioning the code into modules with explicit boundaries preserves the choice of extracting one later. Two things really are in place. First, the seams: the backend is split into feature packages and the Angular app mirrors them, so pulling out, say, agile or mission means moving a package and putting an HTTP call where a Java call was. Second, the layering is

genuinely respected in the direction that matters most: 109 of the 110 `@PreAuthorize` annotations sit on service methods, all 121 `@Transactional` annotations are on services and none on controllers, and only one controller in the whole backend touches a repository directly, `UserController` for a read-only role list. So the transaction and authorization boundary is the service layer, which is exactly the boundary a future service would expose. The honest gap is horizontal: 20 files import a repository that belongs to another module, which ADR-016 rule 1 forbids. Those are the joins I would have to convert into service calls before an extraction, and until that is done reversibility is a design property I have preserved, not one I have demonstrated.

POINT TO `report/chap_03.tex` – "Reversibility" bullet ·
`backend/src/main/java/com/pms/user/controller/UserController.java` – the single controller that injects a repository · `DECISIONS.md` – ADR-016 rule 1 "No cross-module repository access" · `docs/ENHANCEMENTS.md` – decision log D-1

THEN THEY ASK

Which module would you extract first, then? — agile, because it is the newest and the least entangled: it imports project, team and user but nothing imports it back, so it is a leaf I could cut with one API contract.

DO NOT SAY

Do not claim the architecture is "ready to be split". It is not, and he can check in thirty seconds by opening `KpiService`. Claim the option, not the readiness.

135

Why is the dependency graph between your modules intentionally acyclic?

HARD

SAY THIS

Because a cycle means two modules are really one: you cannot test, understand or extract either alone. The direction I designed is acyclic and mostly holds. I audited it again for this defence and found one real cycle, between project and billing, which I can show you.

IF THEY PUSH

The reason to forbid cycles is practical. If A depends on B and B depends back on A, you cannot compile, test or reason about either one on its own, and the reversibility argument dies immediately. So ADR-016 rule 4 says no circular module dependencies, and chapter 3 says the graph is deliberately acyclic and converges towards kpi. When I grepped every `com.pms` import for this preparation, the direction is right almost everywhere: shared is a pure sink that the others depend on, user sits just above it, project is the hub that team, workload, mission, governance, agile and billing all point to, and kpi sits on top pointing down at everything. There is one genuine cycle and I am not going to hide it: `ProjectService` imports `com.pms.billing.service.JalonService` to recompute milestone amounts when a budget changes, while `JalonService` and `AvenantService` import `com.pms.project.repository.ProjectRepository`. That is project depending on billing and billing depending back on project. The clean fix is an event: `ProjectService` publishes a budget-changed event and billing listens, which is exactly rule 4's own advice to prefer events for fan-out side effects. And the

reason I only found this by grepping is that nothing enforces the rule; an ArchUnit test, which is one test class using `slices().matching("com.pms.*").should().beFreeOfCycles()`, would have failed the build the day I wrote that import.

POINT TO `backend/src/main/java/com/pms/project/service/ProjectService.java` – line 3 imports `com.pms.billing.service.JalonService`, line 163 calls `jalonService.recomputePrevuMontants(saved)` ·
`backend/src/main/java/com/pms/billing/service/JalonService.java` and `AvenantService.java` –
import `com.pms.project.repository.ProjectRepository` · DECISIONS.md – ADR-016 rule 4 ·
`backend/pom.xml` – no ArchUnit and no Spring Modulith dependency

THEN THEY ASK

Why did you not add ArchUnit during the project? — Honestly because the rule lived in a document and I trusted my own discipline; that is precisely the mistake, and it is a half-day of work I would do first if I continued.

DO NOT SAY

Do not say flatly "my graph is acyclic". The examiner may have the repo open, and one grep would contradict you. Owning the project-billing cycle first makes the rest of the answer credible.

136

Why does the KPI module depend on other modules rather than being a dependency of them?

HARD

SAY THIS

Because KPI only reads. It produces no fact of its own, it consumes the facts the other modules own, so it has to sit at the end of the chain. If any module depended on kpi, I would have a cycle instantly, since kpi already reads from five of them.

IF THEY PUSH

The rule I applied is that the module that owns the data sits below, and the module that only derives numbers sits on top. KPI derives everything: the margin, the EAC, the ETC and the EV all come from other modules' rows. So the arrows go downwards from kpi and nothing points back at it — I checked, and no runtime class outside kpi imports `com.pms.kpi`, only the demo data seeders do, and those are bootstrap code, not a module. Now the honest part, because the direction is right but the coupling is too deep. `KpiService` injects eight repositories, and seven of them belong to other modules: `ProjectRepository`, `PlanChargeRepository`, `ChargeReelleRepository`, `ResourceRepository`, `TccAnnuelRepository`, `LivableRepository` and `JalonFacturationRepository`; only `SnapshotKpiRepository` is its own. That breaks ADR-016 rule 1, which says a module talks to another module's service, not its repository. One dependency does it correctly, `DevisInterneService` from the project module, and that is the shape all eight should have: seven small read-only service methods returning DTOs instead of seven repositories. It works and the numbers are right, but it means a change to the workload schema can break the KPI engine, which is exactly what the rule was written to prevent. The fix is the same ArchUnit test: forbid any class in `com.pms.X` from depending on `com.pms.Y..repository`.

POINT TO backend/src/main/java/com/pms/kpi/service/KpiService.java – the eight injected repositories plus DevisInterneService · report/chap_03.tex – "converges towards the kpi module, which consumes the others without being consumed by any" · DECISIONS.md – ADR-016 rules 1 and 2 · DECISIONS.md – ADR-015 (hybrid KPI, snapshot per project and per month)

THEN THEY ASK

Would events not be better than eight read calls? — For reading, no: a dashboard needs the current value, and an event bus for one process computing sums over a few hundred rows is accidental complexity, which is why my decision log rejects the async KPI pipeline; events belong on the write side, like the project-billing case.

DO NOT SAY

Do not claim KPI only talks to services. It injects repositories and the file is 294 lines long; state the number yourself, with the seven-out-of-eight split, before he finds it.

137

Why did you choose a layered architecture with Controller, Service, Repository and Domain?

MEDIUM

SAY THIS

Because each layer has exactly one job, and that makes the important boundary obvious: the service is where the transaction starts, where authorization is checked, and where the Excel rules live. Controllers just translate HTTP, repositories just talk to SQL.

IF THEY PUSH

Each layer answers one question. The controller answers "what does HTTP look like": it takes the request, validates the shape, and returns a DTO. The service answers "what is the business rule": it opens the transaction, checks the permission and applies the Excel method. The repository answers "how do I get the rows", including filtering out logically deleted ones. The domain holds the entities and the invariants. The test that this is real and not decoration is the annotations: all 121 @Transactional annotations sit on services and not one is on a controller, and 109 of the 110 @PreAuthorize annotations are on service methods. That means you cannot reach a business operation without crossing the security and transaction boundary, whatever the entry point is. It also made the project testable as one developer: my 97 tests in 13 test classes mostly target services, which is why the modules where the money is computed are the best covered — auth at 81.3% of lines, kpi 78.9%, team 76.7%, agile 75.5%, billing 75.0%, project 72.4% — against 41.2% overall. The cost of layering is boilerplate, four files where one class would do, and I accepted that because financial correctness mattered more than line count.

POINT TO DECISIONS.md – ADR-016 "Controller → Service → Repository → Domain" · report/chap_03.tex – "Layered organisation", Figure fig:layers · backend/src/main/java/com/pms/ – 18 controllers, 25 services, 22 repositories, 38 entities · backend/target/site/jacoco – 41.2% lines overall, per-module figures

THEN THEY ASK

Is 41.2% coverage not low? — It is low as an average, and I do not defend the average; I defend the distribution, because the modules that compute money are between 72% and 81%, while the weak numbers are mission at 3.9% and user at 4.6%, which are CRUD, and those are the next tests I would write.

DO NOT SAY

Do not describe layering as "best practice" with no evidence. Give the annotation counts; they prove the layers are enforced rather than merely drawn.

138

Why should business rules be in the Service layer rather than in Angular?

MEDIUM

SAY THIS

Because the browser is not trustworthy and it is not the only client. Anyone can call my REST API with curl and skip Angular completely. If the margin rule lived in TypeScript, there would be no rule at all, just a suggestion.

IF THEY PUSH

Three reasons, and the first one is security. Angular runs on the user's machine, so a rule enforced there is a rule the user can edit. That is why all 109 service-level `@PreAuthorize` checks are on the backend and the Angular guards and the permission directive only hide buttons; hiding a button is comfort, not protection. The second reason is that there must be one single version of the truth. The whole point of PMS is to replace an Excel workbook of 21 linked tabs and 2088 formulas where every project had its own copy and the copies drifted. If I reimplemented the margin in TypeScript for display and in Java for storage, I would have recreated exactly the problem I was hired to remove. So the Excel method lives once, in `KpiService` and `DevisInterneService`, and Angular only renders what the API returns. The third reason is data integrity: rules often need several tables at once and must be all-or-nothing, and only the backend has a transaction. Angular does keep some logic, and I want to be precise about that: form validation for fast feedback, formatting, and choosing which screen to show. It duplicates a check, it never owns one.

POINT TO `backend/src/main/java/com/pms/kpi/service/KpiService.java` – the margin, EAC, ETC and EV calculations · `backend/src/main/java/com/pms/project/service/DevisInterneService.java` – the devis interne (internal quote) amounts, computed at read time and never stored · `DECISIONS.md` – ADR-021 (permission $\wedge$ data scope, enforced server side) · `report/chap_03.tex` – "Dynamic authorization"

THEN THEY ASK

So the Angular guards are useless? — No, they are a user-experience layer: they stop a manager from seeing a menu he cannot use, but the backend refuses the call anyway, so removing the guards would make the app uglier and not less safe.

DO NOT SAY

Do not say Angular has "no logic at all". It has validators and guards, and he can open the code; say instead that it duplicates checks and owns none.

139

Why did you use DTOs instead of returning JPA entities directly from your REST API?

EASY

SAY THIS

Because an entity is shaped for the database and a DTO is shaped for the caller, and those are not the same shape. Returning entities would leak my schema, leak fields nobody should see, and tie my API contract to every migration I write.

IF THEY PUSH

There are four concrete problems and the DTO solves all of them. Confidentiality first: the User entity holds the BCrypt password hash and the token version counter that drives session revocation, and neither belongs in a JSON response; my UserResponse simply does not have those fields, so it cannot leak by accident. Second, lazy loading: 28 of my relations are FetchType.LAZY, and serialising an entity straight out of a controller either explodes with a LazyInitializationException or silently pulls half the database into the response. Third, contract stability: the schema is owned by Flyway with Hibernate in validate mode, so it changes whenever I write a migration, and I do not want a column rename to become a breaking API change for the Angular client. Fourth, the API often needs things the table does not have: ProjectResponse carries directorName and chefProjetName, which are joins, and effectiveBudget, which is the budget plus the avenants (contract amendments) and is computed, not stored. I have 52 DTOs against 38 entities, which is the normal ratio once you have separate request and response shapes, and ADR-016 states the rule plainly: entities never cross the API boundary.

POINT TO backend/src/main/java/com/pms/project/dto/ProjectResponse.java and project/mapper/ProjectMapper.java – directorName, chefProjetName, effectiveBudget · backend/src/main/java/com/pms/user/entity/User.java – password hash and token version, absent from the DTO · DECISIONS.md – ADR-016 ("entities never cross the API boundary (DTOs do)") and ADR-019 (Flyway owns the schema, JPA in validate) · backend/src/main/java/com/pms/ – 52 DTOs, 38 entities, 28 LAZY relations

THEN THEY ASK

Is that not a lot of duplicated classes? — Yes, and I accept it as the price of the boundary; MapStruct removes the copying code so the duplication is only in the field declarations, which is the part I actually want to control.

DO NOT SAY

Do not justify DTOs only with "good practice" or "avoids the LazyInitializationException". The password-hash leak is the argument the jury remembers, so lead with it.

SAY THIS

Because it generates the mapping code at compile time. There is no reflection, so it is fast, and I can open the generated class and read exactly what it does. I made it mandatory in an ADR so that no module would drift into hand-written mappers.

IF THEY PUSH

MapStruct is an annotation processor: I declare an interface with `@Mapper(componentModel = "spring")` and at build time it writes a plain Java class that copies field by field, which Spring then injects like any other bean. I compared it against two alternatives. Hand-written mappers work but are pure boilerplate and rot silently when someone adds a field. Reflection-based mappers like `ModelMapper` need no code but resolve the mapping at runtime, which costs performance and, worse, turns a mistake into a wrong value in production instead of a compile error. MapStruct gives me both: no boilerplate and errors at build time. It also handles the derived fields cleanly — in `ProjectMapper` I map `directorName` through a `@Named` helper and `effectiveBudget` through an expression, so the computed value stays in the entity where the rule lives. ADR-018 makes it mandatory across all 18 mappers precisely so the codebase stays uniform. One honest gap: ADR-018 also required `unmappedTargetPolicy = ERROR`, so that a forgotten target field fails the build, and no mapper actually sets it. Right now a field I forget comes back as null instead of breaking the compile. That is one attribute to add in 18 files and it is on my list.

POINT TO DECISIONS.md – ADR-018, including the `unmappedTargetPolicy = ERROR` requirement ·
backend/pom.xml – `mapstruct 1.5.5.Final` plus the `mapstruct-processor` in the compiler plugin ·
backend/src/main/java/com/pms/project/mapper/ProjectMapper.java ·
backend/src/main/java/com/pms/ – 18 mapper interfaces, none setting `unmappedTargetPolicy`

THEN THEY ASK

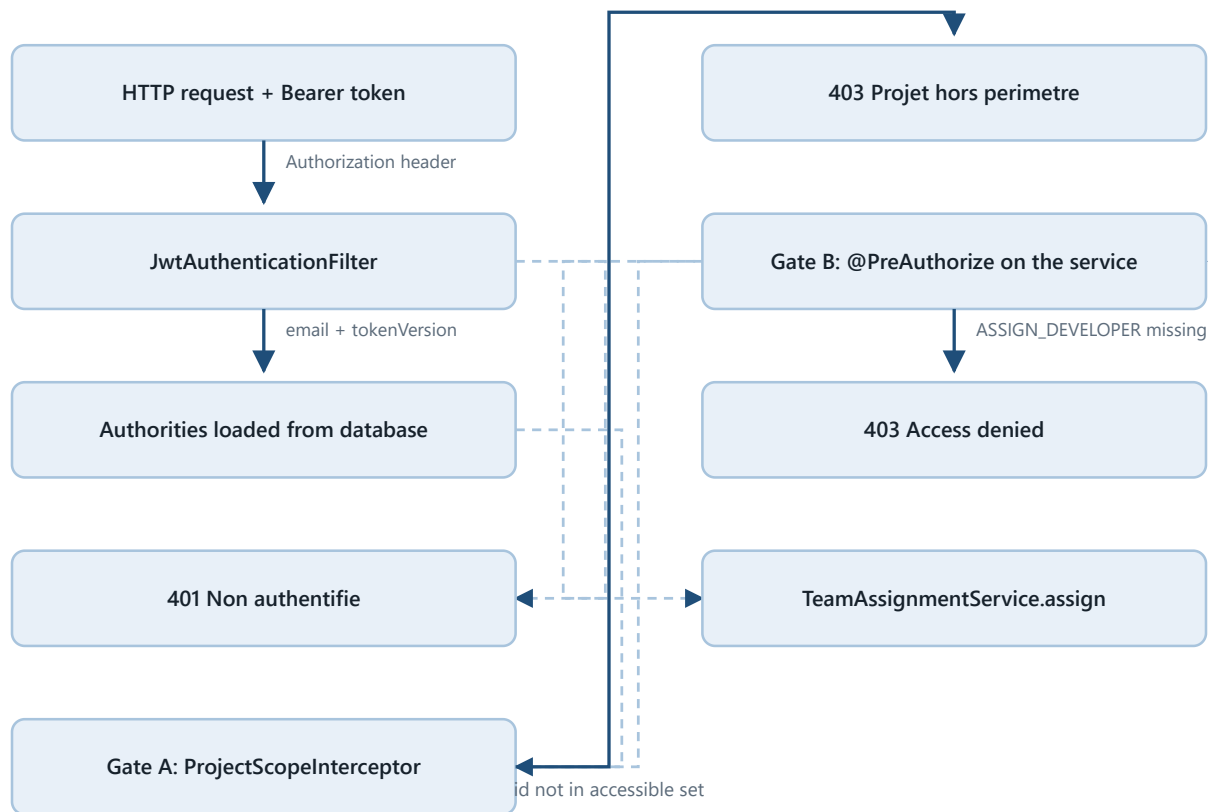
What happens if the annotation processor is misconfigured? — The build fails loudly, because the generated implementation class simply does not exist and Spring cannot inject the bean; the one real trap is processor ordering with Lombok, which is why both are declared together in the compiler plugin.

DO NOT SAY

Do not say MapStruct is "automatic" as if it needed no thought. The interesting part is that it is compile-time and readable, and admitting the missing `unmappedTargetPolicy` shows you know what the tool is actually for.

E. Security and dynamic RBAC

This examiner wants to know if you understand your own security, or if you copied a tutorial. Every answer should point at a real file, a real column or a real migration. Two ideas carry the whole block: authorization is data in the database, not role names in the code; and a permission says what you may do while the scope says which projects you may do it on. Own the two honest weaknesses (the access token in localStorage, and revocation being per user not per device) before he finds them.



The two gates: permission and project scope — It shows that a permission alone never reaches the data: the request must also survive the scope check, and in Spring the interceptor happens to run before the method-security check on project URLs.

SAY THIS

My backend is a stateless REST API and the Angular app is a separate origin on its own port. With a signed JWT the server checks a request with the signing key alone, so I keep no session in memory and no sticky sessions in Docker. The token also carries the token version, which is what lets me revoke a session instantly.

IF THEY PUSH

The two parts of PMS are separate programs: Spring Boot on port 8080 and Angular on 4200, each in its own Docker container. A classic HTTP session would mean server-side state, a session store, and sticky routing if I ever ran two backend containers. I set `SessionCreationPolicy.STATELESS` in `SecurityConfig.java` and send the identity in the Authorization header instead. The token is signed with HMAC-SHA512 from a secret read from the `JWT_SECRET` environment variable, which must be at least 64 characters; `application.yml` documents that. The claims are the e-mail as subject, the role name, the permission codes, the `tokenVersion` and a type claim. One honest detail: the permissions claim inside the token is not what the server trusts. `JwtAuthenticationFilter` reloads the role's permissions from the database (through a small cache keyed on email plus token version), so a permission change takes effect even on a token that was already issued. That claim is effectively dead weight today and I would remove it in a next version.

POINT TO `backend/src/main/java/com/pms/auth/service/JwtService.java` ·
`backend/src/main/java/com/pms/shared/config/SecurityConfig.java` ·
`backend/src/main/resources/application.yml` (`pms.jwt`) · report chap_03, "Authentication and session revocation"

THEN THEY ASK

So your system is not really stateless? — Correct, and that is on purpose: the signature check is stateless, but I do one cheap cache-backed lookup per request to compare the token version and reload permissions, which is the price of instant revocation.

DO NOT SAY

Do not say "JWT is more secure than sessions". It is not; it is more convenient for a separated frontend and backend. Saying that invites a hard follow-up.

142

What is the difference between an access token and a refresh token?

EASY

SAY THIS

The access token is the one used on every call. It lives fifteen minutes and travels in the Authorization header. The refresh token lives seven days, never touches JavaScript, and its only job is to obtain a new access token from one endpoint.

IF THEY PUSH

In application.yml access-token-expiration is 900 seconds and refresh-token-expiration is 604800 seconds, so fifteen minutes against seven days. The access token carries the subject e-mail, the role, the permission codes, the token version, a type claim set to "access" and the firstLogin flag. The refresh token carries only the subject, the token version and type "refresh". That type claim is the separation: JwtAuthenticationFilter authenticates only a token whose type is "access", and AuthService.refresh rejects anything whose type is not "refresh", so neither can be used in the other's place. The refresh token is also single-use: every successful refresh calls user.revokeAllTokens(), which bumps the token version, so the cookie that was just presented stops working immediately. That is rotation, and a replay of an old cookie is logged as a possible theft before returning 401. One correction to my own report: chapter 3 calls the refresh token "opaque". It is in fact a signed JWT as well; only its storage and its reach are different.

POINT TO backend/src/main/java/com/pms/auth/service/JwtService.java ·
 backend/src/main/java/com/pms/auth/service/AuthService.java (refresh) ·
 backend/src/main/resources/application.yml ·
 backend/src/test/java/com/pms/auth/AuthControllerTest.java ("Rotation : le refresh token original est invalidé")

THEN THEY ASK

Why fifteen minutes and not one hour? — Fifteen minutes bounds the damage of a stolen access token, and because the Angular interceptor refreshes silently on a 401 the user never notices the boundary.

DO NOT SAY

Do not say the refresh token is opaque just because the report says so; he may have read JwtService.java. Correct it yourself, it costs nothing and shows you know your code.

143

How can you immediately revoke a JWT if JWTs are normally stateless?

MEDIUM

SAY THIS

I keep one integer per user called token_version. Every token I issue carries the value it was issued with, and every request compares that value with the one in the database. Adding one to that column makes every token of that user fail on the next call.

IF THEY PUSH

The users table has a token_version column and the User entity exposes revokeAllTokens(), which simply increments it. JwtService puts the current value into every access and refresh token. On each request JwtAuthenticationFilter builds a cache key of "email:version"; on a cache miss it loads the user and, if user.getTokenVersion() differs from the value in the token, it returns without setting an Authentication, so the request ends as 401. The version is bumped on logout, on every refresh (that is the rotation), on a password change, and when an administrator deletes or deactivates an account; each of those also evicts the old cache entry for email plus old version, so a revoked token can never be served from a warm cache. It also works from SQL: migrations V12, V13 and V25 each end with UPDATE users SET token_version = token_version + 1 for the affected roles, so a change to the permission matrix pushes those users back through authentication. The honest trade is that this makes the authorization step slightly stateful: one small cached lookup per request, which I accepted in exchange for revocation that takes effect on the very next call instead of after a fifteen minute wait.

POINT TO backend/src/main/java/com/pms/user/entity/User.java (revokeAllTokens) ·
backend/src/main/java/com/pms/auth/security/JwtAuthenticationFilter.java ·
backend/src/main/java/com/pms/auth/service/AuthService.java (logout, refresh,
changePassword) · backend/src/main/resources/db/migration/V12__fix_rbac_matrix.sql ·
report chap_04, Sprint 1, "Immediate revocation by token version"

THEN THEY ASK

What if the cache node is restarted? — Nothing breaks: an empty cache just means the filter reads the user row from the database, which is the authoritative value anyway.

DO NOT SAY

Do not claim revocation is "free". It costs one lookup per request; say so, and say why the cache keeps that cost small.

144

Why did you use a token version rather than maintaining a blacklist of revoked JWTs?

MEDIUM

SAY THIS

A blacklist stores one row per revoked token and every request has to search it, so the store only grows and it brings back the shared state I was trying to avoid. A version is one integer that already sits in the user row I load anyway, and adding one to it kills every token of that person at once.

IF THEY PUSH

A blacklist means a second store, in practice Redis or an extra table, plus a cleanup job to drop entries once the tokens expire, plus a lookup on the hot path of every request. The token version costs nothing extra: JwtAuthenticationFilter already loads the user to build the authorities, so the comparison is free. It also matches the real requirement at ST2i. The company asked for "this person must not get in any more" when someone leaves or an account is deactivated, never for "kill this one token but keep the others". One increment handles the first case perfectly. There is a second advantage that decided it for

me: a policy change can be pushed from a migration. V12 rebuilt the whole permission matrix and ended with one UPDATE that bumped the version of every affected user, so nobody kept an old authority set. A blacklist cannot do that, because it does not know which tokens exist. The limit I accept is that revocation is all-or-nothing per user: I cannot sign a user out of one laptop while keeping their phone signed in. For a company of thirty to forty people that was a conscious and reasonable trade.

POINT TO backend/src/main/java/com/pms/auth/security/JwtAuthenticationFilter.java ·
backend/src/main/java/com/pms/user/entity/User.java ·
backend/src/main/resources/db/migration/V12__fix_rbac_matrix.sql ·
backend/src/main/resources/db/migration/V25__rbac_admin_module.sql · report chap_04,
Sprint 1, "Revocation by version rather than by blacklist"

THEN THEY ASK

How would you add per-device revocation later? — I would move from one integer to a small table of active sessions per user, keyed by a session id placed in the token, and compare on that id instead of on a single version.

DO NOT SAY

Do not pretend the version approach has no downside. Name the per-device limit before he does.

145

Why is the refresh token stored in an HttpOnly cookie?

MEDIUM

SAY THIS

HttpOnly means JavaScript cannot read the cookie, so a cross-site scripting attack cannot steal my long-lived credential. I also pinned its path to the refresh endpoint and set SameSite to Strict, so the browser sends it to exactly one URL and never on a request started by another site.

IF THEY PUSH

AuthController writes the cookie as pms_refresh with HttpOnly, Path=/api/auth/refresh, SameSite=Strict and Max-Age equal to the seven-day expiry; the Secure flag comes from pms.security.cookie-secure, driven by the COOKIE_SECURE environment variable, so it is on behind HTTPS and off in local development. The reasoning is about what is worth stealing. The refresh token can mint new sessions for seven days, so it must be the part that script on the page cannot touch. Combining that with rotation means even a stolen cookie stops working the moment the real user refreshes, and the replay is logged as a warning. I have to be honest about the other half: the access token is kept in localStorage in auth.service.ts under the key pms_access, so a successful XSS could read it. I bounded that rather than eliminated it: that token lives fifteen minutes and cannot produce new tokens, because the refresh endpoint ignores the header and only looks at the cookie. Our architecture note says the access token is kept in memory; the implementation uses localStorage so a page reload does not log the user out, and that is the one place where the document is ahead of the code.

POINT TO backend/src/main/java/com/pms/auth/controller/AuthController.java (addRefreshCookie) ·
frontend/pms-frontend/src/app/core/services/auth.service.ts ·
backend/src/main/resources/application.yml (pms.security.cookie-secure) ·
docs/ARCHITECTURE.md §5.3

THEN THEY ASK

You disabled CSRF protection, is the cookie not a risk? — The cookie is SameSite=Strict and scoped to one path, so a third-party site cannot make the browser send it, and even if the refresh call fired the attacker could not read the JSON response because of the CORS policy.

DO NOT SAY

Do not say "HttpOnly protects against CSRF". It protects against XSS reading the value; SameSite and CORS are what handle the cross-site case.

146

Why is hiding a button in Angular not enough to protect an operation?

EASY

SAY THIS

The Angular app runs on the user's own machine, so everything in it can be changed. Anyone can open the developer tools, edit the permission list in localStorage, or skip the interface completely and call the API with curl. The button is comfort; the real check is on the server.

IF THEY PUSH

My frontend does gate the interface: permissionGuard blocks a route when the route's required permission code is missing, and the sidebar only builds a menu item if auth.hasPermission returns true. But that whole decision is taken from a permission array kept in localStorage under pms_context, which the user can edit in two seconds. Nothing in the browser is a security boundary. That is why the same rule exists twice, and only the server copy counts: the service methods carry @PreAuthorize with the permission code, so the operation fails with 403 no matter how the request was built. I proved it in a test rather than claiming it: ProjectControllerTest has a case titled "RBAC dynamique : viewer (VIEW_PROJECT seul) peut lire mais pas créer" which signs in with a token holding only VIEW_PROJECT, gets 200 on the list and 403 on the create, with no interface involved at all. The frontend rule exists so a user is not shown a screen that would only fail, not to protect anything.

POINT TO frontend/pms-frontend/src/app/core/guards/permission.guard.ts · frontend/pms-frontend/src/app/core/services/auth.service.ts (hasPermission, localStorage pms_context) ·
backend/src/test/java/com/pms/project/ProjectControllerTest.java ·
backend/src/main/java/com/pms/project/service/ProjectService.java

THEN THEY ASK

So the frontend check is useless? — No, it is a usability rule: it keeps a user from opening a page they cannot use, but it is never the thing that stops an attacker.

DO NOT SAY

Do not describe the guard as a "first line of defence". It is not defence at all; calling it that suggests you think it contributes to security.

147

Where is authorization actually enforced in your application?

MEDIUM

SAY THIS

On the server, in four places. The filter chain requires a valid token, the JWT filter turns the user's permissions into Spring Security authorities, `@PreAuthorize` on each service method checks the permission code, and a scope interceptor checks the project is inside the user's perimeter.

IF THEY PUSH

First, `SecurityConfig` requires authentication for every request except login, refresh, the Swagger pages and the container health probes; anything else without a token is 401. Second, `JwtAuthenticationFilter` checks the signature and the expiry, checks the type claim is "access", checks the token version against the database, then loads the role's permissions and turns each code into a `SimpleGrantedAuthority`. Third, the operation itself: `@PreAuthorize("hasAuthority('CODE')")` appears 132 times across the modules, and I put it on the service method rather than the controller, so any route that reaches the operation is covered. Fourth, `ProjectScopeInterceptor`, registered in `WebMvcConfig` on `/api/projects/**`, checks the data perimeter before the controller runs. There is a fifth, narrower one: `FirstLoginFilter` blocks every endpoint except change-password, logout and refresh while the token says `firstLogin` is true, so the forced password change is a server rule and not a screen the user can skip. Everything on the Angular side, the `authGuard`, the `permissionGuard` and the dynamic menu, is interface behaviour only.

POINT TO `backend/src/main/java/com/pms/shared/config/SecurityConfig.java` ·
`backend/src/main/java/com/pms/auth/security/JwtAuthenticationFilter.java` ·
`backend/src/main/java/com/pms/auth/security/FirstLoginFilter.java` ·
`backend/src/main/java/com/pms/shared/config/ProjectScopeInterceptor.java` ·
`backend/src/main/java/com/pms/shared/config/WebMvcConfig.java`

THEN THEY ASK

Why put `@PreAuthorize` on the service and not on the controller? — Because a service can be called from more than one controller or from a scheduled job; putting the rule on the operation means it cannot be bypassed by a second entry point.

DO NOT SAY

Do not answer only "in the backend". Name the four places and the file each one lives in, otherwise the answer sounds memorised.

Why did you choose Role -> Permission -> Module instead of simply checking the user's role in the code?

HARD

CAREFUL — THE QUESTION IS NOT QUITE RIGHT

The chain that enforces anything is Role -> Permission. "Module" is a column on the permission row, not a class or a table, and it is used to group the admin screen by business area.

SAY THIS

Because a role check freezes the policy inside compiled code, so every change of who may do what becomes a release. The chain that actually does the work is Role to Permission: the code only ever names a permission code, and the link between a role and its permissions is a row in a table.

IF THEY PUSH

I have to correct one word in my own report. The part that carries the authorization is Role to Permission. "Module" is not a third class and not a third table: it is a module column on the permissions row, a label such as PROJET, EQUIPE or FACTURATION. It exists so the administration screen can group the permission checkboxes by business area, and it is what the ModuleGroup type in role-list.component.ts uses. Today there are 25 live permission codes spread over ten module labels. The reason for the intermediate level is the cost of change. If a service said `hasRole('CHEF_PROJET')`, then moving one right to another profile means editing Java, opening a pull request, running the pipeline and redeploying. Because the code says `hasAuthority('ASSIGN_DEVELOPER')`, the same move is one row in `role_permissions`. That is not theory: during the project the matrix moved twice. V12 deleted every grant for the four business roles and rebuilt them to match the audited target, and V13 added `VIEW_ALL_PROJECTS` to the Director. Not one line of Java changed in either migration. I also applied it to the scope rule itself, which is why ProjectScopeService tests for the `VIEW_ALL_PROJECTS` capability rather than for the name "DIRECTEUR".

POINT TO `backend/src/main/java/com/pms/user/entity/Permission.java` (module column) ·
`backend/src/main/java/com/pms/user/entity/Role.java` ·
`backend/src/main/resources/db/migration/V12__fix_rbac_matrix.sql` ·
`backend/src/main/resources/db/migration/V13__add_view_all_projects.sql` · `frontend/pms-`
`frontend/src/app/features/admin/roles/role-list.component.ts` (ModuleGroup)

THEN THEY ASK

Is there any role name left in your code? — The four names are referenced by the seed migrations and marked `is_system` so they cannot be renamed or deleted, but no service method and no controller ever branches on a role name.

DO NOT SAY

Do not agree that Module is an entity in the model just because the report draws it that way. Correct it in one calm sentence, then explain what the column is really for.

149

What would happen if tomorrow the company wanted the Director to receive a permission previously given only to the Project Manager? Would you need to modify the source code?

MEDIUM

SAY THIS

No source change, no rebuild, no restart. An administrator opens Administration, Roles and permissions, edits the DIRECTEUR role, ticks the permission and saves. That writes one row in role_permissions and the Director has the right on his next request.

IF THEY PUSH

The exact path is this. The screen is /admin/roles, RoleListComponent, reached from the sidebar item "Roles & permissions", which is itself gated by MANAGE_ROLES. Opening the DIRECTEUR row shows the full permission catalogue as checkboxes grouped by module; ticking one and saving sends PUT /api/admin/roles/{id} to RoleController, which calls RoleAdminService.update, guarded by @PreAuthorize("hasAuthority('MANAGE_ROLES')"). The only thing that changes in the database is the role_permissions join table: one new row holding the id of DIRECTEUR and the id of that permission. No migration, no redeploy. On timing, update() ends by clearing the securityContext cache, so the very next request from any signed-in Director rebuilds its authorities straight from the database and the @PreAuthorize passes immediately; he does not have to sign out. The Angular menu is the part that lags, because it uses the permission list stored at login, so a new menu entry appears after the next silent token refresh, at most fifteen minutes, or at once if he signs in again. One nuance worth stating: DIRECTEUR is flagged is_system, so it cannot be renamed or deleted, but its permission set stays fully editable, which is exactly the point of the design.

POINT TO frontend/pms-frontend/src/app/features/admin/roles/role-list.component.ts · frontend/pms-frontend/src/app/app.routes.ts (path 'admin/roles', permission MANAGE_ROLES) · backend/src/main/java/com/pms/user/controller/RoleController.java · backend/src/main/java/com/pms/user/service/RoleAdminService.java (update, evictSecurityContextCache) · backend/src/main/resources/db/migration/V25__rbac_admin_module.sql

THEN THEY ASK

And if the company wants a brand new role, not one of the four? — The same screen creates it: RoleAdminService.create inserts a non-system role with whatever permission set is chosen, and only assigning users to it is a separate step in the user screen.

DO NOT SAY

Do not say the change takes effect "on the next token". The backend picks it up on the next request because the cache is cleared; only the menu waits for the next token. Being precise here is the whole answer.

150

Suppose a Project Manager has the permission ASSIGN_DEVELOPER but tries to assign someone to a project

HARD

outside their scope. What prevents that?

SAY THIS

The second gate stops him. `ProjectScopeInterceptor` reads the project id out of the URL and calls `ProjectScopeService.assertCanAccess`, which throws `AccessDeniedException` and returns 403 because that project is not in his perimeter. The request never reaches the assignment service, so nothing is written.

IF THEY PUSH

The permission gate passes, because he really does hold `ASSIGN_DEVELOPER`, and that annotation sits on `TeamAssignmentService.assign`. The scope gate is separate. `ProjectScopeInterceptor` is registered in `WebMvcConfig` on `/api/projects/**`; its `preHandle` matches the pattern `^/api/projects/(\d+)/(.*)?$`, so a POST to `/api/projects/42/team` yields `projectId 42`, and it calls `ProjectScopeService.assertCanAccess(42, email)`. That method first lets through anyone holding the `VIEW_ALL_PROJECTS` capability, which is how the Director sees the whole portfolio; otherwise it calls `accessibleProjectIds`, backed by a single JPQL query, `findAccessibleProjectIdsByEmail`, returning the ids of active non-archived projects where the user is the `chefProjet` or has a non-deleted `TeamAssignment`. Project 42 is not in that set, so `AccessDeniedException` is thrown and Spring turns it into 403 "Projet hors périmètre". Because it is an interceptor and not a line inside each service, every route of the shape `/api/projects/{id}/...` is covered at once: `team`, `jalons`, `avenants`, `governance`, `missions`, `KPI` and `agile`. That was deliberate, since one forgotten predicate in one service would leak another manager's project. Two honest limits: the perimeter includes projects where the user is only a team member, so a manager who is a simple member elsewhere would pass the scope gate there; and endpoints that do not follow the `/api/projects/{id}` shape, such as `GET /api/users/{userId}/assignments`, are guarded by `VIEW_TEAM` only and are not covered by the interceptor. That one I would close by calling the same assertion inside the service.

POINT TO `backend/src/main/java/com/pms/shared/config/ProjectScopeInterceptor.java` (`preHandle`) ·
`backend/src/main/java/com/pms/project/service/ProjectScopeService.java` (`assertCanAccess`) ·
`backend/src/main/java/com/pms/project/repository/ProjectRepository.java` (`findAccessibleProjectIdsByEmail`) ·
`backend/src/main/java/com/pms/team/service/TeamAssignmentService.java` (`@PreAuthorize ASSIGN_DEVELOPER`) · `backend/src/test/java/com/pms/project/ProjectScopeServiceTest.java`

THEN THEY ASK

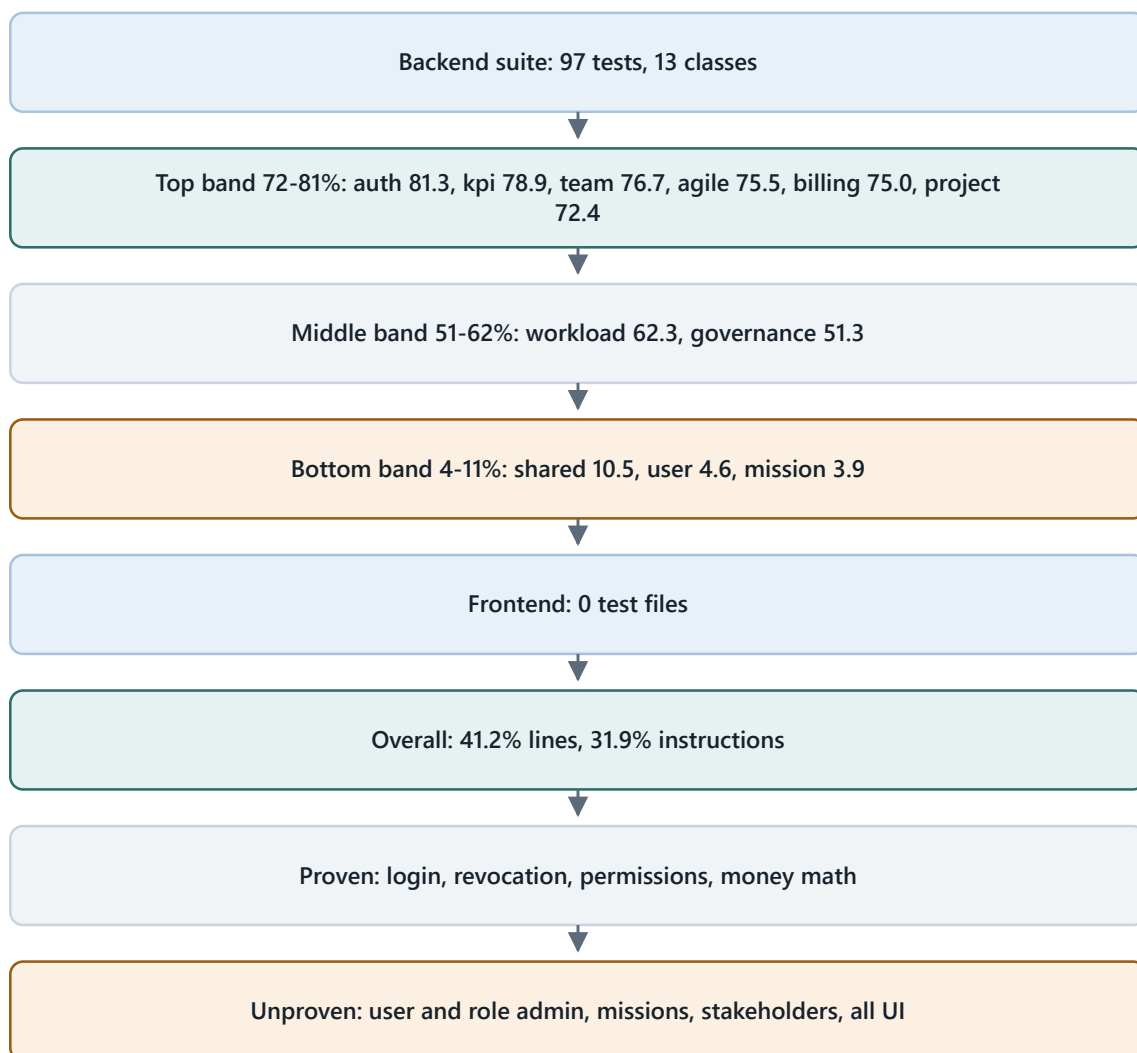
Which of the two gates fires first? — For a project URL the scope interceptor runs before the service check, because Spring runs interceptors before method security, so he gets the "hors périmètre" 403 first; conceptually the rule is still permission and scope together, and both must pass.

DO NOT SAY

Do not say "the service checks the project owner". It does not; the check lives in the interceptor and the scope service, and naming the wrong class is the worst possible slip on this question.

F. The ten that can hurt you most

These ten questions are not really about facts. They are about whether you made the decisions yourself. The examiner wants to hear a reason, a trade-off you weighed, and a weakness you name before he does. So for each one: give the decision in one sentence, give the reason in the next, then say out loud what it costs you. A candidate who says "here is the weak spot and here is how big it is" wins the room. A candidate who defends everything loses it.



The honest coverage map of PMS — Drawn in the room, this picture says the tests were aimed at money and permissions rather than at a percentage, and it names the two gaps before the examiner can.

SAY THIS

Because the business domain is one tight financial calculation. A single indicator read touches workload, cost rates, billing and the internal quote at the same moment, inside one transaction. Microservices would have turned that one read into four network calls and bought me nothing, so I built one deployable split into ten modules.

IF THEY PUSH

The decision is recorded as ADR-016 and it rests on three reasons. First, transactional consistency: my KpiService reads seven repositories in one read-only transaction to produce a margin. If those tables lived in four services I would need distributed transactions to get one correct number, which is a large cost for zero business gain. Second, scale: PMS serves one company internally, a handful of chefs de projet and one directeur, so the running cost of a distributed system would never be repaid by the load. Third, reversibility: I kept the option open instead of closing it. The ten feature packages under com.pms have a dependency graph that is deliberately acyclic and converges on the kpi module, which consumes the others and is consumed by none, so kpi or agile could be lifted out later without touching the rest. I should be honest about one thing: ADR-016 also says a module must never touch another module's repository, and my kpi module imports seven foreign repositories, so I break my own rule there. The boundaries are a convention in my code, not something the build enforces, and adding an ArchUnit test that fails the build on a cross-module repository import is on my list.

POINT TO DECISIONS.md – ADR-016 (Modular monolith, layered, feature-based) ·
backend/src/main/java/com/pms/kpi/service/KpiService.java · report chap_03.tex
§Architectural style – fig:three-tier, fig:pkg, fig:layers

THEN THEY ASK

So when would you split it? — When one module needs its own release rhythm or its own scaling, and the acyclic package graph plus the DTO-only API boundary mean kpi could be extracted without restructuring the other nine.

DO NOT SAY

Do not say microservices are bad or overrated in general; the examiner will read that as a candidate who chose the easy option and is defending it.

SAY THIS

Because the company's own documents disagreed on who may assign developers. If I had written the role name into the code, every change of responsibility would have needed a new release. So the role-to-permission table lives in the database, and every endpoint checks a permission code, never a role name.

IF THEY PUSH

This is ADR-001, and ADR-005 is the real story behind it. The Project Plan, the SRS and the Use Case Specification all say the Directeur cannot assign developers. The Cahier des Charges says the Directeur builds the team and, two lines later, that he cannot assign developers. The company then told me both the Directeur and the chef de projet must be able to do it, with the Directeur able to take it over later. Three documents, three answers. With hard-coded roles that argument costs a code change, a build and a deployment every time it is re-opened. With the model I built, it costs one row in the `role_permissions` table. Concretely: the Role entity holds a many-to-many set of Permission rows; a Permission is just a code, a module and a description; at login the permissions become Spring Security authorities and every protected method carries `@PreAuthorize("hasAuthority('...')")`. There are 24 distinct permission codes used across 22 classes in the backend, and the Angular menu and route guards read the same codes, so the UI follows the same data. The cost I accept is one extra join at login and an admin screen I had to build to edit roles and permissions. I also protected against the obvious accident: migration V25 flags the four built-in roles as `is_system`, so they cannot be renamed or deleted, and a role still held by users cannot be deleted at all.

POINT TO DECISIONS.md – ADR-001 and ADR-005 · `backend/src/main/java/com/pms/user/entity/Role.java` and `Permission.java` · `backend/src/main/resources/db/migration/V2__seed_rbac.sql`, `V25__rbac_admin_module.sql` · `backend/src/main/java/com/pms/user/service/RoleAdminService.java` · report chap_03.tex
§Cross-cutting design decisions – Dynamic authorization

THEN THEY ASK

What stops an administrator from locking everyone out? — The four business roles are flagged `is_system` and cannot be renamed or deleted, and `RoleAdminService` refuses to delete any role that is still assigned to at least one user.

DO NOT SAY

Do not present it as a generic good practice you read about; lead with the `ASSIGN_DEVELOPER` contradiction, because that is the proof you designed it for a real problem.

SAY THIS

Every user row carries an integer called `token_version`, and every access token carries a copy of it. On each request I compare the two. If they differ, the token is dead, so revoking a session is just adding one to that integer in the database.

IF THEY PUSH

Step by step. At login, `JwtService.generateAccessToken` signs a token holding the email as subject, the role, the permission codes, the current `tokenVersion`, `type="access"` and `firstLogin`; it lives 15 minutes in production. A second token, `type="refresh"`, holds the same version, lives 7 days, and goes back in an `HttpOnly`, `SameSite=Strict` cookie scoped to `/api/auth/refresh` — it is a signed JWT too, not an opaque string. On every request `JwtAuthenticationFilter` runs before Spring's own filter: it verifies the signature and expiry, checks type is "access", then reads the email and the version and builds the cache key `email + ":" + version`. It looks that key up in a Caffeine cache named `securityContext`, configured `maximumSize=2000`, `expireAfterWrite=5m`. On a miss it loads the user with `findActiveByEmailWithRole`; if the user is missing, inactive, or `user.getTokenVersion()` is not equal to the version in the token, the filter simply returns without setting an authentication, and the request ends as unauthenticated. The key detail is that the version is part of the cache key, so a revoked token can never ride on a fresh session's cache entry. Revocation itself is one call, `user.revokeAllTokens()`, plus evicting the old key. It fires on logout, on refresh (the refresh token is single-use — `AuthService` bumps the version, so replaying the old one logs "Refresh token replay detected" and returns 401 "Token révoqué"), on password change, and in `UserCrudService` on role change, account reset, deactivation and deletion. A permission change on a role is different: `RoleAdminService` does not bump versions, it clears the whole cache, so every user's next request misses, reloads the permission set from the database, and gets the new authorities at once — tokens stay valid, authorities are fresh. The honest limit: that cache is in-process, so this is a single-node design. With two backend instances, a revocation is instant on the node that handled it and the other node can still accept the old token until its 5-minute `expireAfterWrite` or its next miss. Moving the cache to Redis fixes it, and that is a configuration change, not a redesign.

POINT TO `backend/src/main/java/com/pms/auth/security/JwtAuthenticationFilter.java` ·
`backend/src/main/java/com/pms/auth/service/JwtService.java` and `AuthService.java` ·
`backend/src/main/java/com/pms/user/service/UserCrudService.java`, `RoleAdminService.java` ·
`backend/src/main/resources/application.yml` —
`spring.cache.caffeine.spec=maximumSize=2000,expireAfterWrite=5m` ·
`backend/src/test/java/com/pms/auth/AuthControllerTest.java` — logout revokes, rotation invalidates the original

THEN THEY ASK

Is that still stateless? — No, and deliberately not: authentication is stateless but authorization carries one cheap cache-backed lookup, which is exactly what lets me cut a session off immediately instead of waiting out the token's lifetime.

DO NOT SAY

Do not say you use a blacklist or a token store — you do not, and a wrong mechanism said confidently is the worst possible answer here.

154

What is the difference between a permission and a scope?

MEDIUM

SAY THIS

A permission answers "may you do this action at all". A scope answers "on which projects". Real authorization in PMS is the two together: permission AND scope, and a chef de projet holds the same VIEW_KPI permission as the directeur but sees far fewer rows.

IF THEY PUSH

The permission is a global capability: a code such as VIEW_KPI or MANAGE_BILLING, checked by @PreAuthorize on the service method. It says nothing about data. The scope is the row filter, and I derive it from capabilities and relationships, never from a role name, so it stays true to ADR-001. ProjectScopeService does three things: hasAllAccess() looks for the VIEW_ALL_PROJECTS authority in the current security context; accessibleProjectIds(email) runs one SQL query that returns the projects where the user is the chef de projet or an active team member; assertCanAccess(projectId, email) lets the call through if the user has global access or the project is in that set, and otherwise throws AccessDeniedException, which becomes a 403. I did not want that check scattered across thirty controllers, so I applied it in one place: ProjectScopeInterceptor matches the URL pattern `^/api/projects/(\d+)/(.*)?$` and runs the check before the controller. Every sub-resource inherits it — KPI, jalons, risques, charges, livrables. A worked example: the directeur and the chef de projet both hold VIEW_KPI, the same permission. The directeur also holds VIEW_ALL_PROJECTS so no filter is applied and he sees the whole portfolio; the chef de projet has no such capability, so the interceptor limits him to the projects he leads or is assigned to. Two honest bounds. The enforcement point is an HTTP interceptor on one URL shape, not a repository-level Specification as ADR-021 first sketched, so an endpoint that reached project data by a different URL would not be covered — PROJECT_STATUS.md itself marks that ADR as partly done. And the financial wall for developers (BR-050) is done by simply not granting them VIEW_KPI or VIEW_BILLING, which is a permission decision, not a scope one.

POINT TO backend/src/main/java/com/pms/project/service/ProjectScopeService.java ·
backend/src/main/java/com/pms/shared/config/ProjectScopeInterceptor.java ·
backend/src/test/java/com/pms/project/ProjectScopeServiceTest.java (6 tests) ·
DECISIONS.md — ADR-021 · PROJECT_STATUS.md — ADR-021 marked partial

THEN THEY ASK

Why not put the scope in the JWT? — Because a user's project assignments change during a session and a token cannot be updated; keeping the scope as a query means an assignment removed at 10:00 takes effect on the next request.

DO NOT SAY

Do not claim the scope is enforced at the repository layer by Specifications — it is enforced by an interceptor, and the examiner can open the file.

155

Why is Earned Value entered manually instead of calculated?

HARD

SAY THIS

Because earned value is a judgement, not a measurement. My database knows how many days were spent, not how much of the work is truly finished, and those are not the same thing. So the chef de projet types the percentage at the monthly review, and the system records exactly who said what and on which date.

IF THEY PUSH

Earned Value, the EV, is the share of the contract's value that the project has actually produced. The only automatic proxy I have is days consumed, and using it would mean assuming that spending equals earning — the classic mistake that makes a late project look healthy right up to the end. So the input is human and I made that explicit. SnapshotRequest is a record with evPct validated between 0 and 100, a dateFinEstimee and a faitsMarquants text; it is posted to /api/projects/{id}/kpi/snapshots behind the EDIT_PROJECT permission. KpiService.createSnapshot writes an immutable SnapshotKpi row, and only one per project per day — a second attempt on the same date is rejected. computeLive() takes the EV from the most recent snapshot, so the live dashboard shows the last judgement a human stated, never a figure the system invented. That EV then drives three numbers: CA production = budget TND × EV%, FAE (facture à établir) = CA production – total facturé, and marge actuelle = CA production – coût actuel. If no EV has ever been entered, the engine returns null for those three and pushes a warning string to the screen instead of guessing. I did not leave the manager unchecked, though. Right beside his EV the dashboard shows deliveryPct, which the system computes automatically as livrables livrés or validés divided by livrables planifiés, and tauxConsommation, the budget burn. If a manager claims 80% earned while 30% of deliverables are done and 90% of the budget is gone, the screen makes the contradiction visible. The weakness is real and I own it: a human number feeds a financial indicator, so it can be optimistic. It is bounded by validation, frozen in a dated immutable snapshot, and it builds a monthly curve that anyone can challenge afterwards.

POINT TO backend/src/main/java/com/pms/kpi/dto/SnapshotRequest.java ·
backend/src/main/java/com/pms/kpi/service/KpiService.java – computeLive, createSnapshot,
latestSnapshotEv · backend/src/main/java/com/pms/kpi/controller/KpiController.java ·
frontend/pms-frontend/src/app/core/models/kpi.model.ts · report chap_06.tex – "The earned
value is entered, not computed"

THEN THEY ASK

Could you not compute it from the deliverables? — I compute that separately as deliveryPct and show it next to the EV, but deliverables are counted not weighted, so a small document and a whole module would count the same; it is a cross-check, not a replacement.

DO NOT SAY

Do not apologise for it or call it a shortcut — it is the correct choice, and treating it as a gap invites the examiner to attack it as one.

156

How did you prove that your KPI engine gives the same results as Excel?

HARD

CAREFUL — THE QUESTION IS NOT QUITE RIGHT

The question assumes a proof exists. It does not: there is no automated comparison against workbook cells, only a manual side-by-side on one project plus unit tests on the formulas, and I should say that in my first sentence rather than let the word "proved" stand.

SAY THIS

I did not prove it formally, and I will not pretend otherwise. What I did was a manual side-by-side on one project: I entered it in PMS, read the indicators in the workbook and on the dashboard, and wherever the two differed I treated the workbook as right and fixed my engine.

IF THEY PUSH

Three things exist and none of them is a validation campaign, so let me name each one and what it is worth. First, a coverage argument. I classified all 2 088 formulas of the workbook into nine categories — 460 date functions, 92 SUM, 31 HLOOKUP, 36 IF/ISBLANK/IFERROR, 7 statistical, about 1 421 arithmetic cost formulas, 13 percentage formulas, 1 426 cross-sheet references, and 169 broken #REF! or #VALUE! cells that I deliberately do not reproduce. Every live formula maps to a named PMS calculation. That proves I left nothing out; it does not prove my numbers match. Second, unit tests on the formulas themselves. `DevisInterneServiceTest` asserts real arithmetic: 100 jours-homme sold at 5 000 in contract currency at a rate of 0.01 gives 5 000 TND, 80 internal days at 30 TND cost 2 400, so the margin is 2 600, which is 52%; add a 5% overhead line and the cost becomes 250 and the whole quote drops to 47%. `AvenantServiceTest` asserts that a budget révisé rises and falls correctly across two consecutive amendments. Those are formula-level checks, not workbook comparisons. Third, the Sprint 8 review, which is what the report records: one project entered on both sides, the indicators read twice, discrepancies resolved in favour of the workbook. One project, by hand, with no written trace of the deltas. What a real proof looks like, and I can say it precisely: take three finished ST2i projects, load their real plan de charge, charges réelles, TCC per year, jalons and avenants as a test fixture, and assert every indicator — coût actuel, ETC, EAC, marge, CA production, FAE, dérive — against the workbook cell to two decimal places, inside the test suite, so any regression fails the build. That is a golden-master test, it is roughly one day of work now that the data model exists, and it is the first thing I would add.

POINT TO docs/BUSINESS_ANALYSIS.md §10 — Excel formula to PMS calculation mapping, 2 088 formulas
• backend/src/test/java/com/pms/project/DevisInterneServiceTest.java
• backend/src/test/java/com/pms/billing/AvenantServiceTest.java
• backend/src/test/java/com/pms/kpi/KpiControllerTest.java (6 tests, structure and access, not values)
• report chap_06.tex §Sprint 8 — Sprint review

THEN THEY ASK

So the engine could be wrong today? — On the paths my unit tests cover it is right, and on one real project it matched the workbook after correction, but I cannot claim it for every project, and that is exactly why the golden-master test is first on my list.

DO NOT SAY

Never invent a validation campaign, a number of projects compared, or a percentage of agreement — a jury that catches one invented figure stops believing the other 150 pages.

157

Why are your governance tests only covering 51% of lines?

MEDIUM

SAY THIS

Because one of the four registers has no test at all. Parties prenantes — stakeholders — is plain CRUD with no business rule, so I spent the time on the three registers that have state machines. That single gap is 44% of everything the governance module misses.

IF THEY PUSH

The numbers first: governance has 413 instrumented lines and 212 are covered, which is the 51.3%. Now what is behind them. GovernanceControllerTest has 8 tests and they target every rule the workbook states but cannot enforce: risk creation and listing; the full livrable state machine EN_ATTENTE to EN_COURS to LIVRE to VALIDE, plus the illegal transition where validating a livrable that was never delivered returns 422; demande de changement created as EN_ATTENTE with no dateDecision, approval setting APPROUVE and stamping the date, and a second approval on an already-decided request returning 422. So the state rules are checked. What is not checked is the stakeholder register: PartiePrenanteService 0 of 32 lines, PartiePrenanteController 0 of 8, PartiePrenanteMapperImpl 1 of 48. That is 89 of the 201 missed lines. A further 88 missed lines are generated MapStruct mapper code — the reverse mapping directions that nothing in the application calls — which is code I did not write. Put together, roughly 89 lines are the untested register and 88 are generated code, which leaves about 25 lines of hand-written logic I chose not to test, mostly the update and delete paths of RiskService. So 51% does not mean half the business rules are unchecked. It means one CRUD register and a code generator are not exercised. One more test class of about eight tests closes it, and I would write it. It also matters that the one governance value the money depends on — the livrable statut, which feeds deliveryPct in the KPI engine — sits on the covered side.

POINT TO backend/target/site/jacoco/jacoco.csv — com.pms.governance rows ·
backend/src/test/java/com/pms/governance/GovernanceControllerTest.java (8 tests) ·
backend/src/main/java/com/pms/governance/service/PartiePrenanteService.java (0 lines covered) · report chap_07.tex \$Sprint 9 Testing — tab:coverage

THEN THEY ASK

Why did you leave stakeholders out rather than risks? — Because a stakeholder row has no invariant to break: it is a name, a role and contact details, while a risk and a change request carry statuses that must move in one direction only, and that is what a test can actually catch.

DO NOT SAY

Do not say the module is simple or low-risk in general; name the one untested register and the generated-mapper share, because vague reassurance reads as not knowing your own report.

158

Why is your workload module only at 62% line coverage?

MEDIUM

SAY THIS

Because the two halves are not tested equally, on purpose. The side where a developer records his own days is well covered, because that is where the security rule lives. The planning side, which only a validator can touch, sits at 46% of its lines.

IF THEY PUSH

Workload has 284 instrumented lines with 177 covered, and it splits cleanly in two.

ChargeReelleService, the saisie of actual days, reaches 50 of its 84 lines and has two test classes on it. ChargeReelleServiceTest has 7 tests, all on rule BR-033 and the H-8 fix: a developer may submit his own charge; he may not submit for somebody else, which throws AccessDeniedException; a validator may submit for a team member; a user who is not on the project team is rejected; a duplicate period is rejected; a validated charge can no longer be modified; a developer cannot modify another person's charge. WorkloadControllerTest adds 7 HTTP tests including 401 without a token, a duplicate month returning 409, validation returning 200, and re-validating an already-validated charge returning 422. PlanChargeService, the plan de charge, reaches only 28 of its 61 lines, and it is exercised only indirectly through three controller tests — create, duplicate, delete. Its update path, its canSeeAllWorkload filter and its team-membership check have no test of their own. On top of that, 34 of the 107 missed lines in the module are generated MapStruct mapper code, about a third. The reasoning behind the imbalance is deliberate: the dangerous action in this module is one person reading or writing another person's time, and that rule is tested seven different ways. Over-planning a month is visible on screen and reversible. The honest gap is PlanChargeService.update, which can rewrite a planned month and has no test at all; that is the first test I would add here.

POINT TO backend/target/site/jacoco/jacoco.csv – ChargeReelleService, PlanChargeService rows ·
backend/src/test/java/com/pms/workload/ChargeReelleServiceTest.java (7 tests) ·
backend/src/test/java/com/pms/workload/WorkloadControllerTest.java (7 tests) ·
backend/src/main/java/com/pms/workload/service/PlanChargeService.java · report
chap_07.tex §Sprint 9 Testing – tab:coverage

THEN THEY ASK

Does that gap affect the financial figures? — Yes, the plan de charge feeds ETC and therefore EAC, so an untested update path is a real risk; the saving grace is that the numbers it writes are visible on the dashboard immediately, but that is a mitigation and not a test.

DO NOT SAY

Do not claim the module is fully tested for security and stop there — say plainly that PlanChargeService.update has no test, because the examiner may well have opened the file.

159

What is the biggest limitation of your current PMS?

MEDIUM

SAY THIS

The biggest one is that nobody at ST2i has used it yet. It is built, containerized and deployable, and it passes its own tests, but it has never been through a real monthly review with real users. So I can say it works. I cannot yet say it fits.

IF THEY PUSH

Let me rank them honestly. First, no end-user evaluation. There is no evaluation chapter in my report because there was no evaluation, so everything I claim about usability is my own judgement and nothing more. Second, whole areas of the code have no test. The user module is at 4.6% of lines and the mission module at 3.9%, and the user one worries me more, because UserCrudService is 0 of 91 lines and RoleAdminService is 0 of 56 — those are exactly the classes that revoke sessions and rewrite the permission matrix. The admin half of my own security model is untested. The Angular frontend has zero test files. Third, this was a single-developer project: nobody reviewed my code, and the CI pipeline reviews the build, not the design. Fourth, module boundaries are a convention, not a rule — ADR-016 forbids cross-module repository access and my kpi module imports seven foreign repositories, and nothing in the build fails when a boundary is crossed. Fifth, the production bundle is 694.76 kB, which is 194.76 kB over the 500 kB Angular budget; it is a warning rather than an error so the build passes, but it is a real first-load cost on a slow connection. There is one more I should mention because it is functional: the indicator engine reads workload, cost rates, livrables, jalons and the devis interne, but it does not read mission costs, so travel spend sits outside the EAC today. None of these is a design flaw. They are unfinished work with a known size, and all of them are written down rather than hidden.

POINT TO backend/target/site/jacoco/jacoco.csv — user 4.6%, mission 3.9%, shared 10.5% · backend/src/main/java/com/pms/user/service/UserCrudService.java and RoleAdminService.java (0 lines covered) · backend/src/main/java/com/pms/kpi/service/KpiService.java — no mission repository · report chap_07.tex §Sprint 10 Testing — 694.76 kB, 194.76 kB over budget · DECISIONS.md — ADR-016 module boundary rules

THEN THEY ASK

Which of those would actually stop the company using it tomorrow? — None of them would stop it, but the untested user and role administration is the one I would not let a company run on without tests, because a bug there silently fails to revoke somebody's access.

DO NOT SAY

Do not give a soft answer like "the reporting export is missing" — a jury reads that as dodging, and you have real weak points that you can bound precisely.

SAY THIS

Three things in order. First, golden-master tests that compare the indicator engine against the real workbook on three finished projects. Second, tests on the administration half of the security model, which is at zero per cent today. Third, a real monthly review with the actual users, and change the interface from what they say rather than what I think.

IF THEY PUSH

The first week goes to the golden-master tests, because that is the question I currently cannot answer: load three finished ST2i projects with their real plan de charge, charges réelles, TCC par année, jalons and avenants as a fixture, and assert every indicator against the workbook cell to two decimals inside the test suite. It turns the workbook from something I remember into a regression net that fails the build. The second job is the untested half of security: UserCrudService and RoleAdminService are at zero lines covered, and every path in them that bumps token_version or clears the security cache needs a test, because a silent failure there means an access that was supposed to be revoked is not. That is the highest-risk untested code in the repository. Third, put it in front of two chefs de projet and the directeur for one real monthly review on live data; no amount of my own judgement replaces watching somebody try to close a month. After that come the two features the design already anticipates. The KIMAI import: ADR-003 designed the workload table with a source discriminator and an accepted flag precisely so a second source could arrive, the schema supports it, and only the import job is missing — it removes the monthly manual typing, which is the biggest remaining human cost. Then the export of the review document in the format ST2i already circulates, which removes the last manual step of the review. Two pieces of housekeeping alongside: lazy-load the Angular feature routes to get the bundle back under its 500 kB budget, and add an ArchUnit test that fails the build on a cross-module repository import so ADR-016 stops being a promise. One thing is not on the list: splitting into microservices. Nothing in the load or the team size justifies it, and the monolith was designed to stay reversible if that ever changes.

POINT TO backend/target/site/jacoco/jacoco.csv – UserCrudService and RoleAdminService, 0 lines covered · DECISIONS.md – ADR-003 (MANUAL | KIMAI source discriminator), ADR-016 · report conclusion.tex – perspectives: KIMAI import, reporting export, integration · report chap_07.tex §Sprint 10 Testing – bundle budget overrun

THEN THEY ASK

Why tests before features, when the company wants features? — Because the platform now holds the margin figures the company steers on; an undetected regression in the engine costs them more than a missing import does, and the import is only useful once they trust the numbers it feeds.

DO NOT SAY

Do not list five new features; a list that starts with tests and an evaluation shows engineering judgement, a list that starts with features shows a student who wants to build more.

The 12 questions that can really hurt you

These came out of a hard, unfriendly reading of your own answers. If you prepare nothing else, prepare these.

Your overall line coverage is 41 percent. Would you put this in production?

The examiner wants to see whether you can defend a number instead of hiding behind it. A candidate who says "coverage is not everything" and does not give the breakdown sounds like he is making excuses for a figure he could not raise.

ANSWER IT LIKE THIS

Forty-one percent of lines and twenty-four percent of branches, measured by JaCoCo over 161 classes on the run of 13 September. That total is honest, but it is not the useful number. The eight modules the suite aims at sit between 51 and 81 percent of lines, and the modules that carry money and permissions are at the top of that range: auth 81, kpi 79, team 77, agile 75, billing 75, project 72. What pulls the total down is shared.config, 1221 lines that are mostly the demonstration data seeder I used to make the screenshots in this report, plus two modules with no test class at all, user and mission. Ninety-seven tests in thirteen classes. I wrote them to cover two things: the authorization rule on every protected endpoint, and the business rules the workbook states but cannot enforce. I did not write them to reach a percentage. Before production I would add the user and mission test classes, take the seeder out of the measurement, and I would not ship the frontend without tests.

Your decision record ADR-015 says dashboards read snapshots, and that domain events start a KpiRecalculationService. Your defence says the indicators are computed again on every read. Which one is the truth?

This is a contradiction that is written down, between your own ADR and your code. If the examiner finds it before you admit it, every other decision record in the report becomes suspect.

ANSWER IT LIKE THIS

The code is the truth, and the ADR is out of date. ADR-015 was written in Phase 2, before the engine existed, and it describes a push design: events fire, a recalculation service writes snapshots, dashboards read only snapshots. That is not what I built. In the code, KpiService.computeLive is marked @Transactional(readOnly = true) and computes again from the sources on every read. createSnapshot freezes a month when you ask for it. There is no KpiRecalculationService, and there is not one ApplicationEventPublisher in the backend. I chose the read path on purpose, because a push model needs every write path in five modules to remember to fire a recompute. One forgotten call shows a wrong margin, and that is the one failure this platform must not have. What I did not do is go back and mark ADR-015 as revised, the way I marked ADR-014 superseded by ADR-026. That is a documentation defect, and it is mine. The rule I would apply from now on is that a decision record which no longer matches the code is a bug, and it gets closed like one.

You wrote that a module never touches another module's repository. Does your code respect that?

Twenty files in the repository break the rule. If you claim you follow it, even for one sentence, you hand the examiner proof that you either do not know your own code, or that you are willing to say more than is true.

ANSWER IT LIKE THIS

No, not fully, and here is the exact count. Twenty files import a repository that belongs to another module. Four of them are seeders in `shared.config`, so sixteen are production classes. Fifteen of those sixteen are the same weak pattern: a service brings in `ProjectRepository` to load the parent project and check it before attaching a child to it, and that is all it takes from the other module. The one strong violation is `KpiService`. It brings in eight repositories, seven of them from other modules, across five modules: workload twice, user twice, billing, governance and project. Against that, the whole backend has only two real service-to-service calls, `KpiService` to `DevisInterneService` and `ProjectService` to `JalonService`. So the direction is right. Nothing reaches into another module's controller or its DTOs, and no module writes through a repository that is not its own. But I read other modules' data directly instead of going through a service they publish. The fix is mechanical, about a day. Each source module publishes a read-only query service that returns a DTO, and `KpiService` holds six service references instead of six repositories. I would keep the rule and enforce it with an `ArchUnit` test from sprint one, instead of trusting myself to remember it in sprint eight.

You say the layering is Controller, Service, Repository, Entity, and that a controller never touches a repository. Is that enforced?

Nothing enforces it except discipline, and there is a real controller that breaks it. One grep by a curious examiner turns an architectural claim into a problem with your credibility.

ANSWER IT LIKE THIS

It is a convention, not a constraint, and there is one real breach. UserController imports RoleRepository, so that controller reaches the data layer directly. I will also be precise about a second one, because it looks like a breach and is not. ProjectController imports ProjectStatus, which is an enum that happens to live in the entity package and is used as the type of a request parameter. It is not an entity crossing the boundary. So across eighteen controllers: one genuine violation, one false alarm. The permissions sit on the services, not on the controllers, which is why the 110 @PreAuthorize annotations are spread over twenty-two files. What holds the layering today is the package layout and my own reading. What should hold it is a ten-line ArchUnit test class with four rules: controllers may not import repositories, no module may import another module's repository, entities may not appear in controller signatures, and no package cycles. It fails the build the day someone breaks one of them. That is the first thing I would add, and it is one file.

You describe a sprint review for each of your ten sprints. Show me the minutes.

The report shows ten reviews as if they were recorded events. If you defend them as documented ceremonies, you are defending something that does not exist, and the examiner only has to ask for one page.

ANSWER IT LIKE THIS

I cannot, because I never wrote minutes. What is in the report is written again from dated working records of a practice that really happened, and I would rather call it a reconstruction than dress it up. The records that do exist, and that cannot be back-dated, are these.

PROJECT_STATUS.md, updated at the end of each phase with what was delivered and the test count at that moment. DECISIONS.md, twenty-seven architecture decision records with context, decision, reason and consequence, some of them dated, ADR-027 dated 22 August. PROJECT_TODO.md, with the validation gates for each task. And the twenty-eight Flyway migrations, V1 to V28, numbered in build order, which cannot be renumbered once they have run against a database. That numbering is the real trace: V1 auth, V3 user and resource, V5 project, V6 team, V7 workload, V8 kpi, V9 billing, V10 missions, V11 governance. I will not offer you the git history, because the repository was started partway through the internship and its dates do not map to the sprint calendar. One dated half page per review would have cost twenty minutes a sprint, and it would have made this question unnecessary. That is the process change I would make first.

Did anyone at ST2i actually use this? Where is your user evaluation?

There is no usability study and no trial with real end users. So any answer that hints at user validation can be destroyed by the industrial supervisor sitting in the room.

ANSWER IT LIKE THIS

Nobody outside the project used it, and I ran no usability study. There is no questionnaire, no measurement of task completion and no session with an end user. I will not call the acceptance I did have user validation, because it was not that. What I had was a different tool, and a stricter one for the part it covers. Every increment was checked against the reference workbook. That means I reproduced on the platform a computation the file already did, then compared the two figures, and treated the workbook as correct when they did not match. That proves the numbers down to the decimal, and it proves nothing at all about whether a project manager finds a screen comfortable. Those are two different validations, and I only ran one. If ST2i wants the second one, it is cheap and I can describe it exactly: three project managers, one closed project each, five timed tasks, entering a monthly workload, adding a billing milestone, reading the margin, and then the System Usability Scale. Two days of work. What I would not do is claim that figure now, because there is no measurement behind it.

Is the platform in production at ST2i? What would the company actually lose if it disappeared tomorrow?

The industrial supervisor can contradict any claim of production use in one sentence. An answer that is bigger than the truth here spoils the way the jury reads everything else you say.

ANSWER IT LIKE THIS

It is not in production, and no real ST2i data has been moved into it. What exists is a three-container stack that starts from an empty database and fills it with made-up demonstration data. I checked that it runs end to end both on my machine and on a clean GitHub Actions runner. The internal quote module ships as an empty structure on purpose, because I was allowed to reproduce the company's structure and never its figures, and that is written down as a decision, not left as an omission. So if it disappeared tomorrow, ST2i would carry on with the workbook exactly as it does today. What it would lose is the analysis: two thousand and eighty-eight formulas made explicit, including the one hundred and sixty-nine cells that already return errors in the copy the company uses to decide whether a project is profitable. It would also lose a running implementation where the margin is no longer visible to anyone who opens a tab. Turning that into a real gain is one rollout decision: load the real cost rates, enter one finished project, and compare the computed margin against the workbook's figure. That single reconciliation is what would earn the company's trust, and it has not been done.

Your production build shows a budget warning. You shipped a bundle that is over its own limit. Why?

It is a defect that anyone can see in your own screenshot. A defensive answer makes a 195 kB bundle look like something you tried to hide, rather than something you measured.

ANSWER IT LIKE THIS

Yes. The initial bundle is 694.76 kB, which is 194.76 kB over my 500 kB warning budget. The error threshold is set at 1 MB, so the build passes and the pipeline is green. I want to be clear that the build is warning me, not failing. I put that screenshot in the report instead of raising the budget to make the warning go away, because moving the line to fit the number is the wrong instinct. The cause is that the application is loaded eagerly. The routes are not lazy, so all the feature modules land in the initial chunk. The fix is known, and it is not research. It is `loadComponent` on the feature routes, so that billing, governance and the agile board are fetched only when you open them. For an application of this shape, that usually cuts the initial chunk by half. I did not do it because sprint 10 was containerisation and CI, and I chose to spend the last weeks making the stack reproducible rather than making it smaller. For twenty to thirty internal users on a company network, that was the right order. For a public application it would not be.

In your class diagram, a backlog item belongs to a project and to a sprint. Who really owns it? Your own supervisor questioned this.

If you get this wrong, or sound unsure, on a relation your supervisor has already flagged, it reads as a modelling error you never resolved. It is the one point in the diagram they know they can press.

ANSWER IT LIKE THIS

The project owns it. The sprint only borrows it. In the code, BacklogItem has project_id declared nullable = false and sprint_id nullable, and Sprint has project_id nullable = false. That is the whole answer. Project to BacklogItem is a composition, zero-or-many, because an item cannot exist without its project and dies with it. Sprint to BacklogItem is an aggregation, zero-or-one to zero-or-many, because an item with a null sprint is a valid and common state: it is an item still in the product backlog, not yet committed to an iteration. The behaviour follows, and it is what you would expect. Deleting a sprint sends its items back to the backlog instead of destroying them, and that note is printed on the detailed diagram on page 94. I should flag one imprecision before you find it. The global domain diagram on page 33 draws those child ends as 1.. *where they should be 0..*, because a new project can quite legitimately have no sprint and an empty backlog. Page 94 is the accurate figure. And the link goes one way only, from the child up. Project does not hold a list of sprints, because a collection in both directions would load a list with no limit that I never need.

You worked alone. Nobody reviewed your code. How do you know any of it is correct?

The honest answer is that no other developer read it. So any confident "the code is clean" invites the examiner to open a random file and test the claim in front of the room.

ANSWER IT LIKE THIS

No other developer read this code, and I will not claim the quality that a review would have given me. What I could do was replace the reviewer with tools that never get tired. There were three of them. First, an outside test of correctness that I could not argue with: every financial increment was checked against the reference workbook, so the judge was the company's own file and not my opinion. Second, the compiler and the schema: 52 DTOs as immutable Java records, MapStruct building the mappings at compile time so a mapping mistake turns the build red, and twenty-eight Flyway migrations so the schema is owned by versioned files and not by memory. Third, a pipeline that runs five jobs on every push. One of them brings the whole three-container stack up on a clean runner, waits for the API to be healthy, logs in through the nginx reverse proxy and checks the Angular routing fallback. That job caught a real defect: a production bundle that still pointed at the development API address. What none of that replaces is a second pair of eyes on design. An audit I ran on myself found the administrator holding twenty-two permissions instead of five, and migration V12 rebuilt the matrix. A reviewer would have caught that in March, not in June.

Your repository contains demonstration accounts with their passwords written down, and you say they skip the forced password change. Is that not exactly the kind of hole you built this platform to close?

It is a real weakness that anyone can check, in a project whose main selling point is access control. If you answer in a panic, the whole security chapter falls apart.

ANSWER IT LIKE THIS

It is, and the difference is which database they live in. Those accounts are created by seeders in `shared.config`. They exist only in the demonstration dataset, and they exist so that the defence video shows real screens instead of empty ones. They are set with `firstLogin` false on purpose, so a recording does not open on a password-change form. None of them is a real ST2i identity, and none of the data behind them is real. The control is in the deployment documentation. Seeding sits behind a flag, `pms.demo.seed-users`, and the first documented step of a real installation is to set it to false and create the administrator by hand. The real secrets are not in the repository at all. The database password and the JWT signing key (the key used to sign the ticket that proves who you are) come from the environment in `docker-compose`, and the pipeline makes throwaway values for its own run. What I would tighten is the default. Today the demonstration seeding is on unless you turn it off. For anything that leaves my machine it should be off unless you turn it on. That is one line, and it is the correct default.

Show me a place where your authorization model does not cover the data.

The examiner is inviting you to do one of two things: bluff a perfect model, which is never true, or find a hole live in front of the room. The only way to stay in control is to offer one yourself, precisely, with its limits.

ANSWER IT LIKE THIS

There is one, and I can name it exactly. My scope is enforced in two places. First, a `@PreAuthorize` on the service method, which decides whether you hold the permission at all. Second, a `ProjectScopeInterceptor`, which matches the URL pattern `/api/projects/{id}` and anything under it, and throws `AccessDeniedException`, giving a 403, if the project is outside your perimeter. Almost every business endpoint lives under that prefix, which is why the interceptor covers them. The exception is `GET /api/users/{userId}/assignments` in `TeamController`. It is not under `/api/projects`, so the interceptor does not fire on it, and the service guards it only with `VIEW_TEAM`. So a user who holds `VIEW_TEAM` can list another user's assignments, including project codes for projects outside their own perimeter. It exposes assignment rows, not budgets, margins or cost rates, so the rule that matters most, that a developer never sees financial data, is not affected. The fix is two lines: either limit the result to the projects the caller can already access, or require the caller to be the user in question. What I would prefer is for the URL-prefix convention itself to be the thing the build checks, so that a new endpoint outside it cannot be merged without an explicit scope call.

Answers that needed to be better

Places where the first answer was unclear, avoided the question, or claimed more than the code actually shows.



Architecture pack — "What are the layers inside a module, and what stops a developer from jumping over them?"

WHAT WAS WEAK

The prepared answer says "one imports a repository and one imports an entity, and both are in the project module". Both halves are wrong. The repository import is in UserController, which is the user module, not project. And the ProjectController import is ProjectStatus, an enum in the entity package used as the type of a request parameter, not an entity crossing the boundary. Getting your own confession wrong is worse than the violation itself.

SAY THIS INSTEAD

Say: "One genuine breach, UserController imports RoleRepository. And a second one that looks like a breach and is not: ProjectController imports ProjectStatus, an enum that lives in the entity package and is used as a request parameter." Then give the ArchUnit fix.



Context pack — "Who are the users of this platform?"

WHAT WAS WEAK

It claims "the scope filter is applied at the repository boundary rather than per controller". The repository does hold the JPQL query findAccessibleProjectIdsByEmail, but the thing that enforces the rule is a Spring HandlerInterceptor, ProjectScopeInterceptor, matching the URL pattern `^/api/projects/(\d+)/(.*)?$`. That is the HTTP boundary, not the data layer. And that difference is exactly what leaves GET `/api/users/{userId}/assignments` unscoped.

SAY THIS INSTEAD

Say: "The permission is checked on the service method. The perimeter is checked once, in one central place, by an interceptor on every URL under `/api/projects/{id}`, which is where nearly all the business data lives. There is one endpoint outside that prefix, and I can tell you which one."



Architecture pack — "Figure 3.2 shows your module dependencies. Why does everything point towards the kpi module?" and "Why is KpiSnapshot in the class diagram at all?"

WHAT WAS WEAK

Both answers state recompute-on-read as the design, and neither mentions that ADR-015, which is in the repository, says the opposite: dashboards read snapshots, and domain events start a KpiRecalculationService that does not exist. An examiner who reads DECISIONS.md finds a contradiction you did not flag.

SAY THIS INSTEAD

Add one sentence at the start: "ADR-015 describes the push design I planned in Phase 2 and did not build. The code computes on read and has no events, and I failed to mark that ADR as revised, the way I marked ADR-014 superseded." Then give the reason the read path won.



Architecture pack — "You wrote that a module must never touch another module's repository. Does your code respect that?"

WHAT WAS WEAK

The numbers are close but not right. It says three seeders and seventeen production classes, and that KpiService injects seven repositories. The repository shows four seeders in shared.config (DataInitializer, DemoDataSeeder, EnterpriseDataSeeder, AgileDemoSeeder) out of twenty files, so sixteen production classes. And KpiService injects eight repositories, of which seven belong to other modules.

SAY THIS INSTEAD

Use these: twenty files, four seeders, sixteen production classes. KpiService injects eight repositories, seven of them from other modules, across five modules. Precise numbers are the whole value of this answer.



Scrum pack — "Can you show me the minutes of those reviews?"

WHAT WAS WEAK

"Reconstructed from my own dated working records" is vague, and being vague on a question about evidence reads as avoiding it. It also never says out loud that some of the artefacts you name are not themselves dated sprint by sprint.

SAY THIS INSTEAD

Name the four artefacts and say what each one proves: PROJECT_STATUS.md end-of-phase entries, DECISIONS.md with ADR-027 dated 22 August, PROJECT_TODO.md validation gates, and the V1–V28 migration numbering, which is the only one that cannot be back-dated. Then say it plainly: no minutes exist.



Security pack — "How are the passwords stored?"

WHAT WAS WEAK

The answer covers BCrypt 12, the dummy-hash timing equalisation and the 5-in-15-minutes logout, and all of that checks out. But it says nothing about the seeded demonstration accounts, whose passwords are published in docs/demo-accounts.md with firstLogin set to false. An examiner who has opened the repository will read that silence as hiding something.

SAY THIS INSTEAD

Offer it yourself at the end: they are demonstration identities only, behind the pms.demo.seed-users flag, documented as the first thing to turn off in a real installation. And add that the flag should default to off, not on.



Context pack — "If PMS disappeared tomorrow, what would ST2i actually lose?" (its closing line)

WHAT WAS WEAK

It ends with "the platform already reproduces that computation, which is how each increment was accepted during the project". That mixes up two different things. You did check your work against a template workbook during development, but no real closed ST2i project has been checked end to end. Said that way, it sounds as if the rollout step has more or less been done already.

SAY THIS INSTEAD

End instead with: "The engine reproduces the workbook's method. It has never been checked against a real closed ST2i project, and that check is the first thing I would run."



Scrum pack — "Why did you choose Scrum for a project where you were the only developer?" (the follow-up about waterfall)

WHAT WAS WEAK

It offers migrations V12 and V17 as proof that working in iterations forced you to adapt, which is fair. But the wider pack leans again and again on the workbook as an "external acceptance criterion", and never admits that you also chose how to read it. Using that phrase over and over with nothing added invites the question you cannot win.

SAY THIS INSTEAD

Admit the limit once, early: "It is external in the sense that the formulas were not mine to negotiate. How to read an ambiguous rule was still my choice, and that is the risk that stays in a one-person Scrum." Said once, it takes the whole line of attack away.